Tell us about your PDF experience.

Track changes with Git

Share your code from within Visual Studio by using source control technologies such as Git and GitHub.

Get started

OVERVIEW

Git & Visual Studio for source control

HOW-TO GUIDE

Clone an existing repo

Create a new repo

REFERENCE

Compare Git & Team Explorer

Learn about Git, GitHub, & Visual Studio

Innerloop dev workflow

HOW-TO GUIDE

Create a new branch

Make a commit

Stage lines of code

Push to remote

Fetch, pull, and sync

Repo management

HOW-TO GUIDE

Browse repos & compare branches

Manage repos

Work with multiple repos

Resolve merge conflicts

Git settings & preferences

How Visual Studio makes version control easy with Git

Article • 05/17/2023

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

Have you ever wished you could go back to a previously working version of your code? Do you find yourself manually storing copies of your code in different locations as a backup? Well, version control is the answer.

Git is the most widely used modern version control system. With Git, you can track the code changes you make over time and you can revert to specific versions. So whether you're a professional developer or if you're learning how to code, Visual Studio's Git experience can be very useful to you.

> **Tip**
>
> To learn about using Git and GitHub in Visual Studio, sign up for the **Git learning series**.

Start with Git & GitHub in Visual Studio

Version control with Visual Studio is easy with Git. We meet you where you are. You can work remotely with the Git provider of your choice, such as GitHub or Azure DevOps. Or, you can work locally with no provider at all.

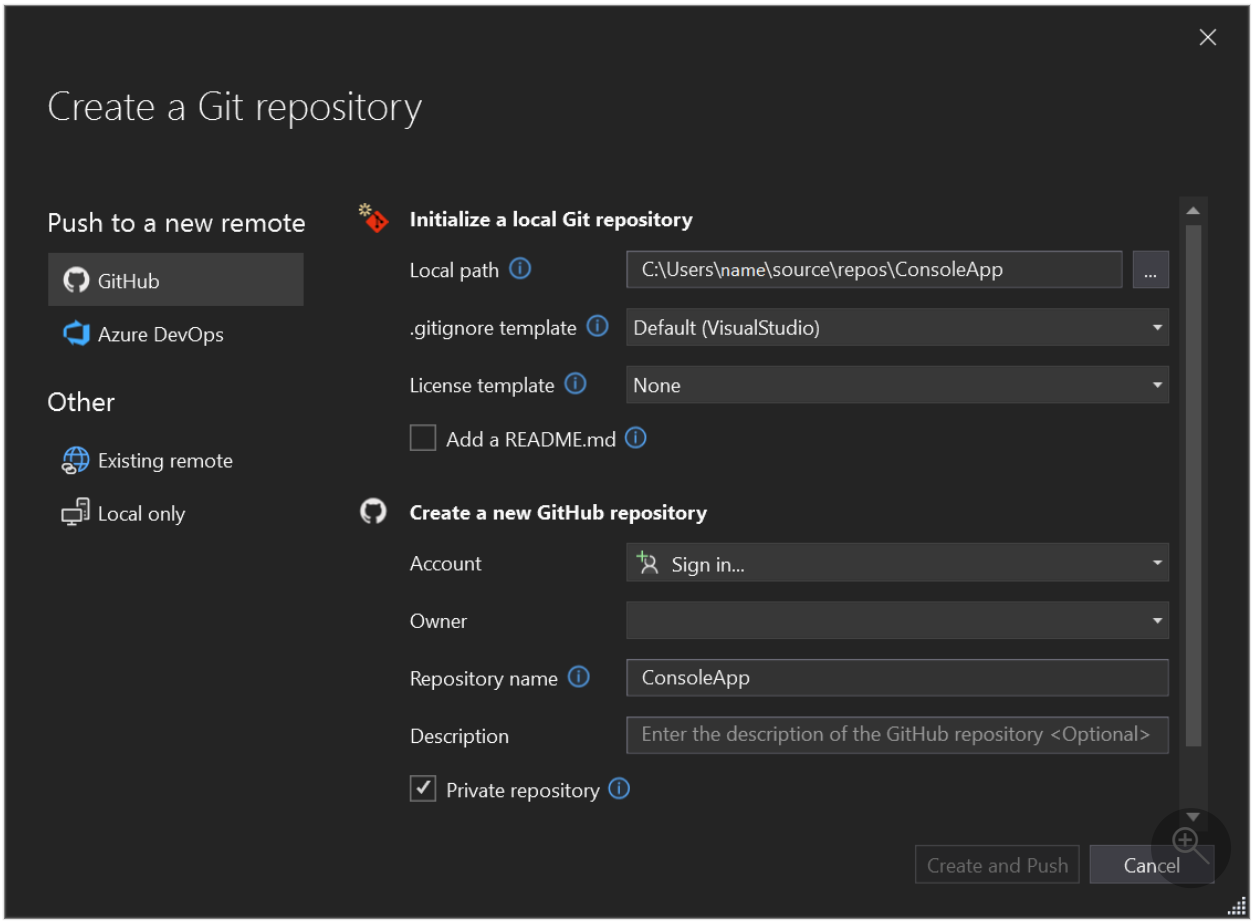
To get started using Git with Visual Studio:

- If you have a Git repo hosted on a Git provider like GitHub, clone the repository to your local machine.
- Otherwise, easily create a new Git repository and add your code. If you don't have a Git provider yet, we recommend you start with GitHub since the Git experience in Visual Studio is optimized for this provider. GitHub offers free and secured cloud code storage where you can store your code and access it from any device, anywhere.

Not only can you add both GitHub and GitHub Enterprise accounts to your keychain, but you can also leverage them just as you do with Microsoft accounts. If you don't have

a GitHub account, follow these steps to create a GitHub account to use with Visual Studio now.

If you're new to Git, the https://git-scm.com/ website is a good place to start.



View files in Solution Explorer

When you clone a repository or open a local repository, Visual Studio switches to the Git context. Solution Explorer loads the folder at the root of the Git repository, and scans the directory tree for any viewable files like `CMakeLists.txt` or those with the `.sln` file extension. For more information, see View files in Solution Explorer.

Intuitive inner-loop workflow

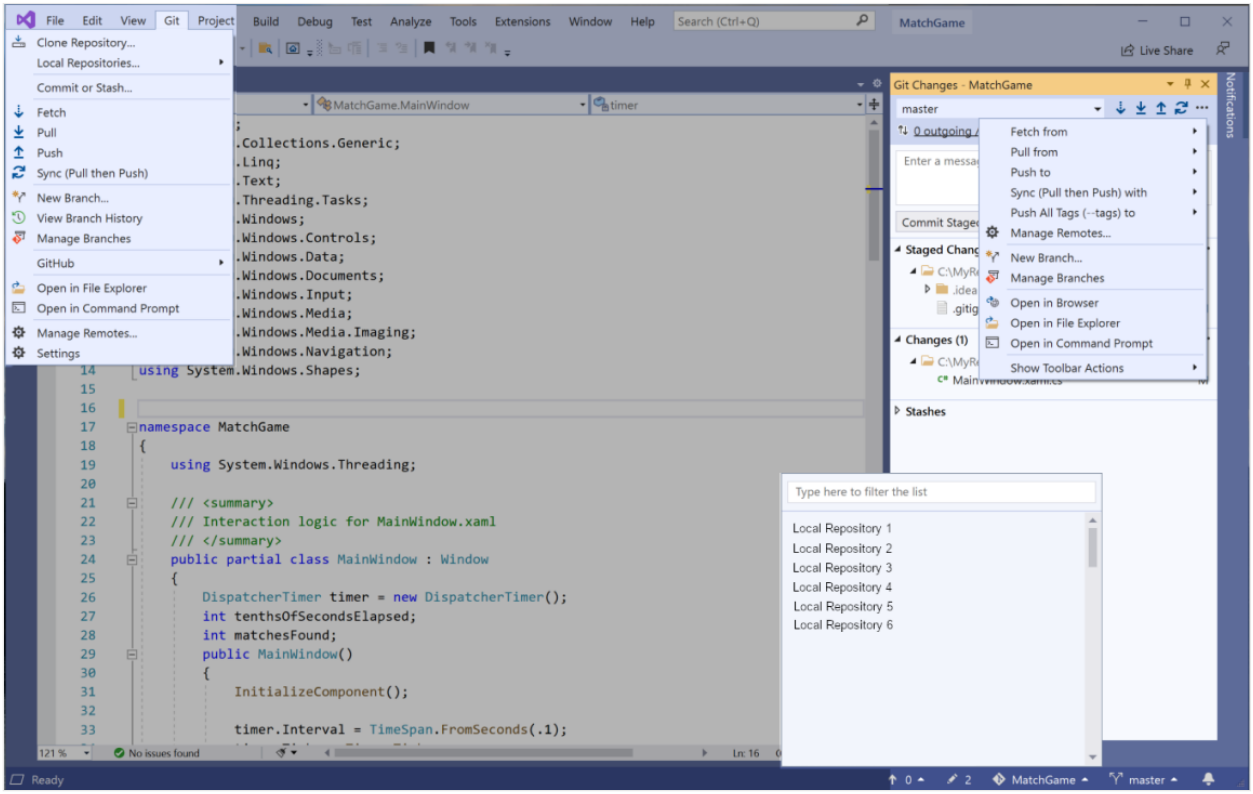
For your day-to-day Git workflow, Visual Studio provides a seamless way to interact with Git while coding without having you switch away from your code.

You are empowered to multi-task and experiment with your code through branches. If you or your team works on multiple features at the same time, or if you'd like to explore ideas without affecting your working code, branching is very helpful. The recommended Git workflow uses a new branch for every feature or fix that you work on. Learn how to create a branch from Visual Studio.

Once you create a new branch and switch to it, you can start working by changing existing files or by adding new ones and then committing your work to the repository. To learn more about making a commit in Visual Studio and to better understand file states in Git, refer to the Make a commit page.

Git is a distributed version control system, meaning that all the changes made so far are local only changes. To contribute these changes to a remote repository, you must push those local commit(s) to a remote.

If you are working in a team or if you are using different machines, you will also need to continually fetch and pull new changes on the remote repository. To learn more about managing Git network operations in Visual Studio, refer to the Fetch, pull, push, and sync page.

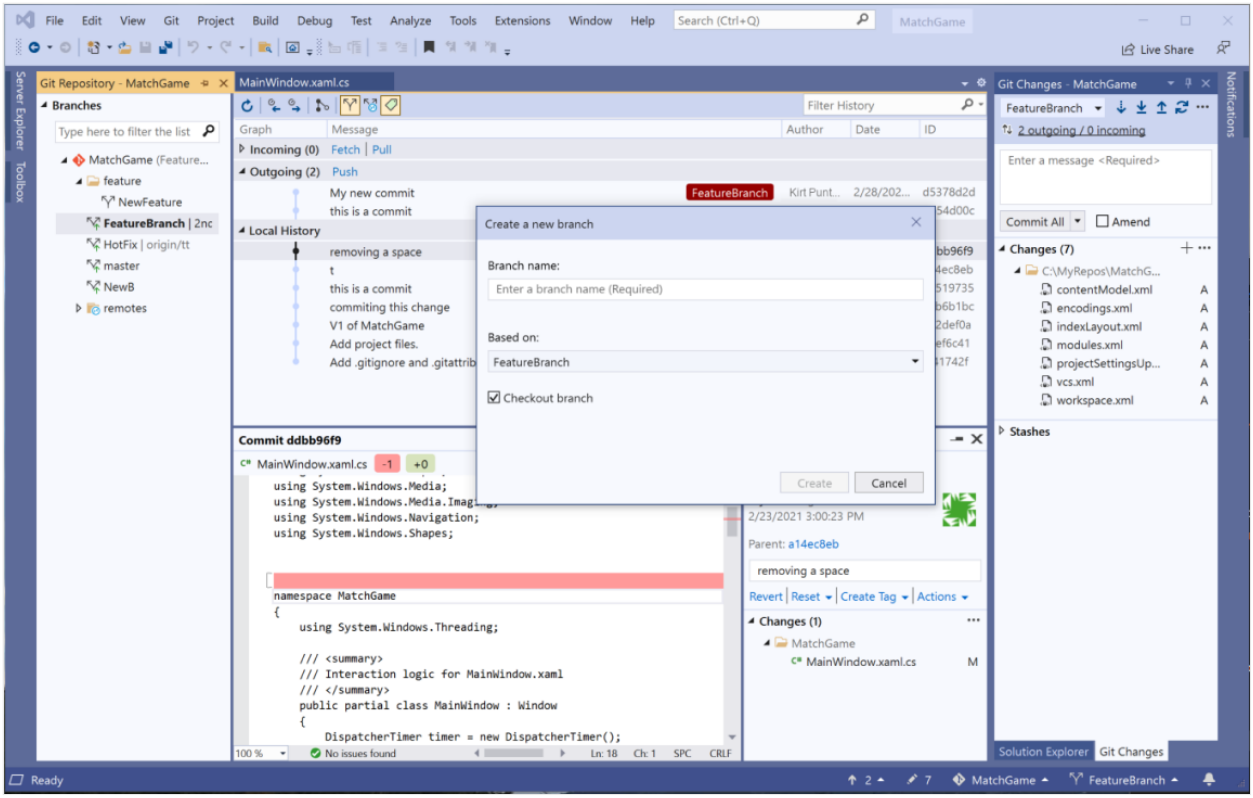


Repository management & collaboration

However, there are times when it makes more sense to focus on your Git repository. For example, you might need to get a good picture of what your team has been working on, or copy a commit from a different branch, or just clean-up your outgoing commits. Visual Studio includes powerful repository browsing and collaboration features that eliminate the need to use other tools.

To help you focus on your Git repository, Visual Studio has a **Git Repository** window, which is a consolidated view of all the details in your repository, including local and

remote branches and commit history. You can access this window directly from either **Git** or **View** on the menu bar or from the status bar.



Browse and manage Git repositories

To learn more about how you can use the Git Repository window in Visual Studio to browse and manage your Git repository, refer to the following pages:

- Browse a repo
- Manage a repo

Handle merge conflicts

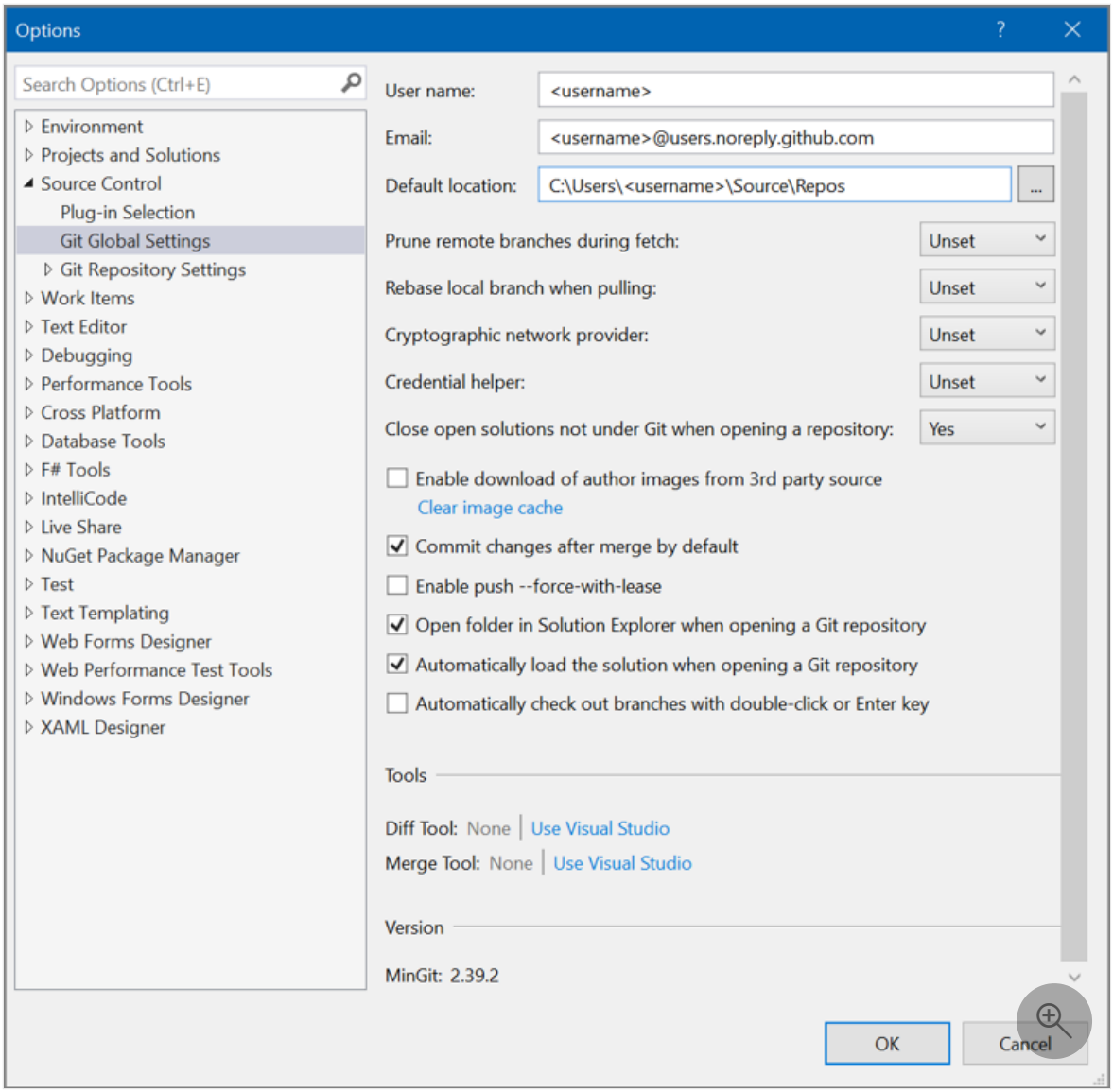
Conflicts can occur during a merge if two developers modify the same lines in a file and Git doesn't automatically know which one is correct. Git halts the merge and informs you that you are in a conflicted state. Learn more in the Resolve merge conflicts page.

Personalize your Git settings

To personalize your Git settings at a repository level as well as at a global level:

1. Go to either **Git** > **Settings** on the menu bar or **Tools** > **Options** > **Source Control** > **Git Global Settings**

2. Choose the options you want.



Performance & productivity enhancements

When it comes to the integrated Git tooling within Visual Studio, we continue to iterate on performance and productivity enhancements. Here are a few notable updates:

- **Commit graph**. In Visual Studio 2022 version 17.2, we introduced a commit graph feature to help reduce the delays you might experience when you're viewing your Git repository or branch history. For more information, see the Supercharge your Git experience blog post.
- **Branch switching**. Also in Visual Studio 2022 version 17.2, we reduced the number of branch switches that need solution reloads by an estimate of 80%. To learn more, see the Performance Enhancements: Git Branch Switching blog post.

There are even more feature updates in Visual Studio 2022 **version 17.6**, to include the ability to search for and link to GitHub Issues and Azure DevOps work items, amongst a few. To get all the details, see the **Git Features to Enhance Productivity** blog post.

Clone a Git repository in Visual Studio

Article • 08/02/2023

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

Visual Studio makes it easy to clone a repository right from the IDE. You can work remotely with the Git provider of your choice, such as GitHub or Azure DevOps. If you need to create a new repository instead of cloning an existing one, see Create a repository from Visual Studio.

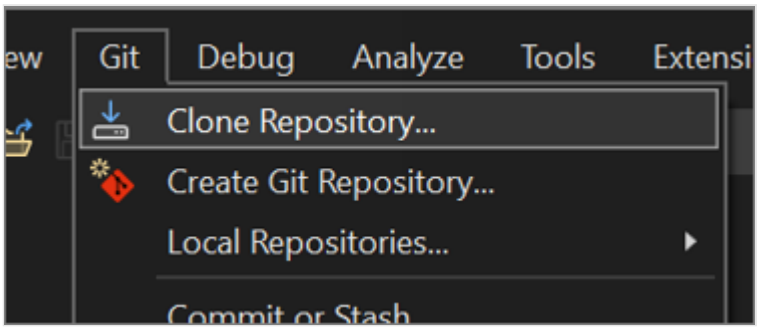
Prerequisites

To follow this article, you'll need:

- Visual Studio installed
- A GitHub user account

Clone a GitHub repo and sign in

1. Open Visual Studio.
2. From the **Git** menu, select **Clone Repository**.

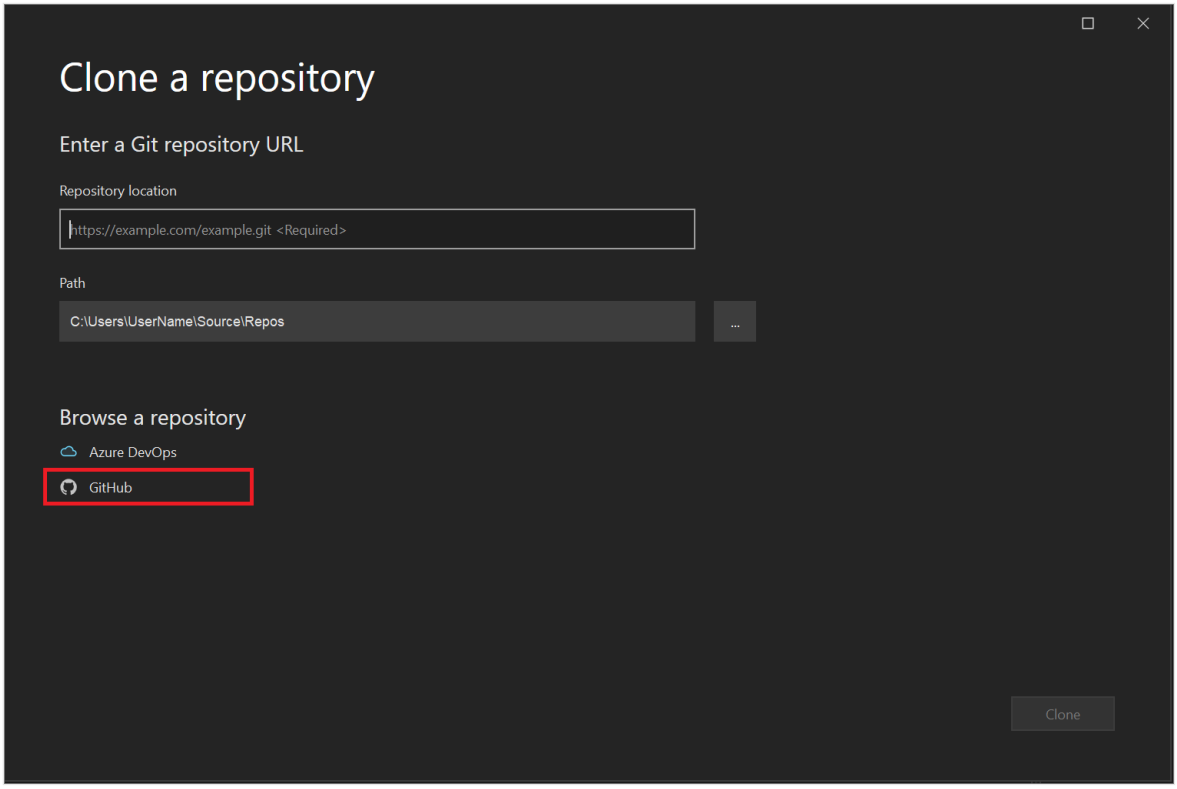


> **Note**
>
> If you haven't interacted with the **Git** menu before, you might see **Clone** instead of **Clone Repository**. If so, select **Clone**.
>
> And, if **Git** isn't on the menu bar, go to **Tools** > **Options** > **Source Control** > **Plug-in Selection**, and then select **Git** from the **Current source control plug-in** dropdown list.

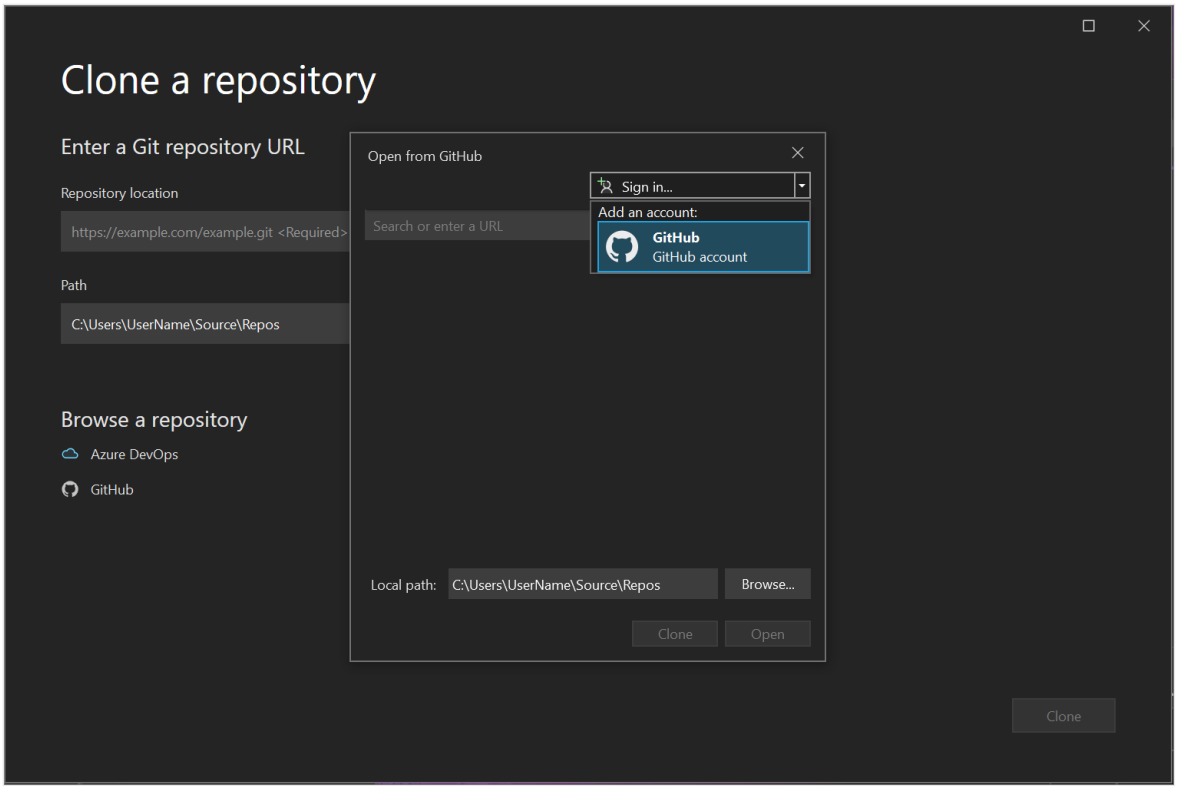
3. In the **Clone a repository** window, under the **Enter a Git repository URL** section, add your repo info in the **Repository location** box.

   Next, in the **Path** section, you can choose to accept the default path to your local source files, or you can browse to a different location.

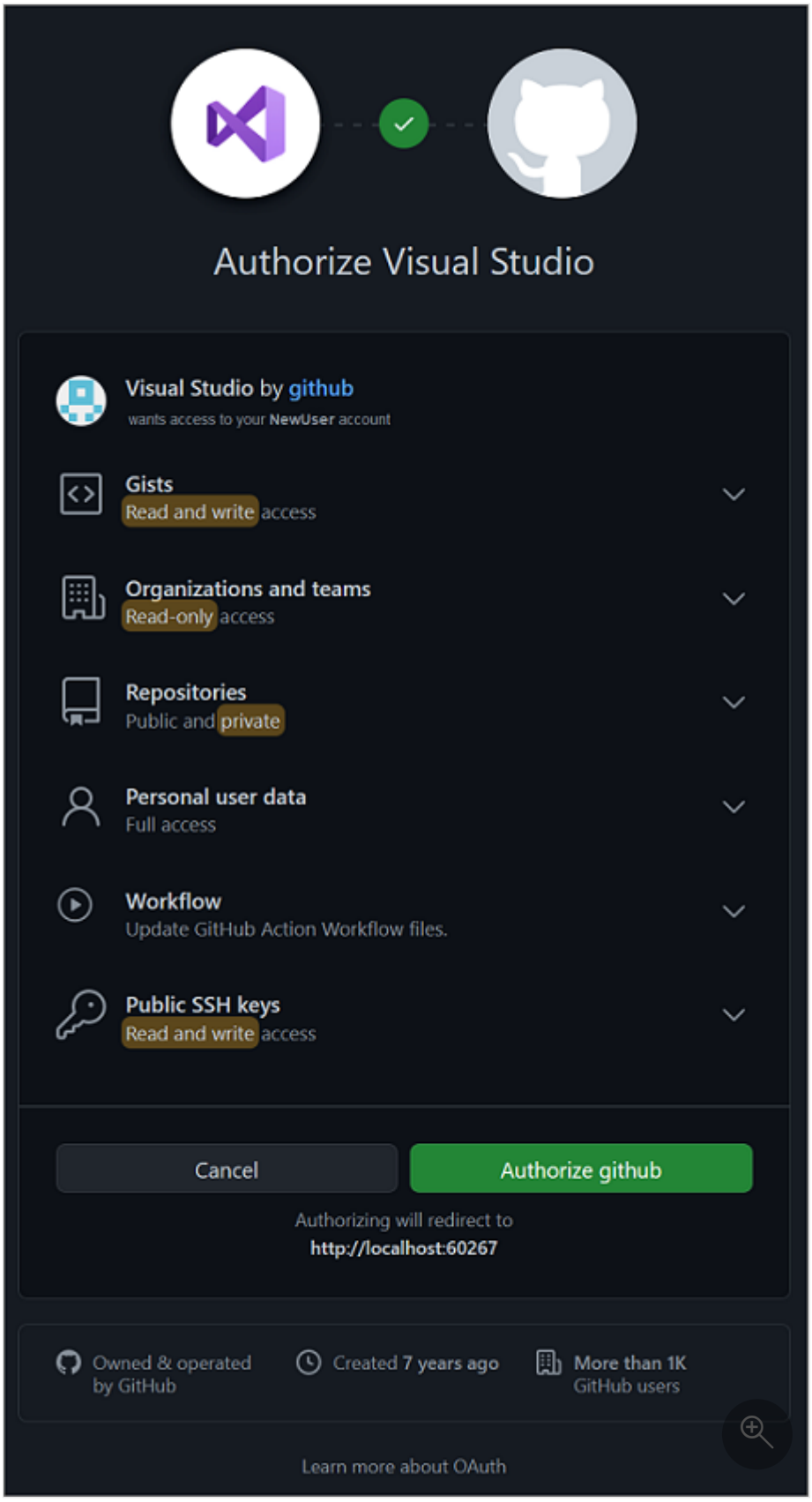
   Then, in the **Browse a repository** section, select **GitHub**.



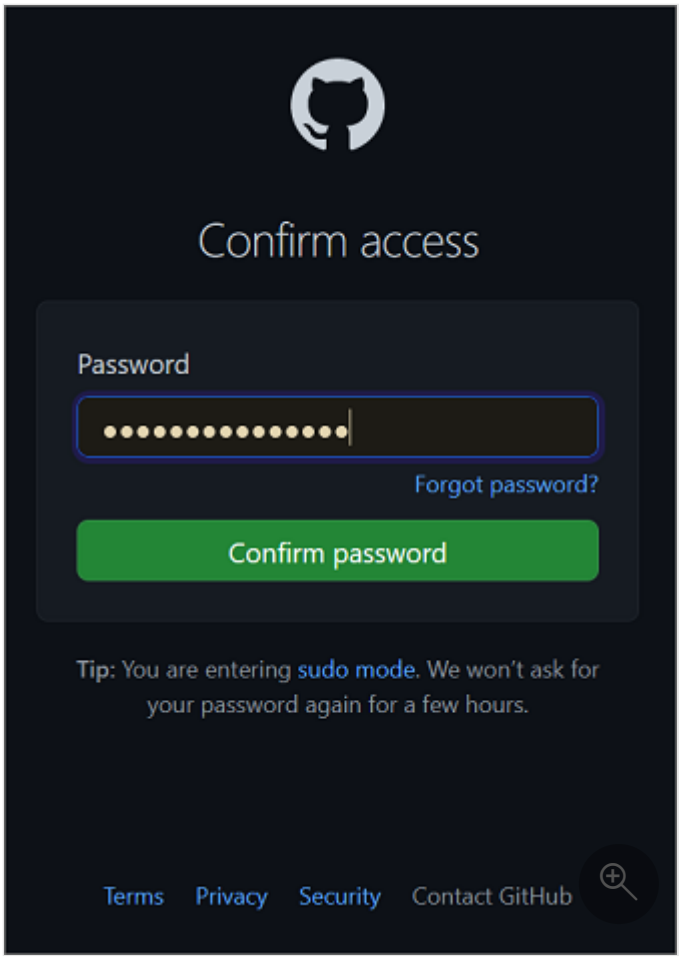
4. In the **Open from GitHub** window, you can either verify your GitHub account information or you can add it. To do so, select **Sign in** from the drop-down menu.



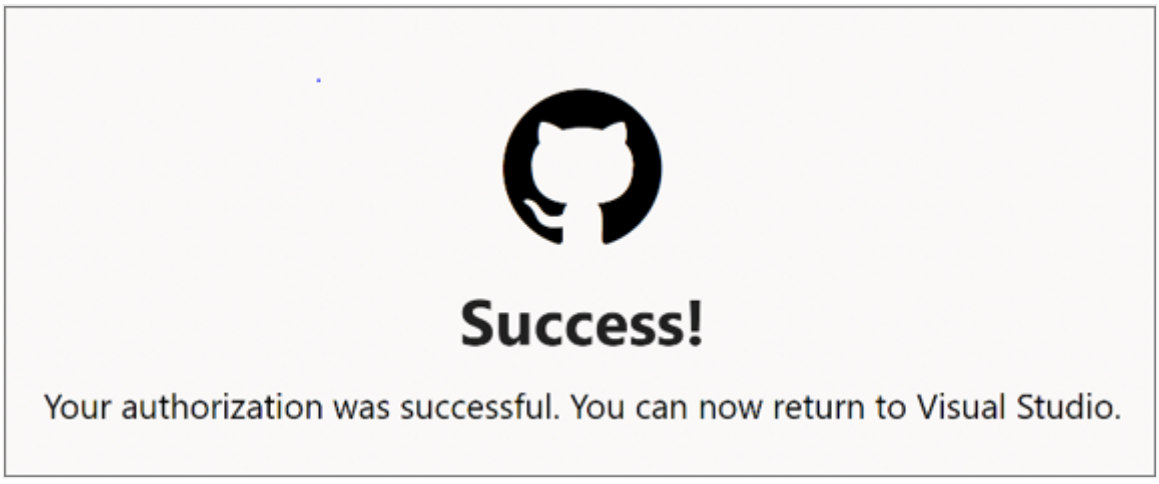
If you're signing in to GitHub from Visual Studio for the first time, an **Authorize Visual Studio** notice appears. Choose the options you want, and then select **Authorize github**.



Next, you'll see an authorization confirmation window. Enter your password, and then select **Confirm password**.

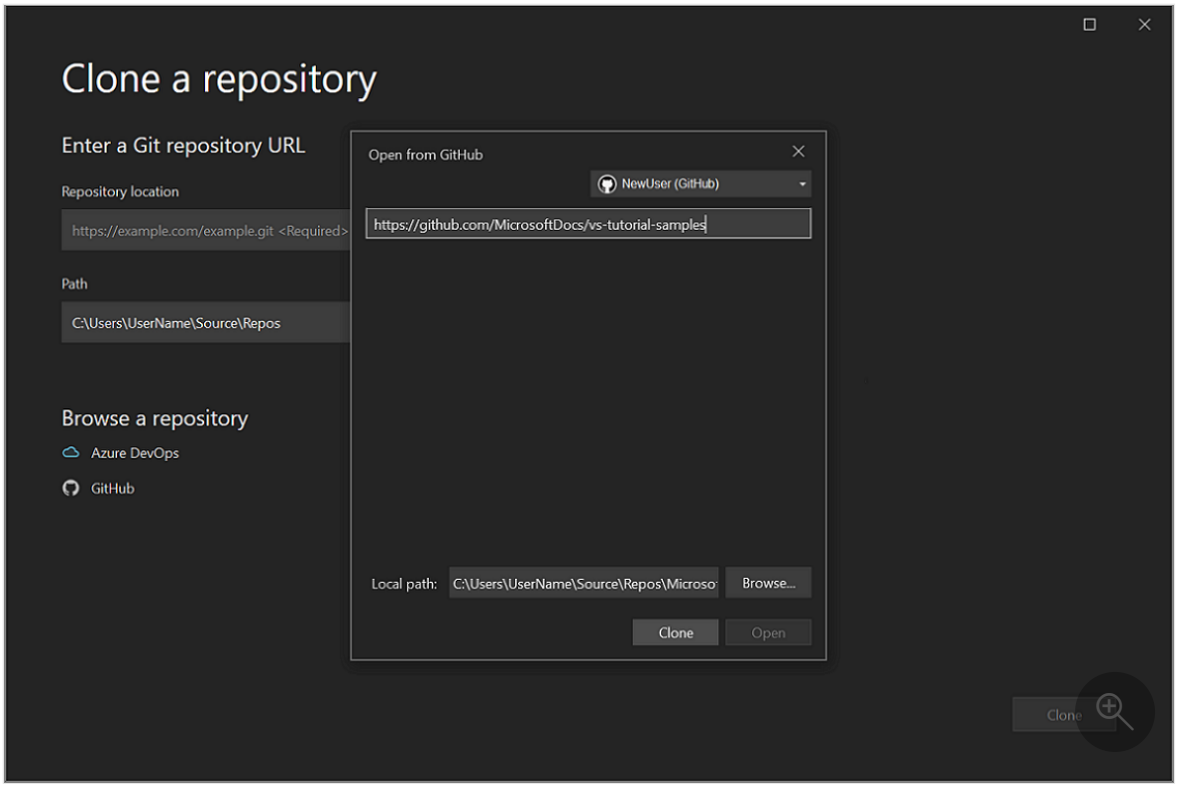


After you link your GitHub account with Visual Studio, a Success notification appears.

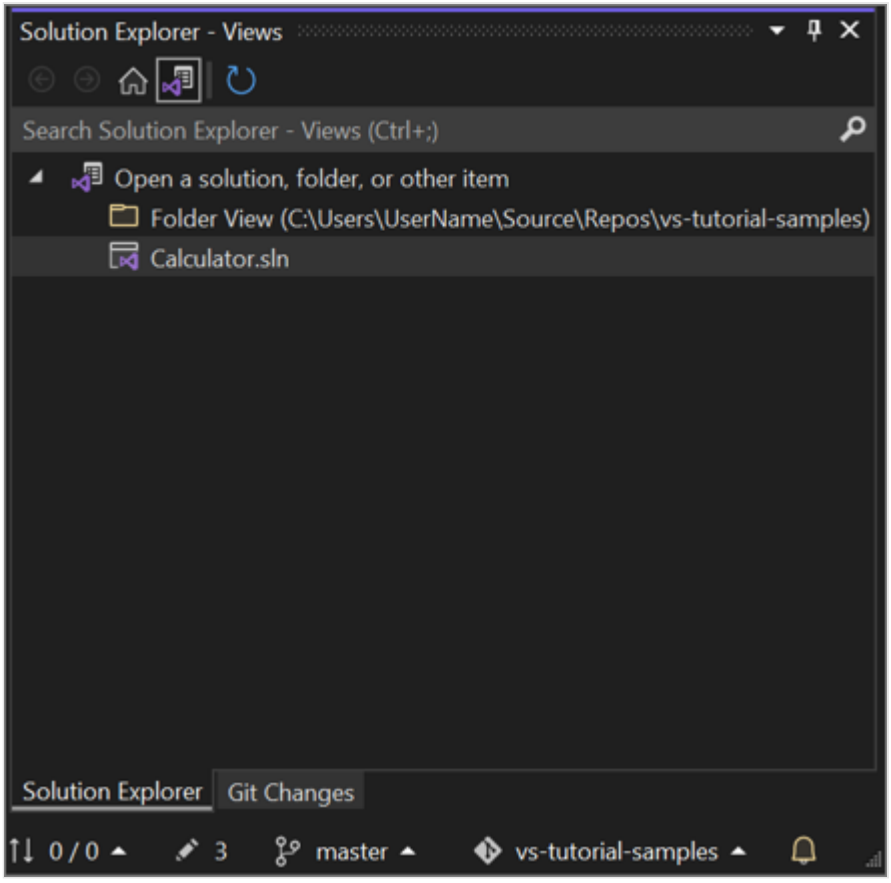


5. After you sign in, Visual Studio returns to the **Clone a repository** dialog, where the **Open from GitHub** window lists all the repositories that you have access to. Select the one you want, and then select **Clone**.

   If a list of repositories doesn't appear, enter the location of your repo, and then select **Clone**.



6. Next, Visual Studio presents a list of solution(s) in the repository. Choose the solution you would like to load or open the **Folder View** in Solution Explorer.



Tip

You can change the default Folder View to Solution View from the **Git** menu. Select **Settings** > **Source Control** > **Git Global Settings** > **Automatically load the solution when opening a Git repository** to do so.

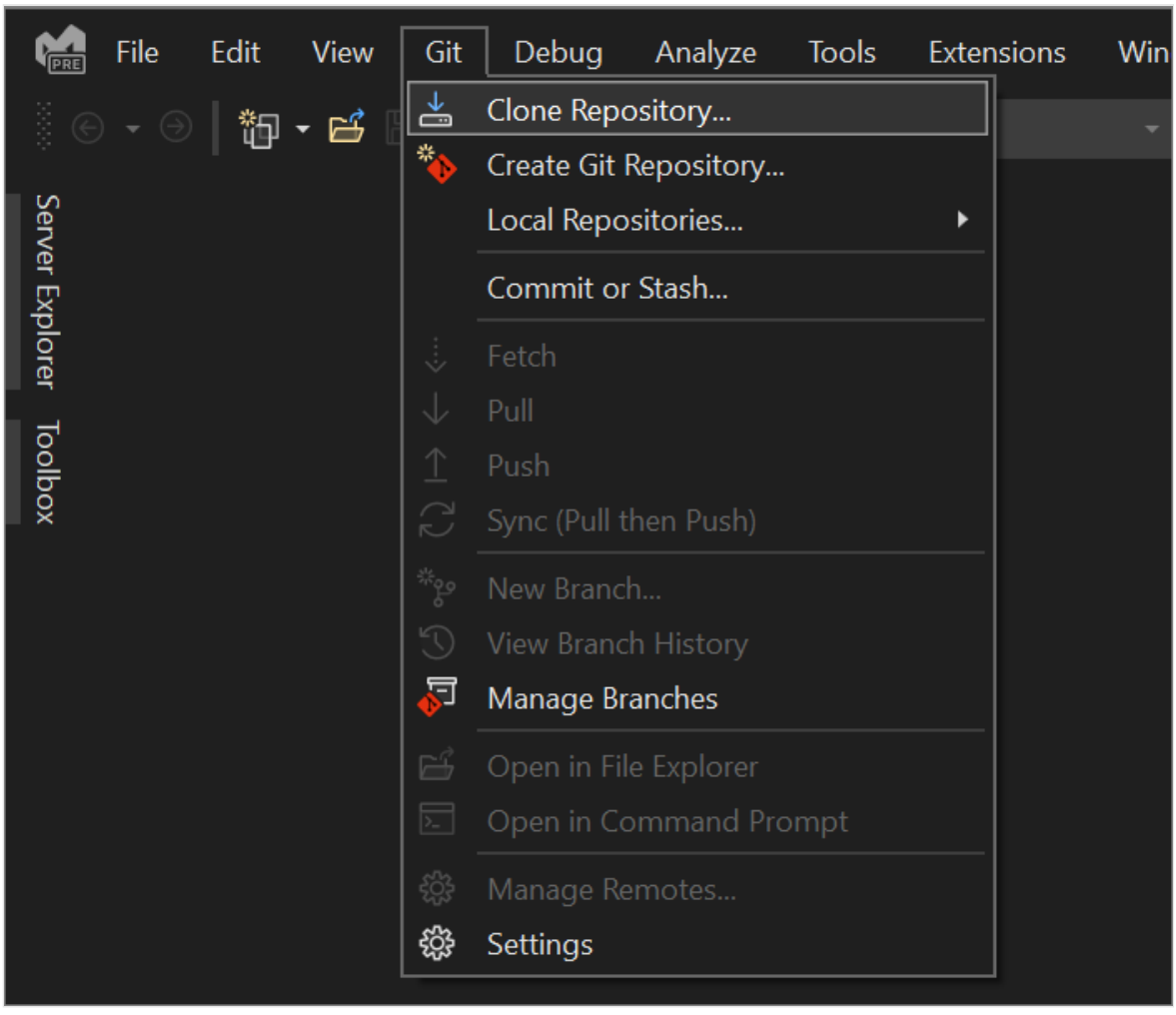
Open an existing local repository

After you've cloned a repo or created one, Visual Studio detects the Git repository and adds it to your list of **Local Repositories** in the Git menu. From there, you can quickly access and switch between your Git repositories.

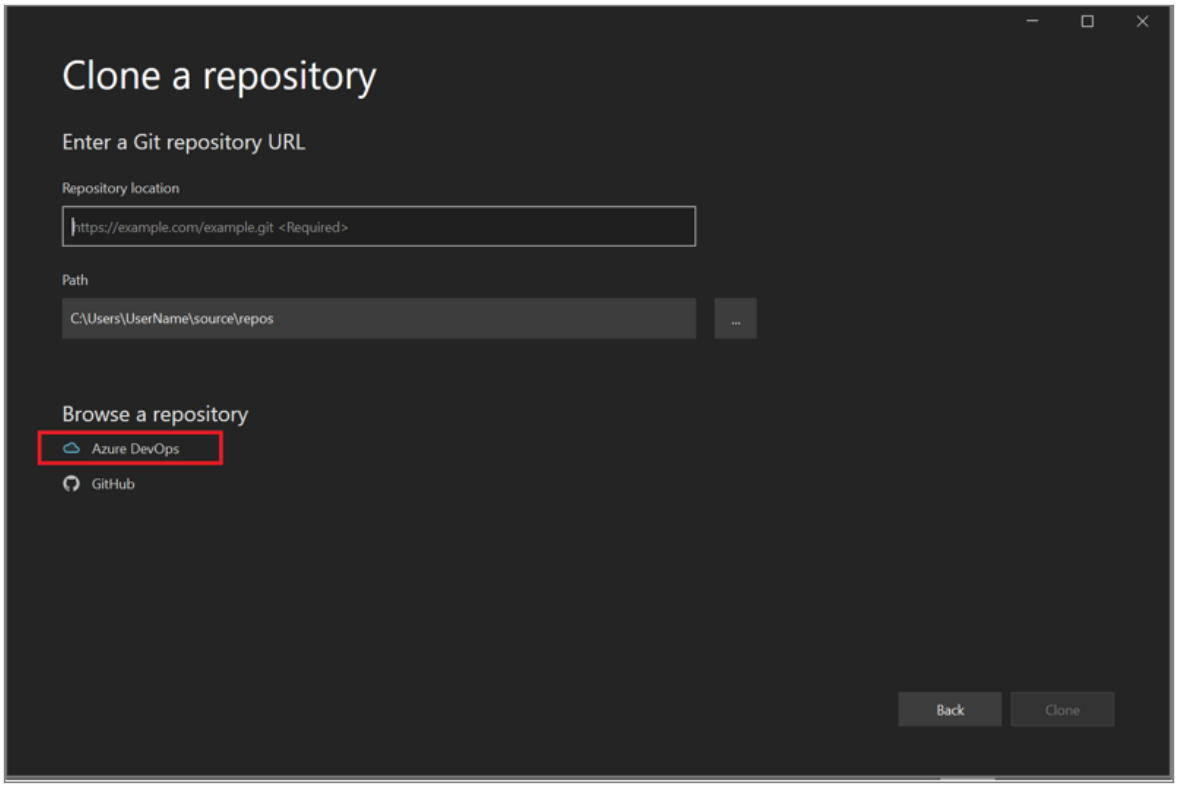
Browse to and then clone an Azure DevOps repo

1. Open Visual Studio.

2. From the **Git** menu, select **Clone Repository**.



3. In the **Browse a repository** section of the **Clone a repository** dialog box, select **Azure DevOps**.



4. A **Connect to a Project** dialog box appears. Follow the prompts to sign in to your Azure account, and then browse to Azure DevOps Server that hosts the files you're looking for.

Next steps

To continue your journey, visit the Create a repo page.

Create a Git repository from Visual Studio

Article • 03/17/2023

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

Visual Studio makes it easy to create a repo right from the IDE. Repository creation from Visual Studio is optimized for GitHub, but you can work remotely with the Git provider of your choice. Here's how.

Prerequisites

To follow this article, you'll need:

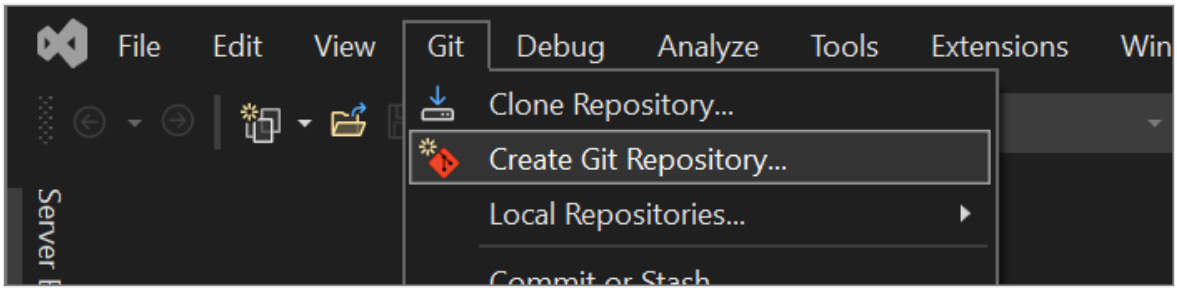
- Visual Studio installed
- A GitHub user account

Create a GitHub repo

1. Open Visual Studio, and then select **Create a new project**.

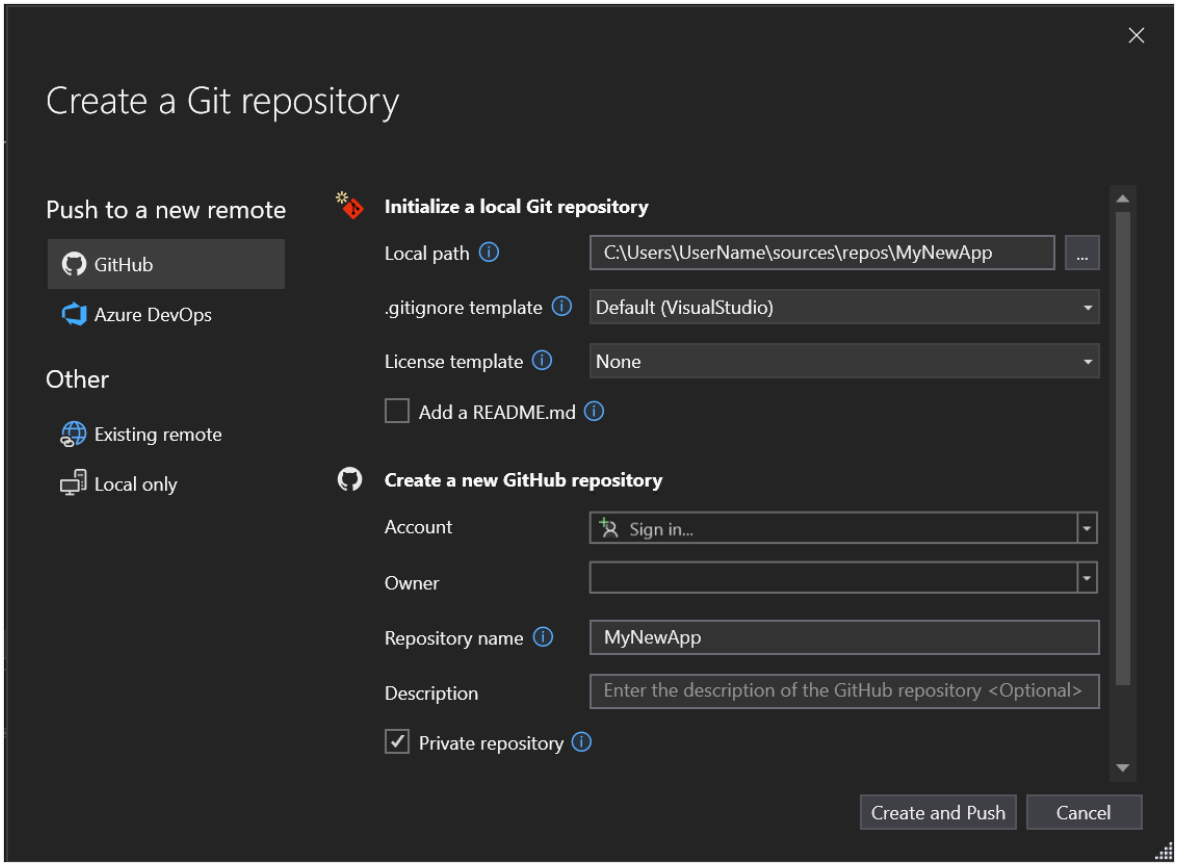
   > **Tip**
   >
   > If you don't already have a project in Visual Studio to add to a repo, you can quickly **create a new C# console app** and name it **MyNewApp**. Visual Studio populates your new app with default "*Hello, World!*" code.

2. From the **Git** menu, select **Create Git Repository**.



3. In the **Create a Git repository** dialog, under the **Push to a new remote** section, choose **GitHub**.

4. In the **Create a new GitHub repository** section of the **Create a Git repository** dialog, enter the name of the repo you want to create. (If you haven't yet signed in to your GitHub account, you can do so from this screen, too.)



   Under **Initialize a local Git Repository**, you can use the **.gitignore template** option to specify any intentionally untracked files that you want Git to ignore. To learn more about .gitignore, see Ignoring files. And to learn more about licensing, see Licensing a repository.

   > **Tip**
   >
   > You can update and change these settings whenever you want to. For detailed instructions, see **Git settings in Visual Studio**.

5. After you sign in and enter your repo info, select the **Create and Push** button to create your repo and add your app.

Open an existing local repository

After you've created a repo or cloned one, Visual Studio detects the Git repository and adds it to your list of **Local Repositories** in the Git menu. From there, you can quickly access and switch between your Git repositories.

Create an Azure DevOps repo

1. Open Visual Studio, and then select **Create a new project**.

   > **Tip**
   >
   > If you don't already have a project in Visual Studio to add to a repo, you can quickly **create a new C# console app** and name it **MyNewApp**. Visual Studio populates your new app with default "*Hello, World!*" code.

2. From the **Git** menu, select **Create Git Repository**.

3. In the **Create a Git repository** dialog, under the **Push to a new remote** section, choose **Azure DevOps**.

4. In the **Create a new Azure DevOps repository** section, sign in to your Azure account, and then select a project from the **Project** drop-down list.

5. Select the **Create and Push** button to create your repo and add your app.

Next steps

To continue your journey, visit the Create a branch page.

See also

- Tutorial: Open a project from a repo
- Work with GitHub accounts in Visual Studio

Git settings and preferences in Visual Studio

Article • 11/30/2022

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

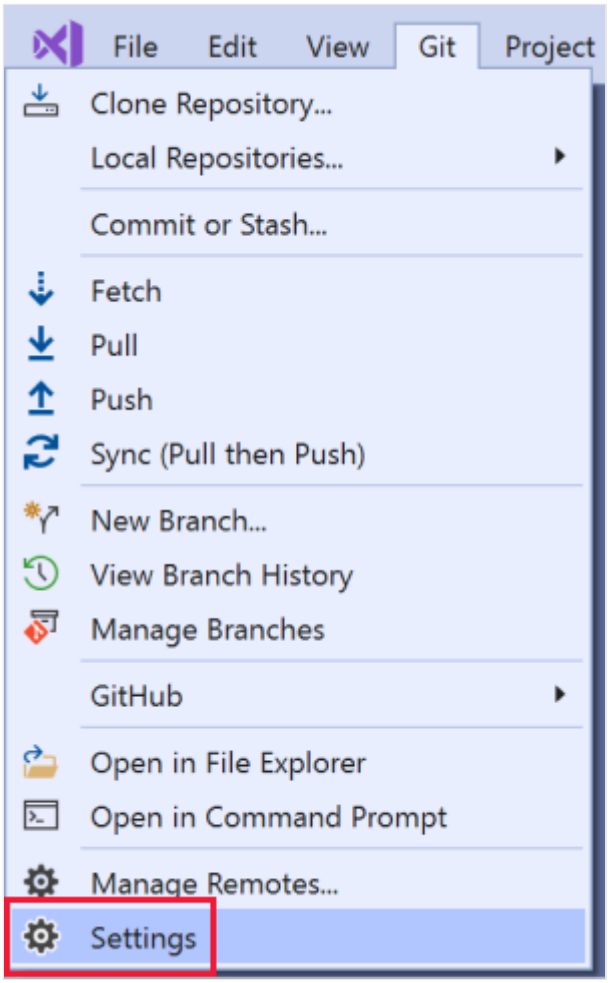
In Visual Studio, you can configure and view common Git settings and preferences, such as your name and email address, your preferred diff and merge tools, and more. These settings and preferences can be viewed and configured in the **Options dialog box** on either the **Git Global Settings** page (applies to all your repositories) or the **Git Repository Settings** page (applies to the current repository).

You can configure two types of settings:

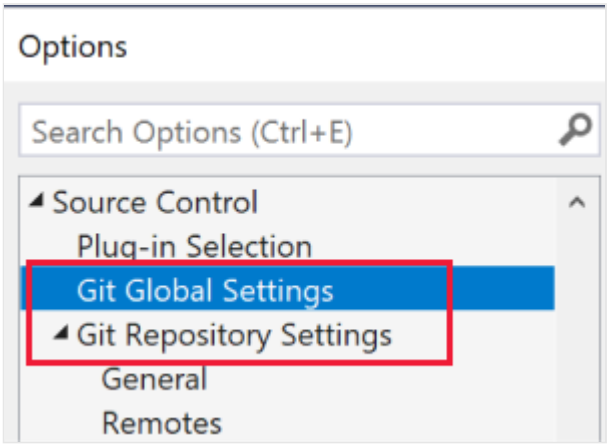
- Git settings - The settings in this section correspond to Git settings that are saved in Git configuration files. These settings can be viewed and modified in Visual Studio, but are managed by Git configuration files.
- Visual Studio settings - The settings in this section configure Git-related settings and preferences that are managed by Visual Studio.

How to configure settings

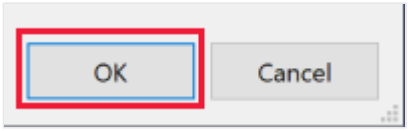
1. To configure Git settings in Visual Studio, choose **Settings** from the top-level Git menu.



2. Choose **Git Global Settings** or **Git Repository Settings** to view and configure global-level or repository-level settings.



3. You can configure several common Git settings, as described in the following sections of this article. After configuring your desired settings, select **OK** to save the updated settings.



Git settings

You can also configure and check some of the most common Git configuration settings. You can view and modify the following settings in Visual Studio, even though they are managed by Git configuration files.

- Name and email
- Prune remote branches during fetch
- Rebase local branch when pulling
- Cryptographic network provider
- Credential helper
- Diff & merge Tools
- Git files
- Remotes
- Other settings

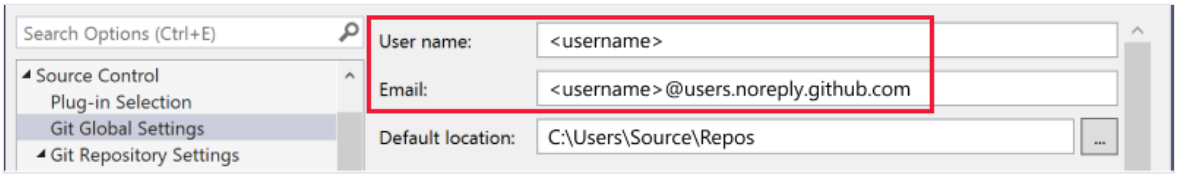
> **Note**
>
> Git settings configured in Visual Studio's **Global Settings** correspond to settings in Git's user-specific configuration file, and the settings in **Repository Settings** correspond to settings in the repository-specific configuration file. For more information about Git configuration, see the **Pro Git chapter on customizing Git**, the **git-config documentation**, and the **Pro Git reference on configuration files**. To configure Git settings not exposed in Visual Studio, use the `git config` command to write a value to your configuration files: `git config [--local|--global|--system] section.key value`.

Name and email

The name and email that you provide will be used as the committer information for any commit you make. This setting is available at both global and repository scopes, and corresponds to the `git config` user.name and user.email settings.

1. From the Git menu, go to **Settings**. To set your user name and email at the global level, go to **Git Global Settings**; to set your user name and email at the repository level, go to **Git Repository Settings**.

2. Provide your user name and email, then choose **OK** to save.



Prune remote branches during fetch

Pruning removes remote-tracking branches that no longer exist on the remote and helps you keep your branches list clean and up to date. This setting is available at both global and repository scopes, and corresponds to the `git config` fetch.prune setting.

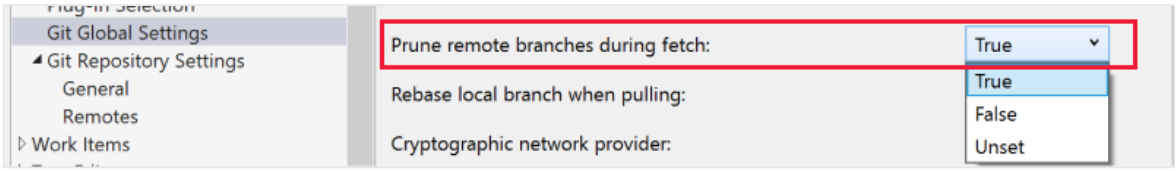
We recommend setting this option to **True** at the global level. Valid settings are, as follows:

- **True** (recommended)
- **False**
- **Unset** (default)

Here's how to change the settings:

1. From the Git menu, go to **Settings**. Go to **Git Global Settings** to configure this option at the global level; go to **Git Repository Settings** to configure this option at the repo level.

2. Set **Prune remote branches during fetch** to **True** (recommended). Select **Ok** to save.



Rebase local branch when pulling

Rebasing sets aside the changes made by commits in the current branch that are not in the upstream branch, resets the current branch to the upstream branch, then applies the changes that were set aside. This setting is available at both global and repository scopes, and corresponds to the `git config` pull.rebase setting. Valid settings are, as follows:

- **True**: Rebase current branch on top of upstream branch after fetch.

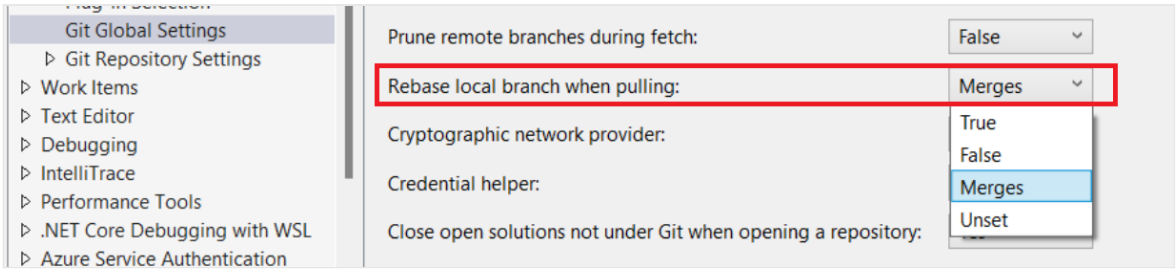
- **False**: Merge the current branch into the upstream branch.

- **Unset** (default): Unless specified in other configuration files, merge the current branch into the upstream branch.
- **Interactive**: Rebase in interactive mode.
- **Merges**: Rebase without flattening locally created merge commits.

> ⓘ Note
>
> In Visual Studio 2022 **version 17.2**, we changed the "Preserve" setting to "Merges" to match a recent update from Git. So, if you're using an earlier version of Visual Studio with Git tooling, your user interface might say "Preserve" instead of "Merges". The functionality remains the same, however.

Here's how to change the settings:

1. From the Git menu, go to **Settings**. Go to **Git Global Settings** to configure this option at the global level; go to **Git Repository Settings** to configure this option at the repo level.
2. Set **Rebase local branch when pulling** to the desired setting, and select **OK** to save.



It's not possible to configure `pull.rebase` to **Interactive** in Visual Studio. Visual Studio doesn't have interactive rebase support. To configure `pull.rebase` to use interactive mode, use the command line.

Cryptographic network provider

Cryptographic network provider is a Git configuration setting at global scope that configures which TLS/SSL backend to use at runtime, and corresponds to the `git config` http.sslBackend setting. The values are, as follows:

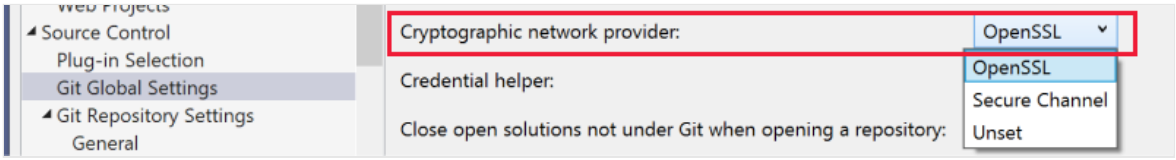
- **OpenSSL**: Use OpenSSL for TLS and SSL protocols.
- **Secure Channel**: Use Secure Channel (schannel) for TLS and SSL protocols. Schannel is the native Windows solution, accessing the Windows Credential Store,

thereby allowing for enterprise-wide management of certificates.
- **Unset** (default): If this setting is unset, OpenSSL is the default.

Here's how to change the settings:

1. From the Git menu, go to **Settings**. Go to **Git Global Settings** to configure this setting.

2. Set **Cryptographic network provider** to the desired value, and select **OK** to save.



Credential helper

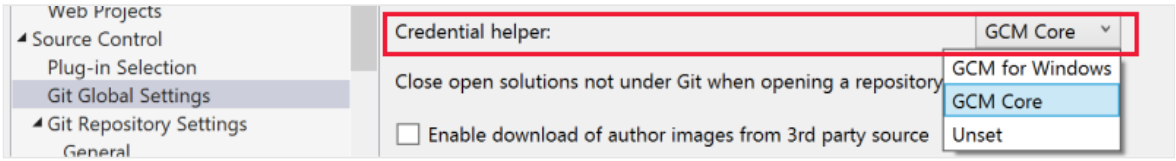
When Visual Studio performs a remote Git operation, the remote endpoint might reject the request because it requires credentials to be provided with the request. At that time, Git invokes a credential helper, which will return the credentials needed to perform the operation, and then will try the request again. The credential helper used corresponds to the `git config` credential.helper setting. It is available at the global scope with the following values:

- **GCM for Windows**: Use Git Credential Manager for Windows as the helper.
- **GCM Core**: Use Git Credential Manager Core as the helper.
- **Unset** (default): If this setting is unset, the credential helper set in the system config is used. As of Git for Windows 2.29, the default credential helper is GCM Core.

Here's how to change the settings:

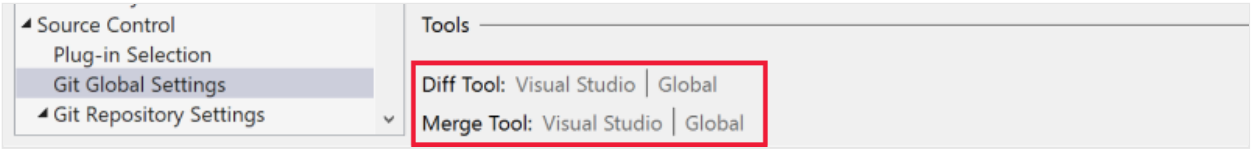
1. From the Git menu, go to **Settings**. Go to **Git Global Settings** to configure this setting.

2. Set **Credential helper** to the desired value, and select **OK** to save.



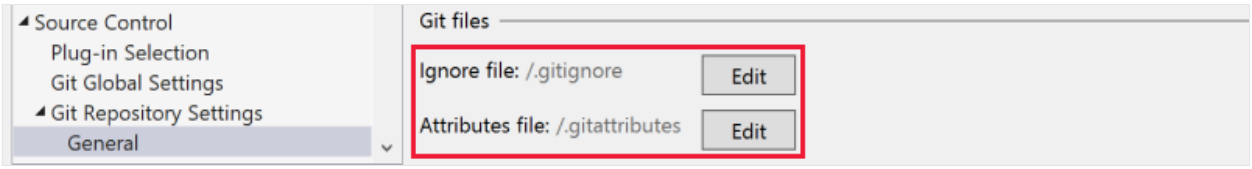
Diff & merge tools

Git will show diffs and merge conflicts in your preferred tools. The settings in this section correspond to the `git config` diff.tool and merge.tool settings. You can configure Git to use Visual Studio as your merge or diff tool in **Git Global Settings** and **Git Repository Settings** by selecting **Use Visual Studio**. To configure other diff and merge tools, use `git config` with the diff.tool or merge.tool switch.



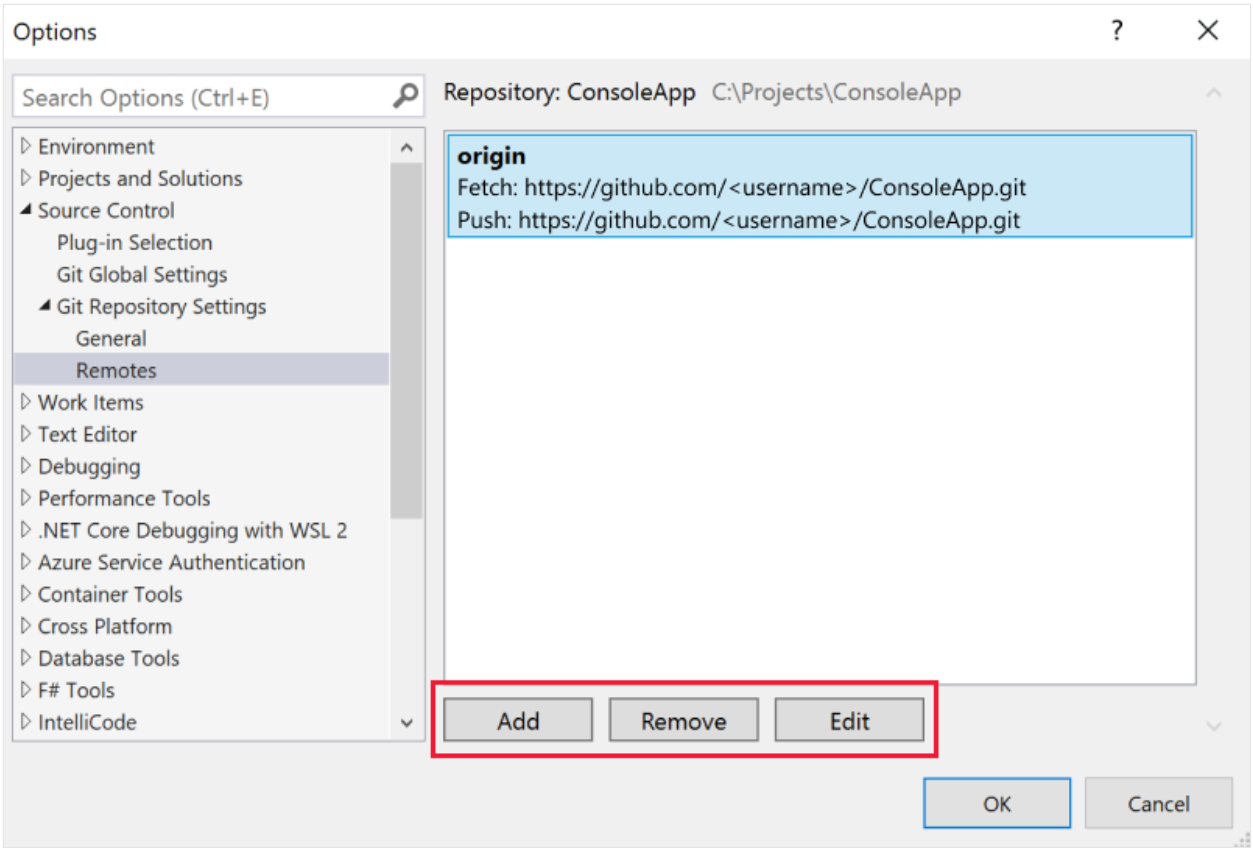
Git files

You can use the **Git files** section in the **Git Repository Settings** scope to view and edit the gitignore and gitattributes files for your repository.



Remotes

You can use the **Remotes** pane under **Git Repository Settings** to configure the remotes for your repository. This setting corresponds to the git remote command and allows you to add, edit, or remove remotes.



Other settings

To view all of your other Git configuration settings, you can open and view the configuration files themselves, or you can run `git config --list` to display the settings.

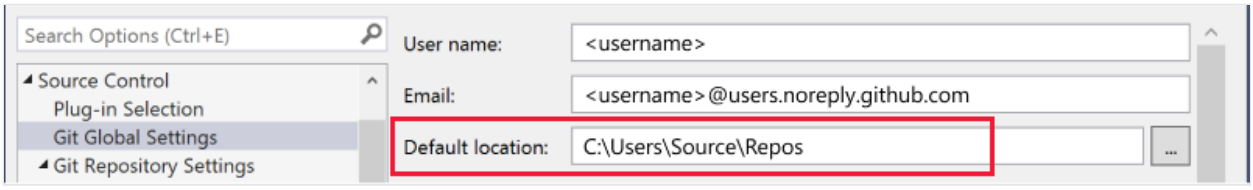
Visual Studio settings

The following settings manage Git-related preferences in Visual Studio, and are managed by Visual Studio instead of Git configuration files. All of the settings in this section are configured in the **Git Global Settings** page.

- Default location
- Close open solutions not under Git when opening a repository
- Enable download of author images from third-party sources
- Commit changes after merge by default
- Enable push --force
- Open folder in Solution Explorer when opening a Git repository
- Automatically load the solution when opening a Git repository
- Automatically check out branches with double-click or the Enter key

Default location

Default location configures the default folder in which repositories are cloned.



Close open solutions not under Git when opening a repository

By default, Visual Studio closes any open solution or folder when you switch to another repository. When it does so, it might also load the solution or folder of the new repository based on if you choose to Open folder in Solution Explorer when opening a Git repository and Automatically load the solution when opening a Git repository. This maintains the consistency between the open code and open repository. However, if your solution is not in the same folder root as your repository, you may want to keep the solution open when you switch to its repository. You can do that with this setting. The values are, as follows:

- **Yes**: When a repository is opened, the currently open solution is always closed
- **No**: When a repository is opened, Visual Studio performs a check as to whether the current solution is under Git. If it is not, then the solution remains open.
- **Always ask** (default): When this is set, you can make a choice through a dialog box per repository open, whether you want to keep the current solution open or close it.

Enable download of author images from third-party sources

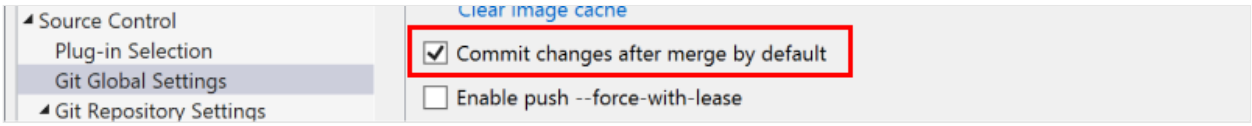
Enable download of author images from third-party sources is a Visual Studio-specific setting at global scope. When checked, author images are downloaded from the Gravatar image service, if available, and displayed in the commit and history views.

> **Important**

In order to provide author images in the Commit and History views, the tool creates an MD5 hash for the author email addresses stored in the active repository. This hash is then sent to Gravatar to find a matching hash value for users that have previously signed up for the service. If a match is found, the user image will be retrieved from the service and displayed in Visual Studio. Users that have not configured the service will return a randomly generated image. Note, email addresses are not recorded by Visual Studio, nor are they ever shared with Gravatar or any other third party.

Commit changes after merge by default

When **Commit changes after merge by default** is enabled, Git automatically creates a new commit when a branch is merged with the current branch.

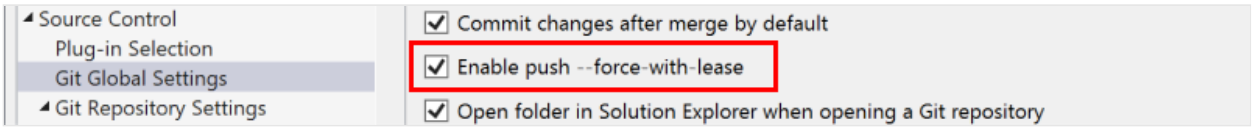


- When checked, `git merge` commands issued by Visual Studio are run with the `--commit` option.
- When unchecked, `git merge` commands issued by Visual Studio are run with the `--no-commit --no-ff` options.

For more information on these options, see --commit and --no-commit and --no-ff.

Enable push --force-with-lease

When enabled, this setting allows you to `push --force-with-lease` from within Visual Studio. By default **Enable push --force-with-lease** is disabled.

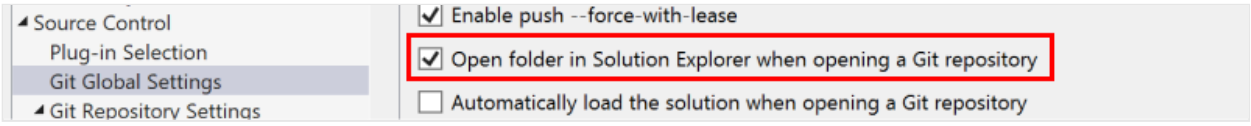


For more information, see push --force-with-lease.

Open folder in Solution Explorer when opening a Git repository

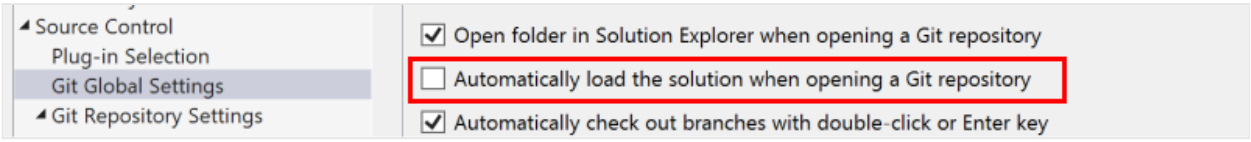
When you use Visual Studio to open or switch to a Git repository, Visual Studio loads the Git content so that you can view changes, commits, branches, and manage your repository from within the IDE. In addition, Visual Studio will also load the code of the

repository in Solution Explorer. Visual Studio will scan the repository folder for solutions, CMakeLists.txt, or any other view files that it recognizes and display them as a list in Solution Explorer. From there, you can select a solution to load or the folder to view the directory contents. When you turn off this checkbox, then Visual Studio will not open the repository folder in Solution Explorer. This will essentially allow you to open Visual Studio as a Git repository manager only. This setting is on by default.



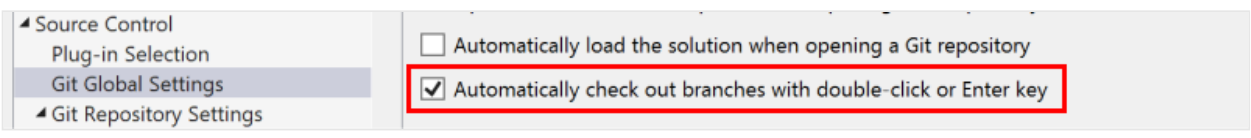
Automatically load the solution when opening a Git repository

This setting is applicable only when the Open folder in Solution Explorer when opening a Git repository setting is turned on. When you open a Git repository in Visual Studio, and the subsequent folder scan detects there is only one solution present in your repository, then Visual Studio automatically loads that solution. If you turn off the setting, then the Solution Explorer will display the single solution present in the repository in the list of views. But it will not load the solution. By default, this setting is off.



Automatically check out branches with double-click or the Enter key

The Git Repository window has a list of branches displayed in a tree structure. Single selecting a branch will switch the commit history pane to display the commits for the selected branch. To check out a branch, you can right-click to open the context menu and select **Checkout**. If you turn on this setting, then double-clicking or pressing the Enter key will check out the branch and display its commits.



See also

> **Important**
>
> If you have a suggestion for us, please let us know! We appreciate the opportunity to engage with you on design decisions via the **Developer Community** portal.

- Getting started with Git in Visual Studio video on YouTube
- Enhanced productivity with Git in Visual Studio blog post
- Options dialog box

Create a Git branch in Visual Studio

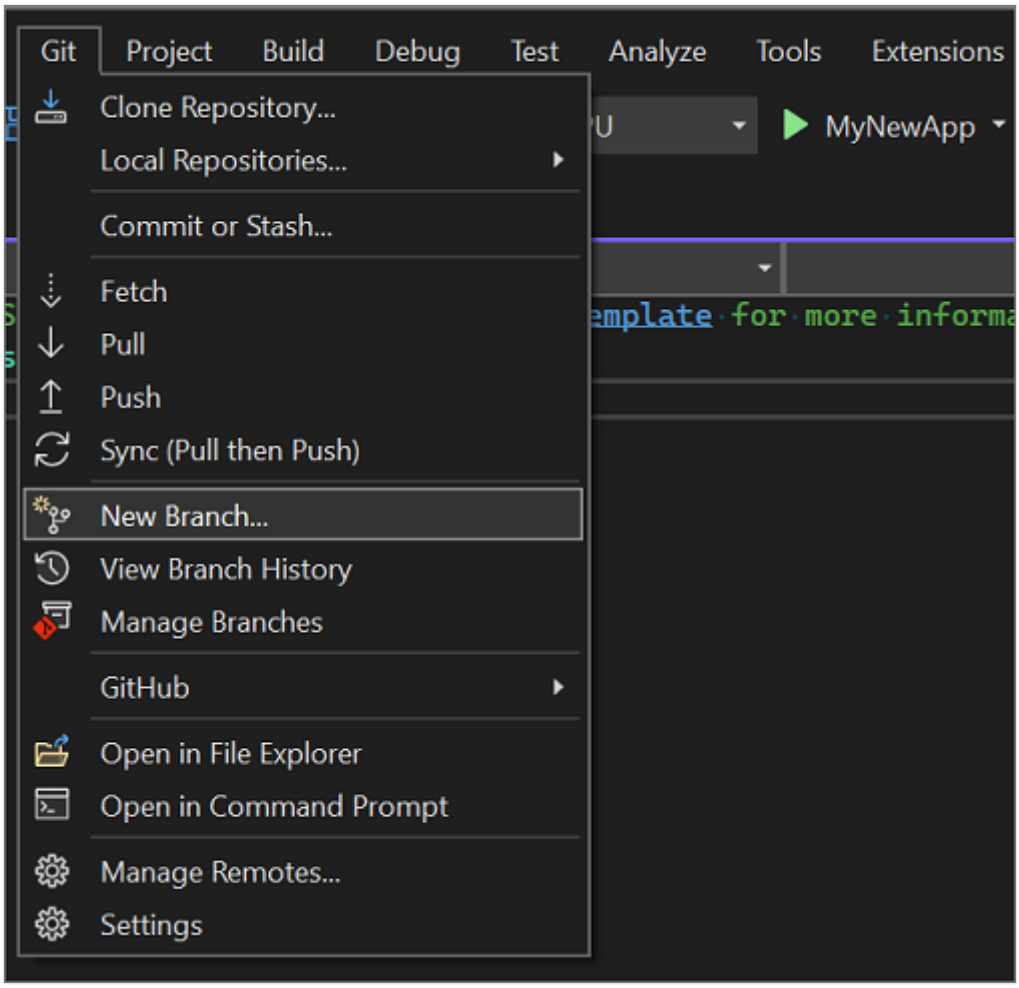
Article • 09/27/2023

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

It's easy to create a new branch in Visual Studio; all you have to do is base it off an existing branch.

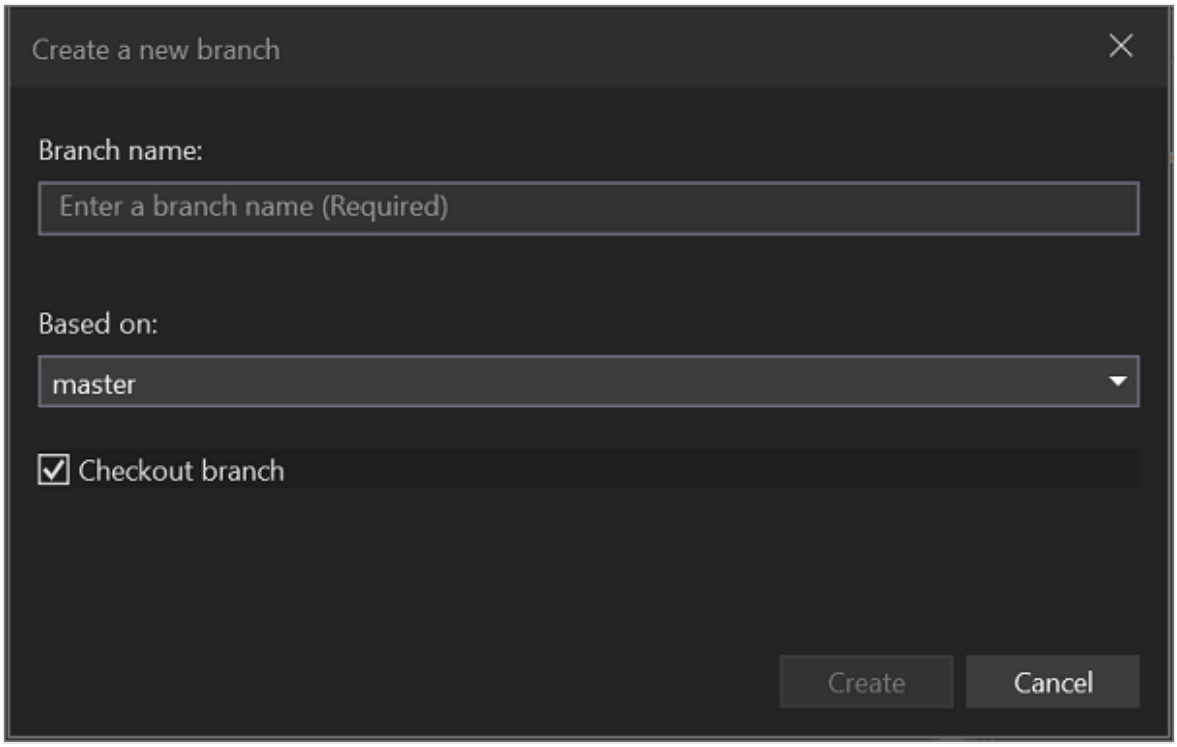
Here's how.

1. To start, make sure you've got a previously created or cloned repo open.
2. From the **Git** menu, select **New Branch**.



3. In the **Create a new branch** dialog box, enter a branch name.

> **Tip**
>
> For branch naming details, see **Special characters in branch and tag names**.



4. In the **Based on** section, use the drop-down list to choose whether you want to base your new branch off an existing local branch or a remote branch.

5. The **Checkout branch** checkbox, which is on by default, automatically switches to the newly created branch. Toggle this option if you want to remain in the current branch.

There you have it; you've created a new branch.

> **Tip**
>
> The equivalent command for this action is `git checkout -b <new-branch> <existing-branch>`.

> **Note**
>
> For more information about the latest updates that improve branch switching, see the **Visual Studio 2022 Performance Enhancements: Git Branch Switching** blog post.

Next steps

To continue your journey, visit the Make a commit page. And to learn more about how to manage branches in Visual Studio, see Merge and rebase branches.

Make a Git commit in Visual Studio

Article • 05/26/2023

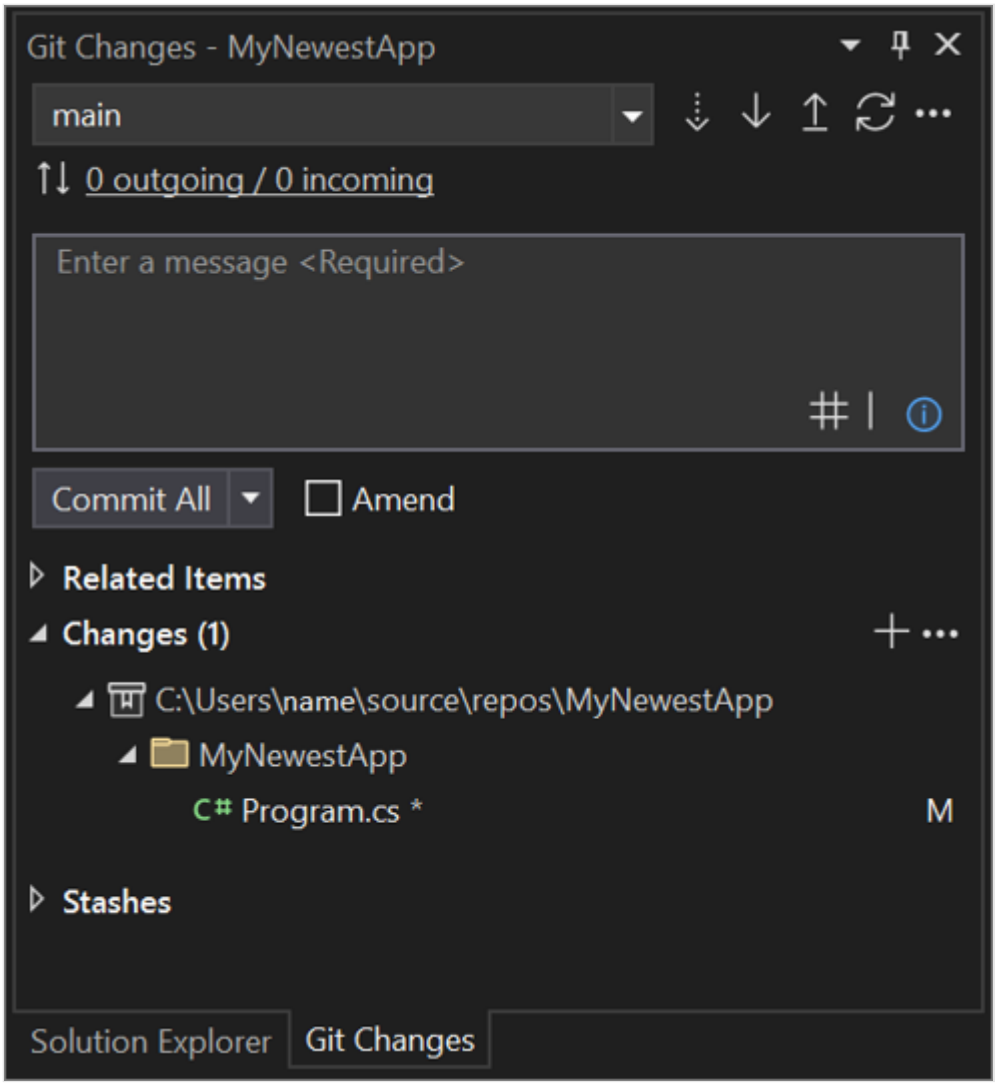
Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

The core part of any Git workflow is modifying files and committing the changes in those files. While this article references GitHub repositories, you can work remotely with the Git provider of your choice, such as GitHub or Azure DevOps. Or, you can work locally with no provider at all.

Git tracks file changes in your repo as you work, and separates the files in your repo into three categories. These changes are equivalent to what you would see when you enter the `git status` command in the command line:

- **Unmodified files**: These files haven't changed since your last commit.
- **Modified files**: These files have changes since your last commit, but you haven't yet staged them for the next commit.
- **Staged files**: These files have changes that will be added to the next commit.

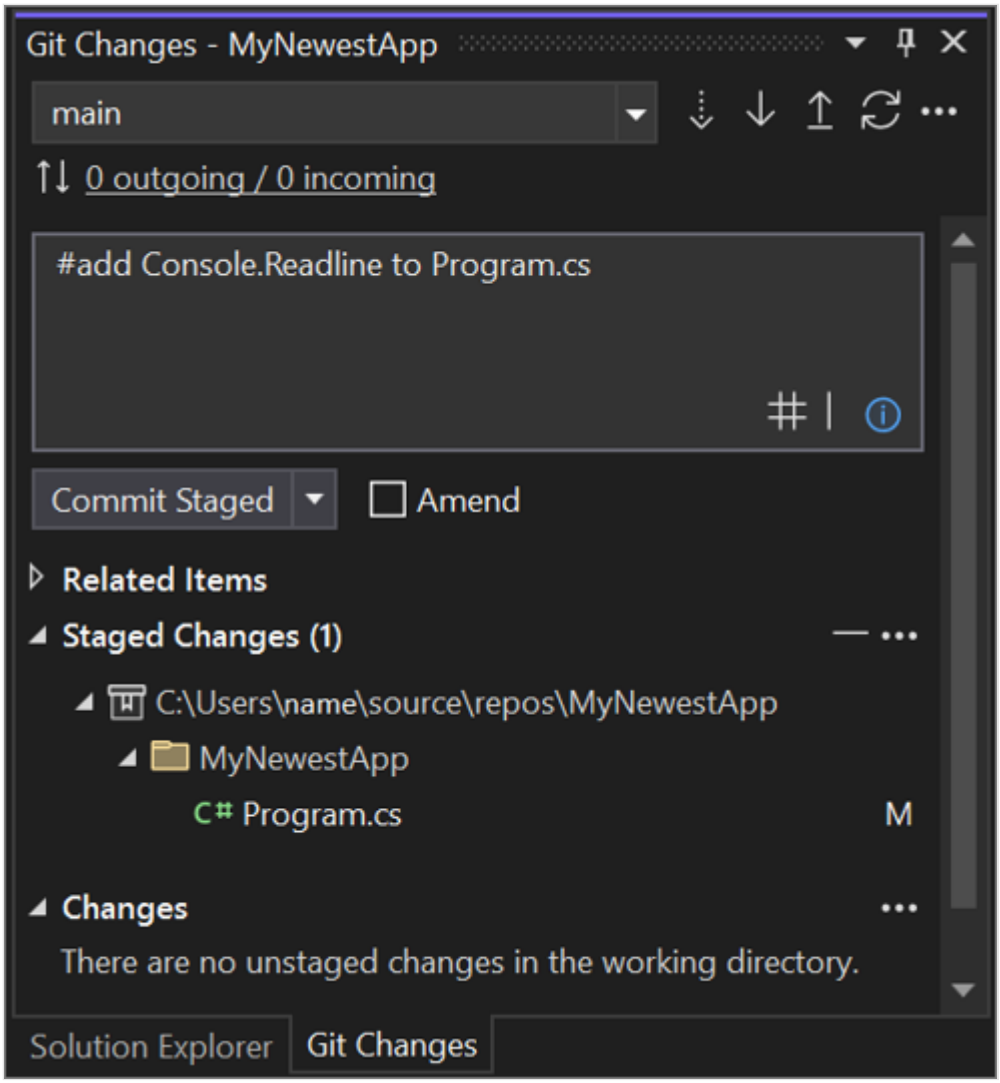
As you do your work, Visual Studio keeps track of the file changes to your project in the **Changes** section of the **Git Changes** window.



New in **version 17.6**: You can also reference an issue or a pull request by typing `#` or selecting the # button in the lower right side of the commit message text box. For more information, see the Reference issues and pull requests blog post.

To stage changes when you're ready, select the + (plus) button on each file you want to stage, or right-click a file and then select **Stage**. You can also stage all your modified files with one click by using the stage all + (plus) button at the top of the **Changes** section.

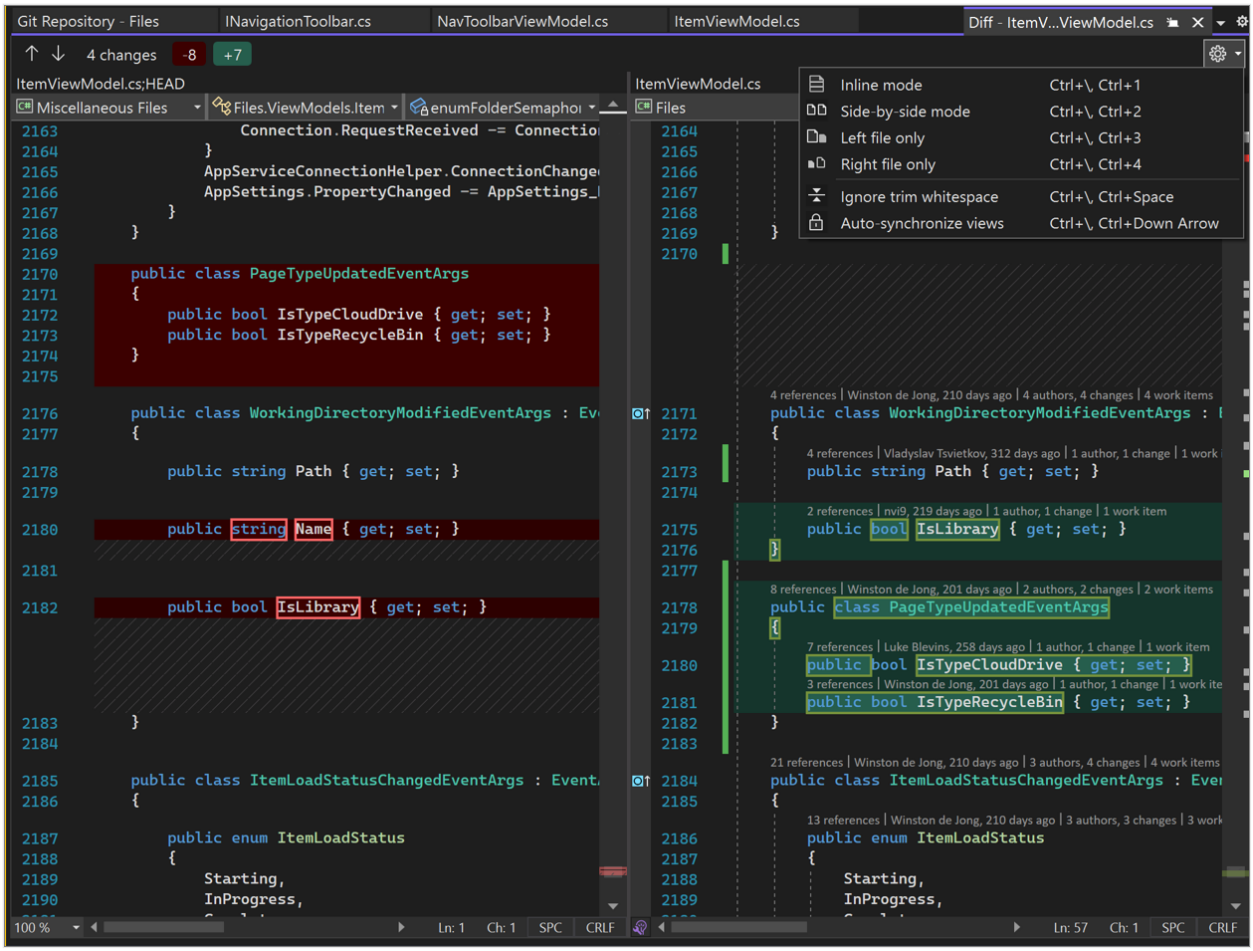
When you stage a change, Visual Studio creates a **Staged Changes** section. Only changes in the **Staged Changes** section are added to the next commit, which you can do by selecting **Commit Staged**. The equivalent command for this action is `git commit -m "Your commit message"`.



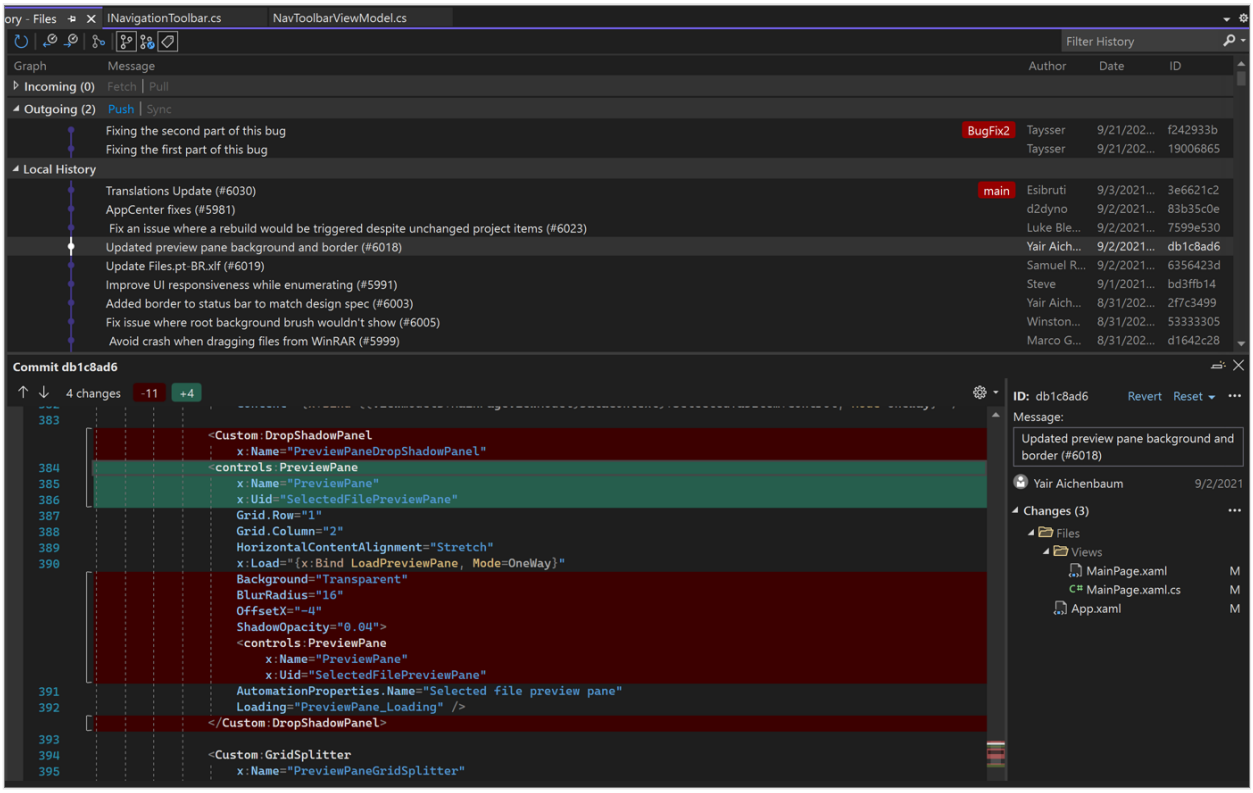
Changes can also be unstaged by clicking the – (minus) button. The equivalent command for this action is `git reset <file_path>` to unstage a single file or `git reset <directory_path>` to unstage all the files in a directory.

You can also choose not to stage your modified files by skipping the staging area. In this case, Visual Studio allows you to commit your changes directly without having to stage them. Just enter your commit message and then select **Commit All**. The equivalent command for this action is `git commit -a`.

Visual Studio also makes it easy to commit and sync with one click by using the **Commit All and Push** and **Commit All and Sync** shortcuts. When you double-click any file in the **Changes** and the **Staged changes** sections, you can see a line-by-line comparison with the unmodified version of the file.



When you double-click a **Commit**, Visual Studio opens its details in a separate tool window. From here you can revert the commit, reset the commit, amend the commit message, or create a tag on the commit. When you click a changed file in the commit, Visual Studio opens the side-by-side **Diff** view of the commit and its parent.



Next steps

To continue your journey, visit the Stage lines of code page.

See also

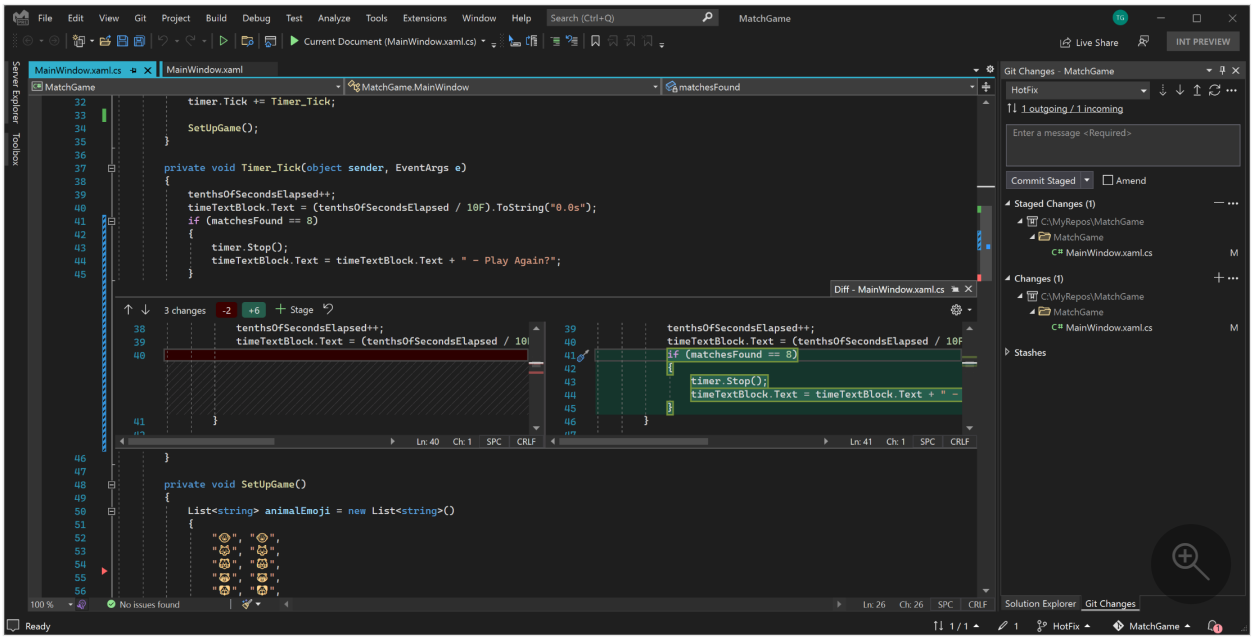
- Git experience in Visual Studio
- Visual Studio & GitHub: Better together

Stage lines of code in Visual Studio

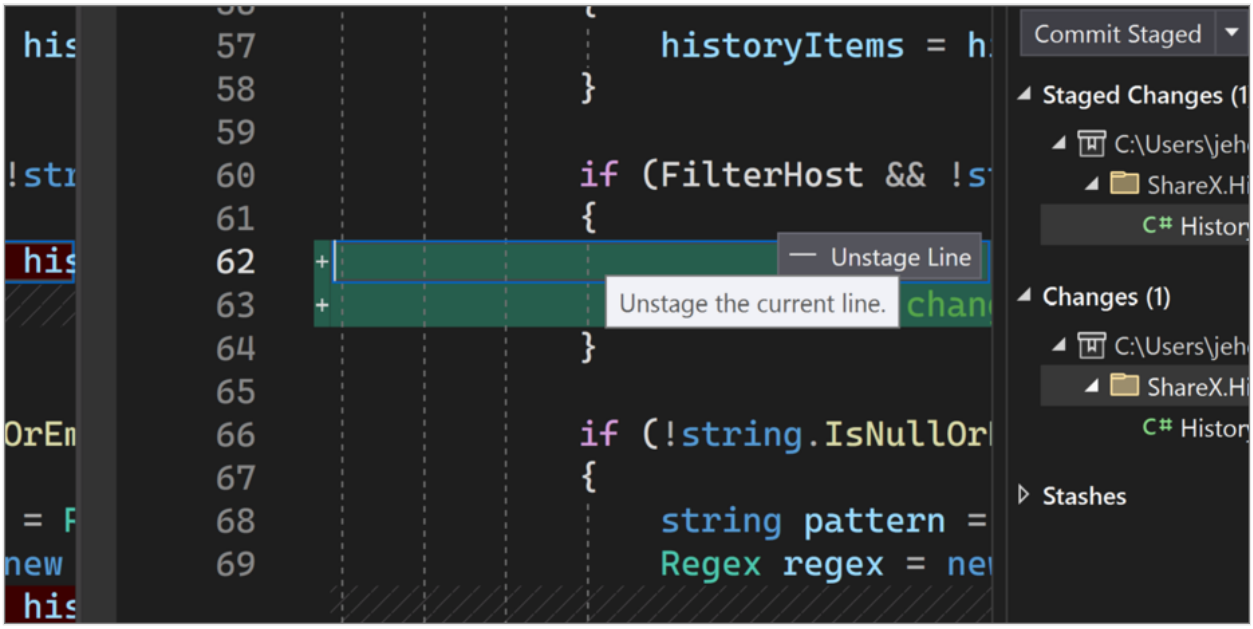
Article • 05/16/2023

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

Line-staging support, also known as interactive staging, is available in Visual Studio 2022 version 17.3 and later. Line-staging lets you stage chunks of changes in your files right from the code editor. Line staging can also be helpful when you want to split changes across different commits.

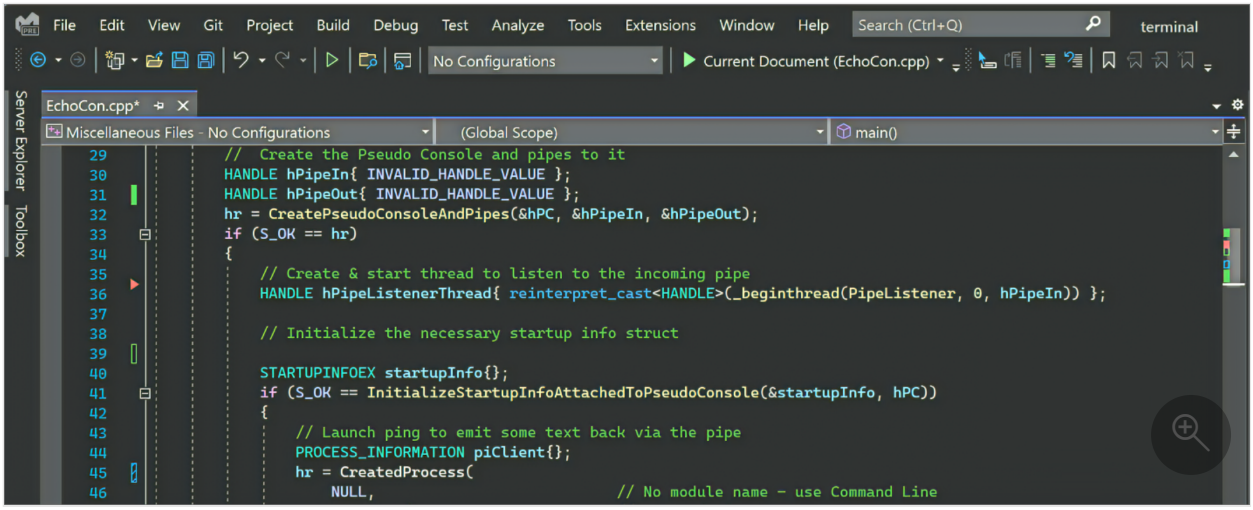


Starting with Visual Studio 2022 version 17.6, you can use the hover option to also *unstage* changes, line by line.



Color margin support

The first thing you might notice about line-staging support is how the editor visualizes Git changes on both the scroll bar and in the margin.

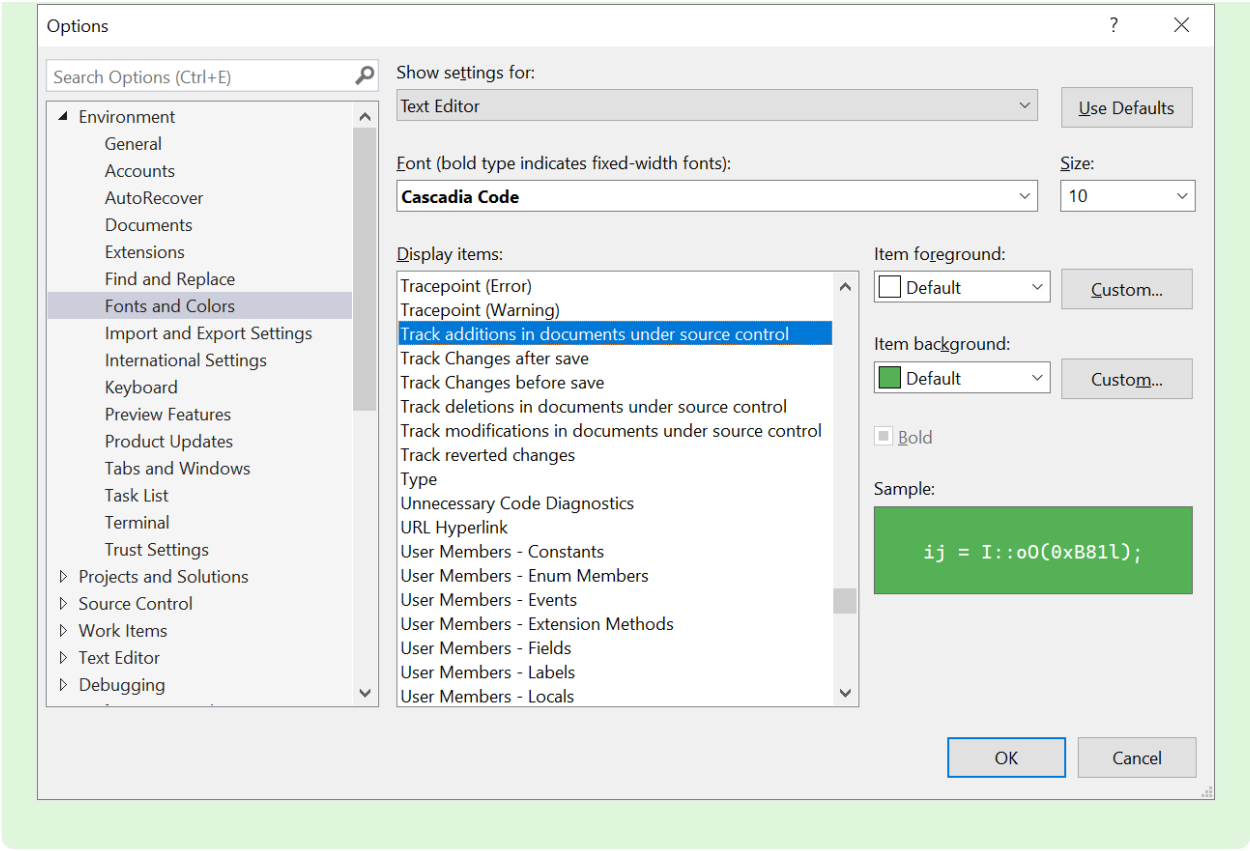


The colors and annotations make it easy to differentiate between saved and unsaved changes that you've added, deleted, or modified.

Tip

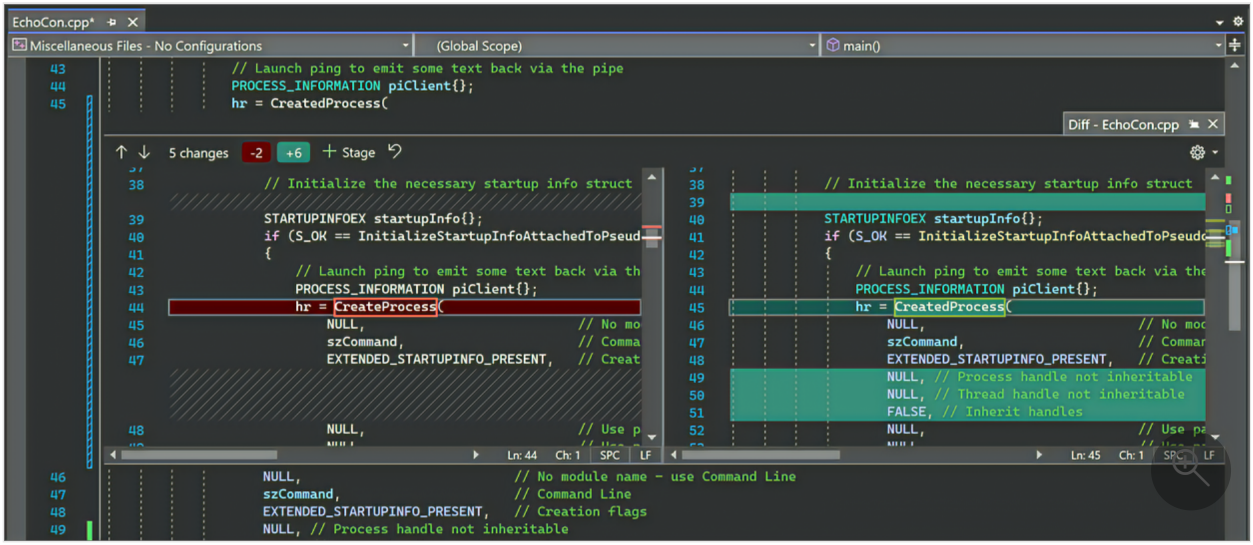
You can change the colors that Visual Studio displays to help you manage changes. Go to **Tools** > **Options** > **Environment** > **Fonts and Colors** and then choose from the following options:

- Track additions in documents under source control
- Track deletions in documents under source control
- Track modifications in documents under source control

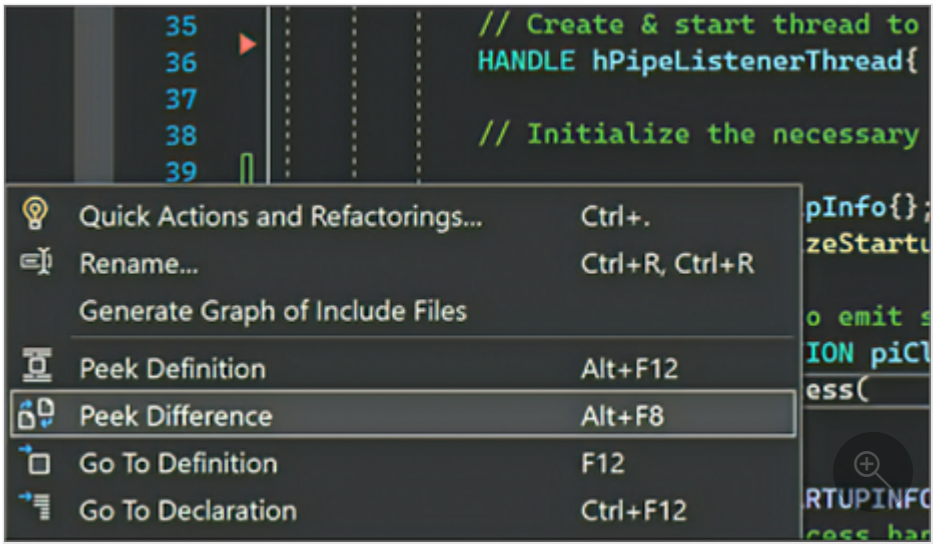


Peek Difference support

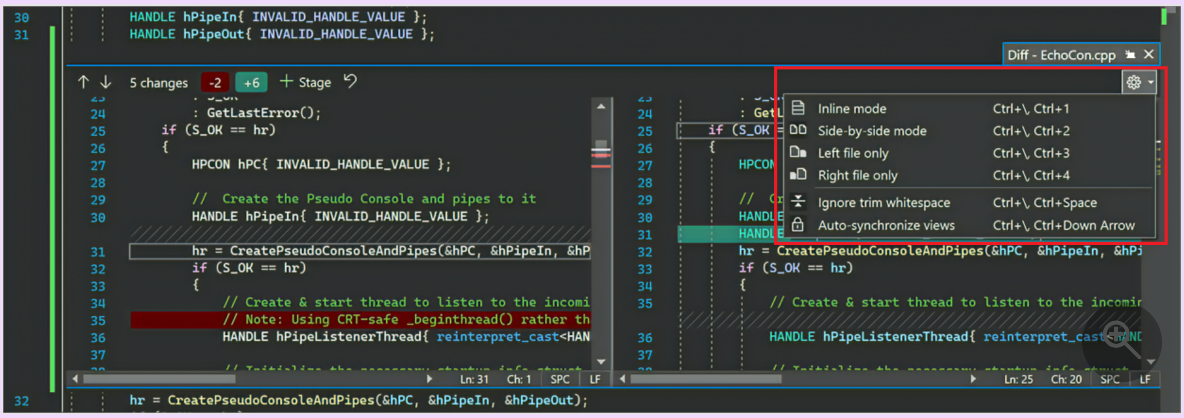
Color margins are interactive. When you select a color margin, Visual Studio opens the **Peek Difference** user interface (UI) where you can view your changes in the editor without switching context. The Peek Difference UI summarizes the number of added and removed lines and makes it easy to navigate between changes by using the Up and Down arrows.



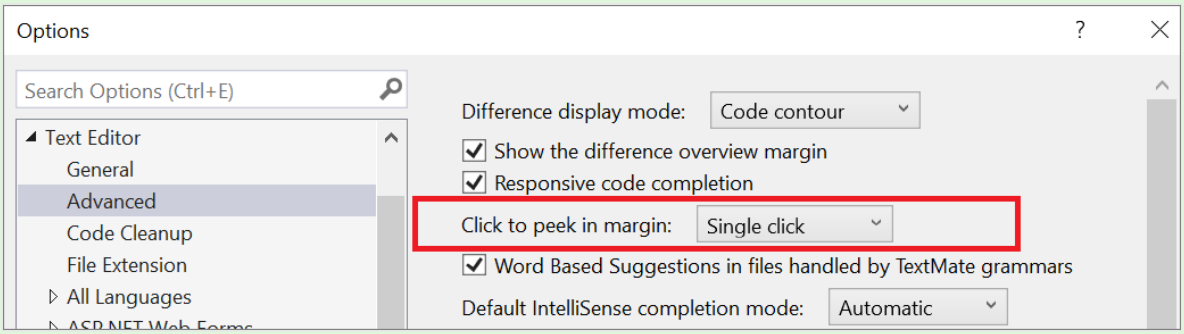
You can also access Peek Difference when you right-click a line that includes changes. A context menu appears where you can select **Peek Difference** from it. Or, you can use the **Alt+F8** keyboard shortcut.



> **Note**
>
> If you prefer an inline difference view, you can use the **Settings** control in the top-right corner of the **Diff** editor to switch to **Inline mode**.



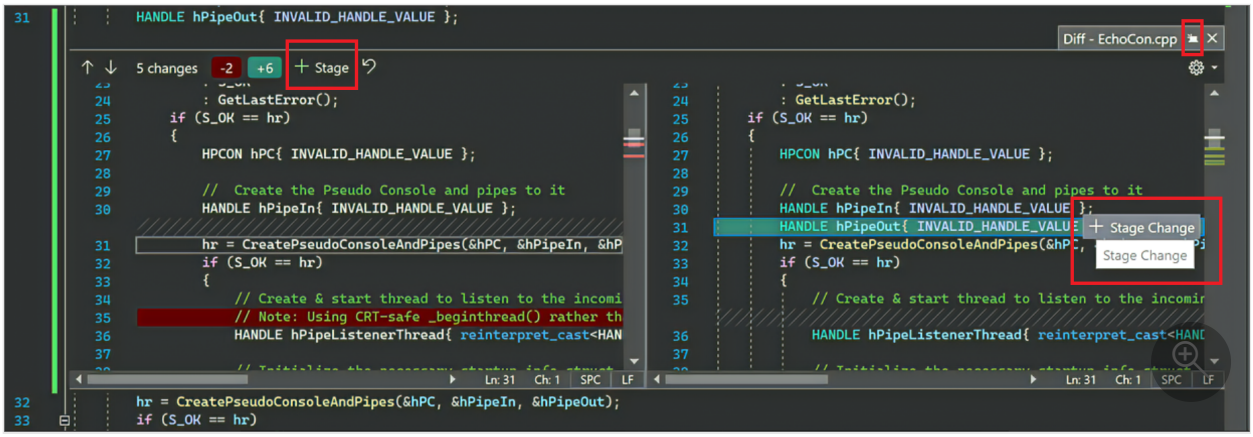
> **Tip**
>
> You can modify how Visual Studio interacts with code changes in the Peek Difference UI by using the **Click to peek in margin** option from **Tools** > **Options** > **Text Editor** > **Advanced**.



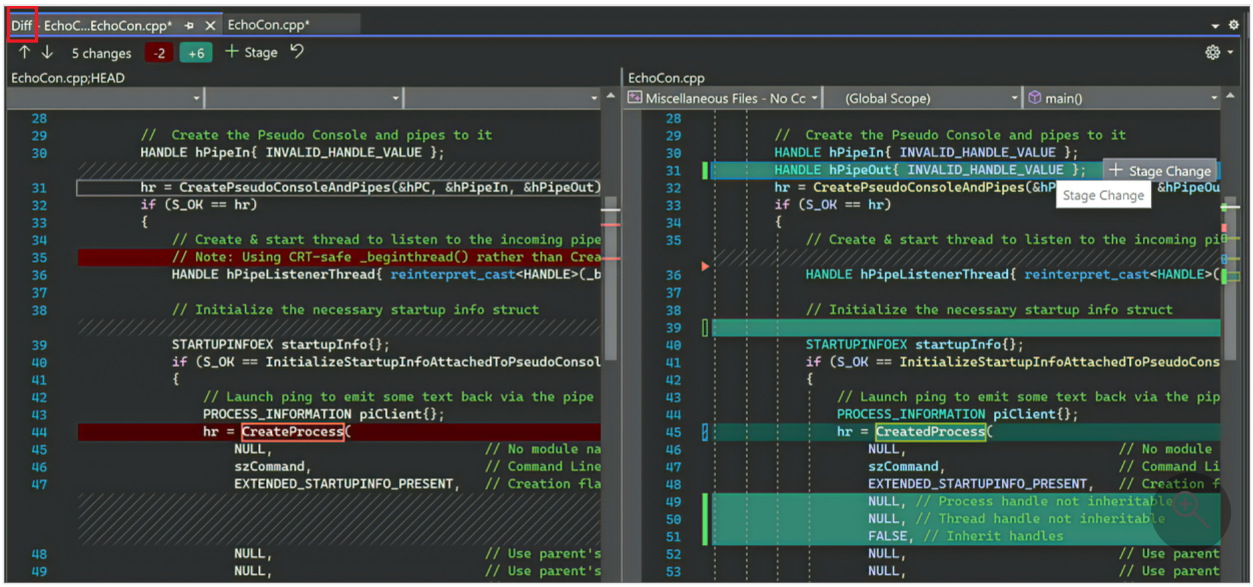
> For example, you can change the default **Single click** setting to **Double click**, or you can select **None** to turn off the Peek Difference UI.

Stage chunks of code

You can stage any chunk of code by using the Peek Difference user interface (UI). To do so, hover over the change you want to stage and select **Stage Change**. Alternatively, use the global **Stage** button if you want to stage *all* the changes you made to a document.

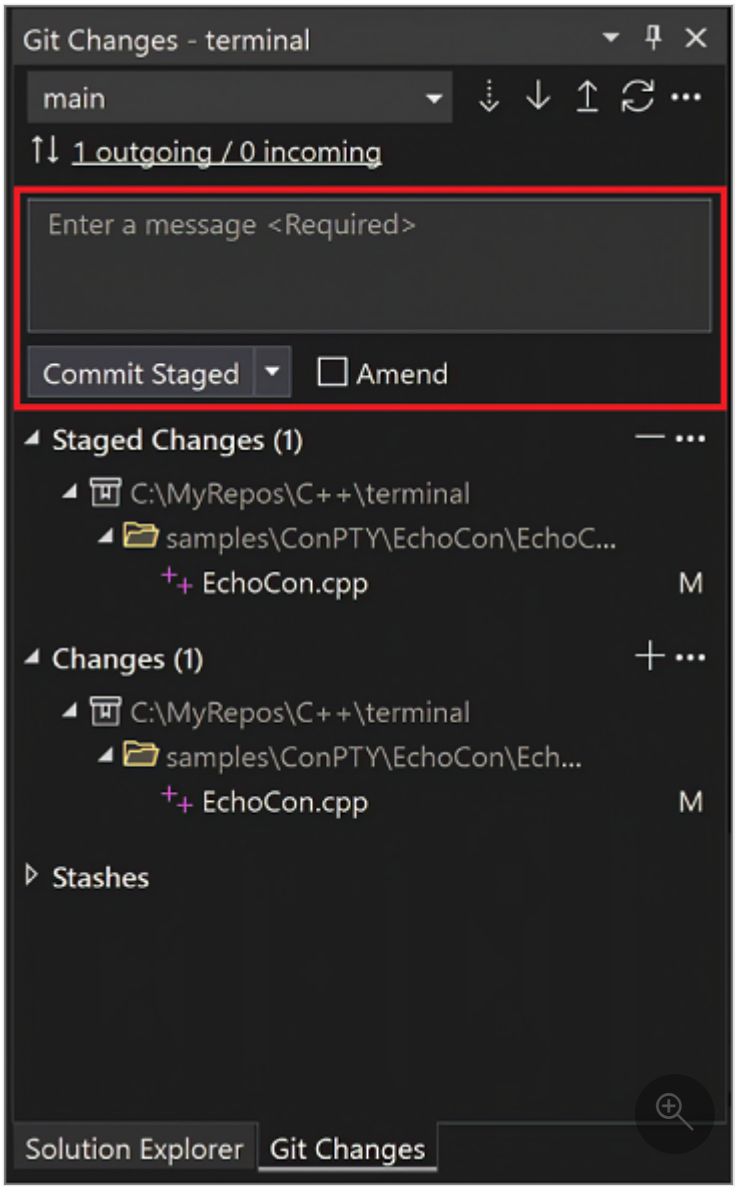


If you prefer a full-screen difference view, you can switch to the **Diff** editor. Here's how: select the **Promote to Document** button on the top-right corner of the Peek Difference UI. Or, you can use the **Ctrl**+**Alt**+**Home** keyboard shortcut. All the features that the Peek Difference UI provides are also available in the **Diff** editor.



Commit staged changes

After you've prepared your commit by reviewing and staging the changes you want to include, you can create your commit by using the **Git Changes** window. Next, enter a commit message and then select **Commit Staged**.



Next steps

To continue your journey, visit the Push to remote page.

See also

- Git experience in Visual Studio
- Visual Studio & GitHub: Better together

Push from Visual Studio to a remote branch

Article • 12/02/2022

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

After you've authenticated to GitHub, Visual Studio can improve your GitHub workflow. One of those improvements is the ability to push (also known as publish) a local project directly to GitHub with a single click. The final stage in a simple Git workflow is to push changes to your remote.

A remote is a safe place to store your code in the cloud. It's typically referred to as **origin/main** (or origin/master), where "origin" is the default name for a remote. For more information about this terminology, see the Git Branching - Remote Branches page on the Git website.

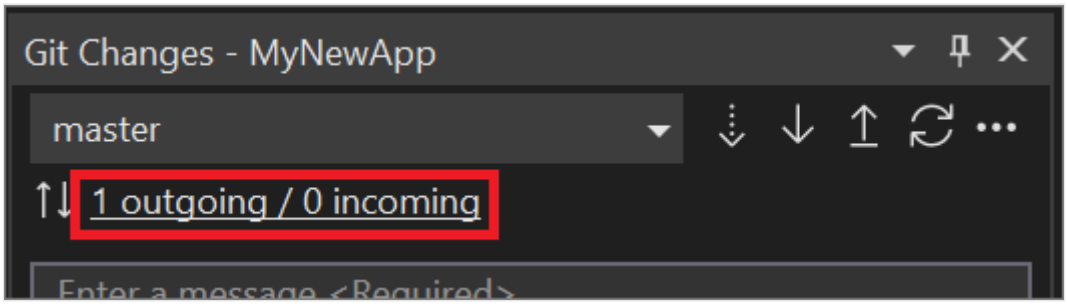
While this article references GitHub repositories, you can work remotely with the Git provider of your choice, such as GitHub, GitLab, or Azure DevOps.

Here's how to push to a remote in Visual Studio.

1. Make sure you've got a file open to work on that's in a previously created or cloned repo.

2. Make a change to the file, save it, select the **Git Changes** tab, and then commit the change.

3. In the **Git Changes** window, notice the link text that includes the number of incoming and outgoing commits. In the following example, the link text reads **1 outgoing / 0 incoming**.



   The "outgoing" text represents the number of commits that haven't yet been pushed to the remote, while the "incoming" text represents the commits that have been fetched but not yet pulled from the remote.

4. To push to your remote, select **Push** button, or select **Push** from the **Git** menu.

Next steps

To continue your journey, visit the Fetch, pull, and sync in Visual Studio page.

See also

- Git experience in Visual Studio
- Visual Studio & GitHub: Better together

Create a pull request in Visual Studio

Article • 10/25/2023

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

If you're authenticated to GitHub or Azure DevOps and have pushed changes to your remote repository, you can create a pull request without leaving Visual Studio. Pull requests are an essential tool in GitHub and Azure DevOps for code review. These reviews help catch issues in new features and allow teammates to share their experience with different regions of the codebase – improving the health of the codebase overall.

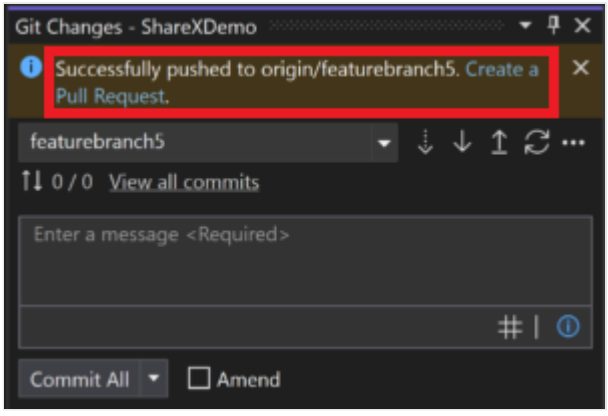
> **Note**
>
> To try out the feature, ensure it's enabled in **Tools** > **Options** > **Preview Features** > **Pull Request**

To create a pull request, follow these steps:

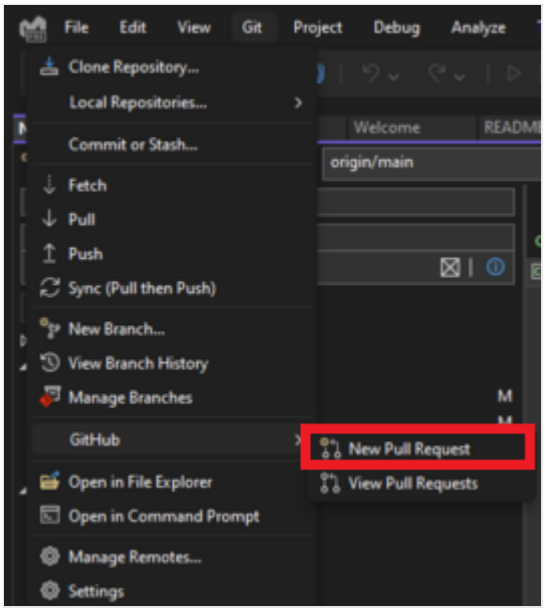
1. Before you can create a pull request, you need to create a new branch for your changes. This branch keeps your modifications separate from the main branch until they're ready to be merged.

2. Then, you can commit and push your changes as you normally would in the Git Changes window.

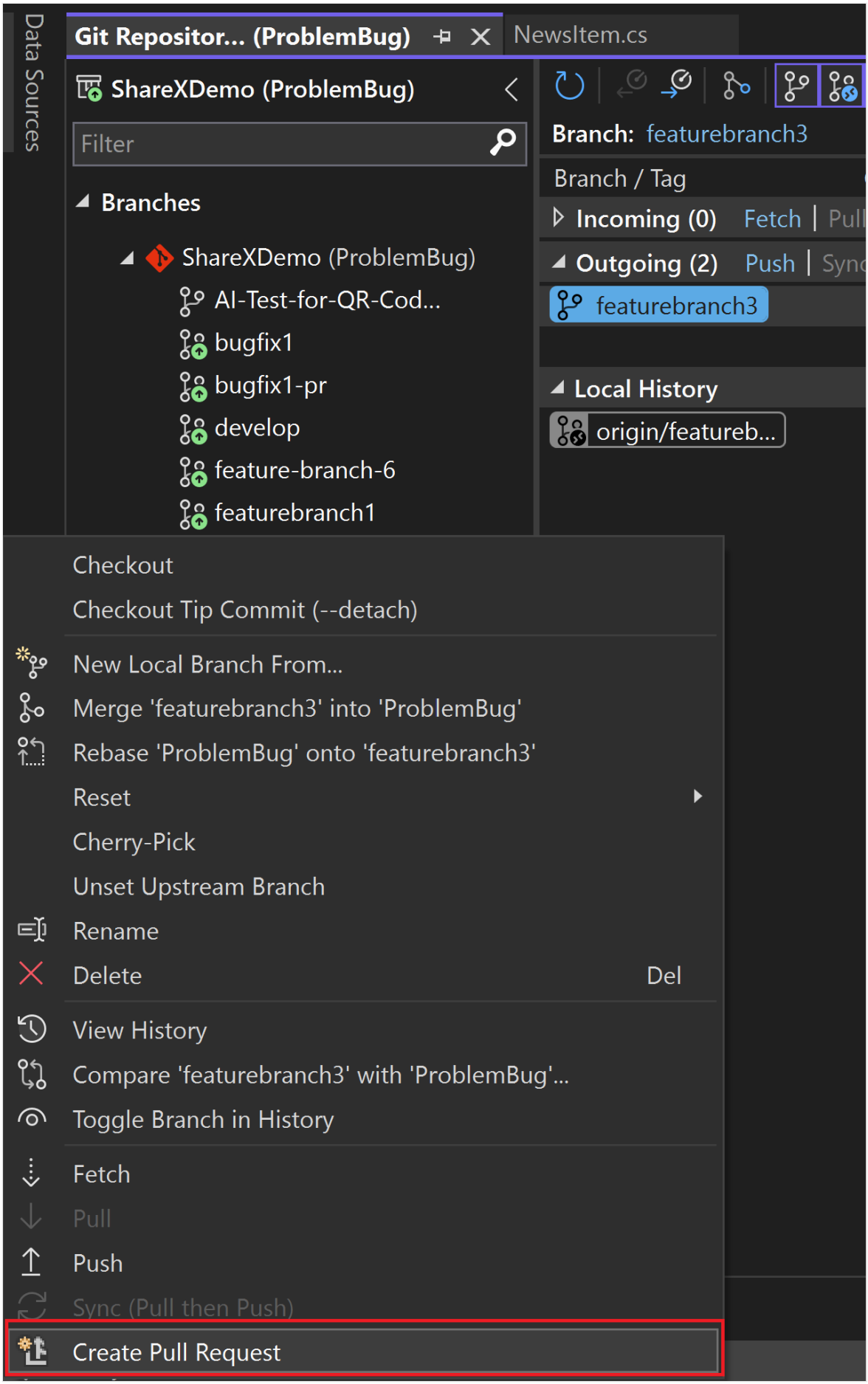
3. Select the link in the notification banner to **Create a Pull Request**.



   Alternatively, you can create a pull request from a remote branch by navigating to the **New Pull Request** window via the top-level menu **Git** > **GitHub** or **Azure DevOps** > **New Pull Request**. Or right-click a branch in the Git Repository Window.

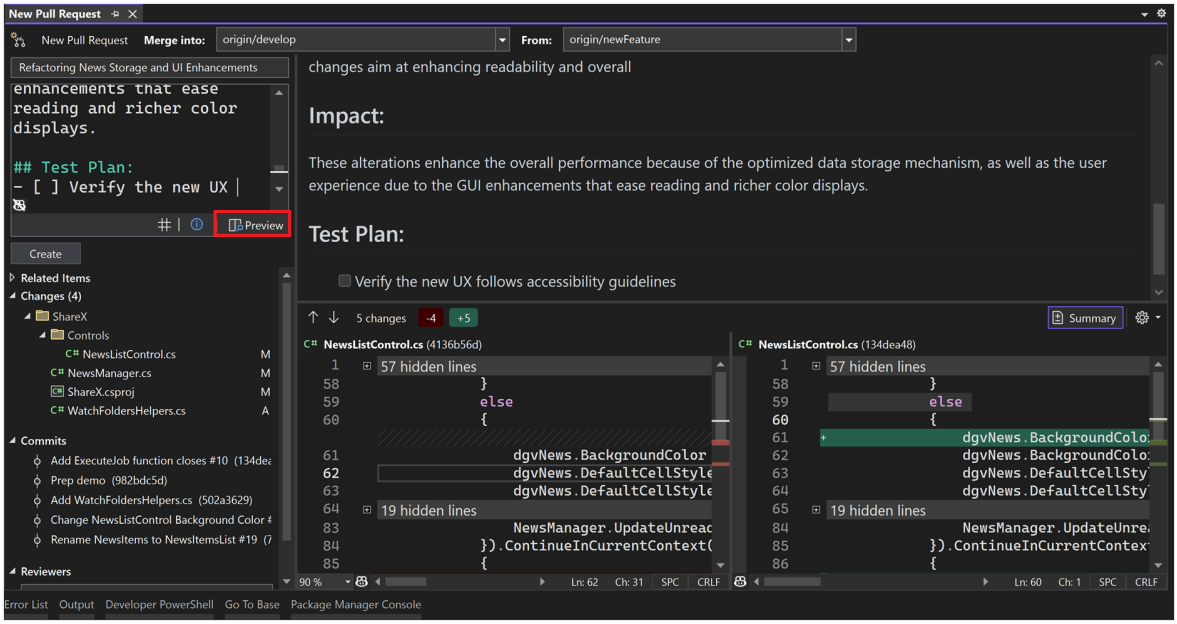


Or right select a branch in the Git Repository Window to open the context menu and select **New Pull Request**.



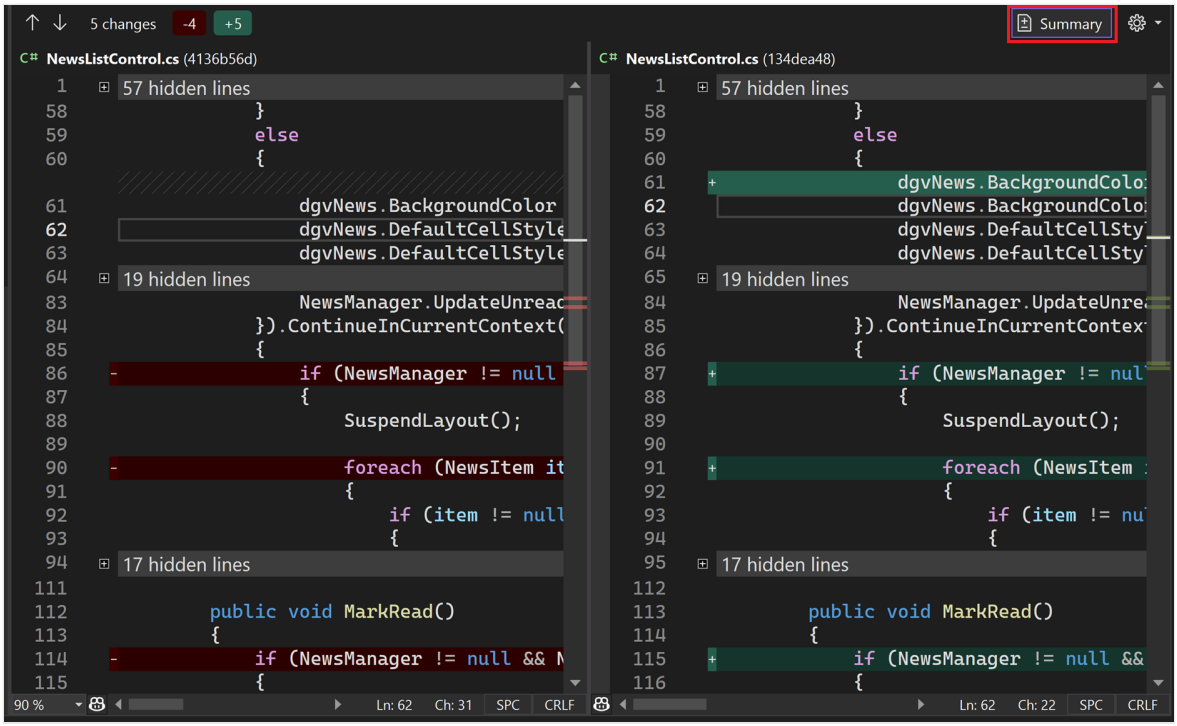
4. Select the branch that you want to merge into and give your pull request a descriptive title and description.

> **Tip**
>
> You can use **markdown syntax** to format your pull request description from Visual Studio. Try it out by typing the markdown symbols in the **Description** box and preview your markdown by selecting the **Preview** button.

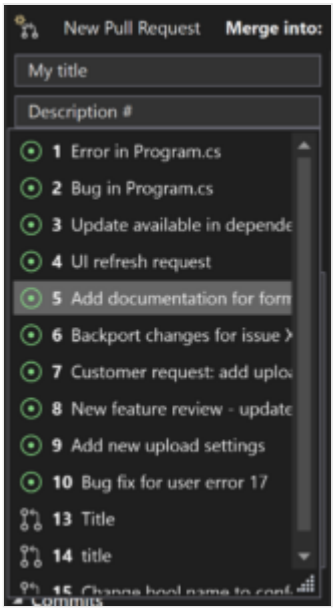


5. The difference preview allows you to see all the changes in the files that have been committed as a part of your pull request.

> **Tip**
>
> The summary difference view option allows you to review only the changed sections of code.



6. You can link work items by referencing them with the issue search, by typing `#` in the description box or pressing the # button in the lower right corner. You can also add your reviewers by typing their GitHub username or Azure DevOps identity or email.



7. Once you've clicked **Create** on your pull request, other developers can review your changes and provide feedback.

Next steps

To continue your journey, visit the Fetch, pull, and sync in Visual Studio page.

See also

- Git experience in Visual Studio
- Visual Studio & GitHub: Better together

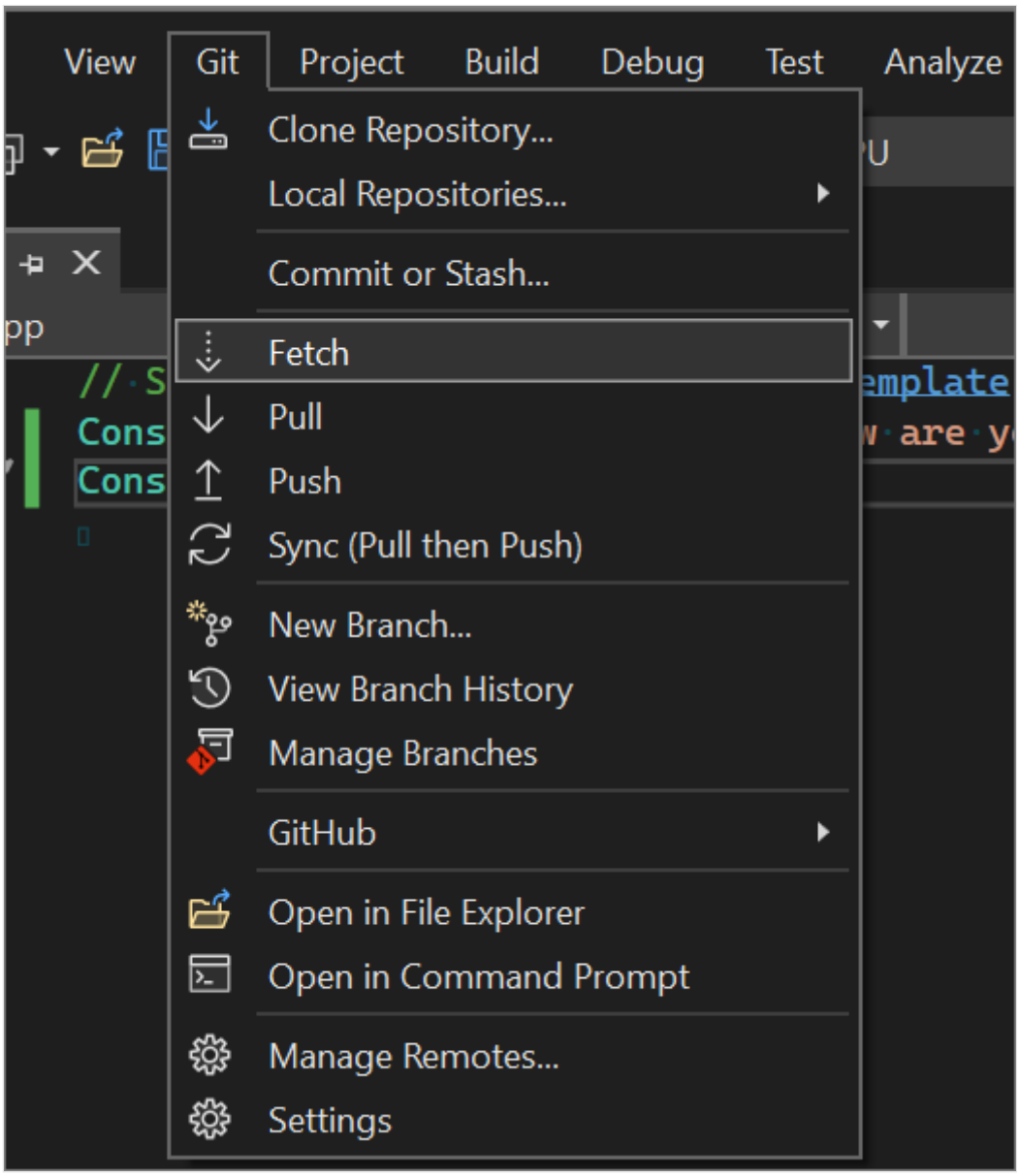
Use git fetch, pull, push and sync for version control in Visual Studio

Article • 09/12/2023

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

Visual Studio helps you keep your local branch synchronized with your remote branch through download (fetch and pull) and upload (push) operations.

You can fetch, pull, and sync in Visual Studio 2022 by using the **Git** menu.

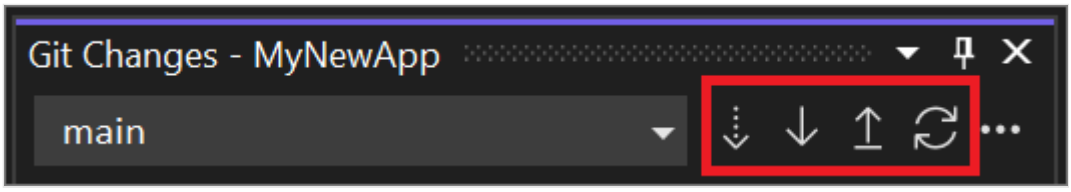


In the preceding screenshot, the **Fetch** option is highlighted. The Git menu also includes the following additional options:

- **Pull**
- **Push**

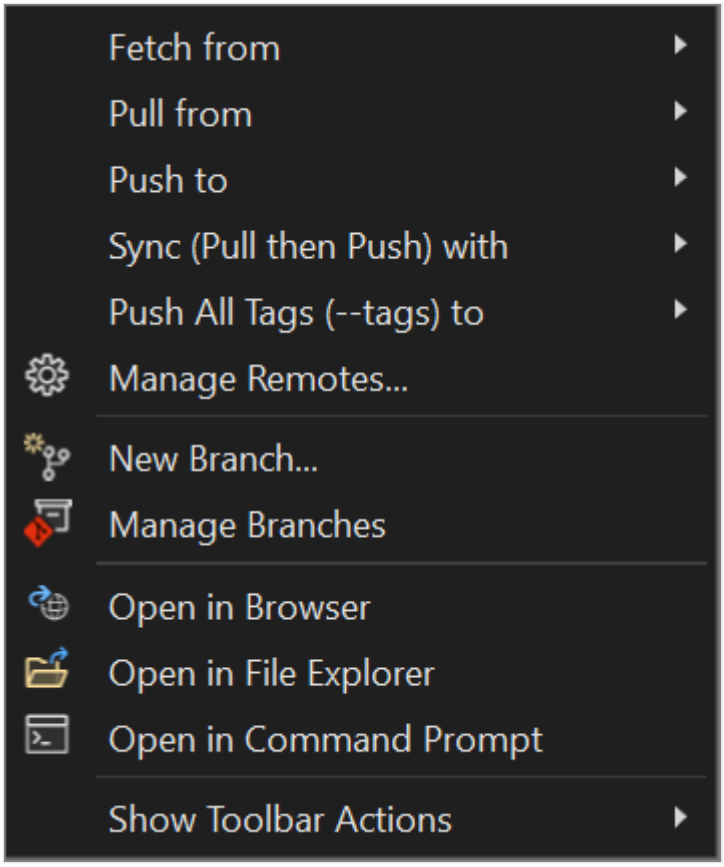
- **Sync (Pull then Push)**

You can also use the button controls in the **Git Changes** window to perform these operations, too.



From left to right, the button controls include **Fetch**, **Pull**, **Push**, and **Sync**.

Additionally, there's also an **ellipsis** (...) button control for more operations. When you select it, a context menu appears. You can use it to fine-tune your fetch, pull, push, and sync operations.



Fetch

It's important to fetch and pull before you push. Fetching checks if there are any remote commits that you should incorporate into your local changes. If you see any, pull first to prevent any upstream merge conflicts.

When you fetch a branch, the **Git Changes** window has an indicator under the branch drop-down, which displays the number of unpulled commits from the remote branch.

This indicator also shows you the number of unpushed local commits.

The indicator also functions as a link to take you to the commit history of that branch in the **Git Repository** window. The top of the history now displays the details of these incoming and outgoing commits. From here, you can also decide to Pull or Push the commits.

Pull

Always pull before you push. When you pull first, you can prevent upstream merge conflicts.

Push

When you create commits, you've inherently saved local snapshots of your code. Use **Push** to push the commits to GitHub, where you can store them as backups or share your code with others.

But, as previously mentioned, always pull before you push. As a safe guard, Visual Studio doesn't allow you to push commits if your local branch is behind the remote branch. If you try to push, a dialog prompts you to pull before pushing.

Sync

Use this operation to both pull, then push, sequentially.

Next steps

To continue your journey, visit the Browse Git repositories page.

See also

- Tutorial: Open a project from a repo
- Visual Studio & GitHub: Better together

Browse Git repositories and compare branches in Visual Studio

Article • 08/08/2023

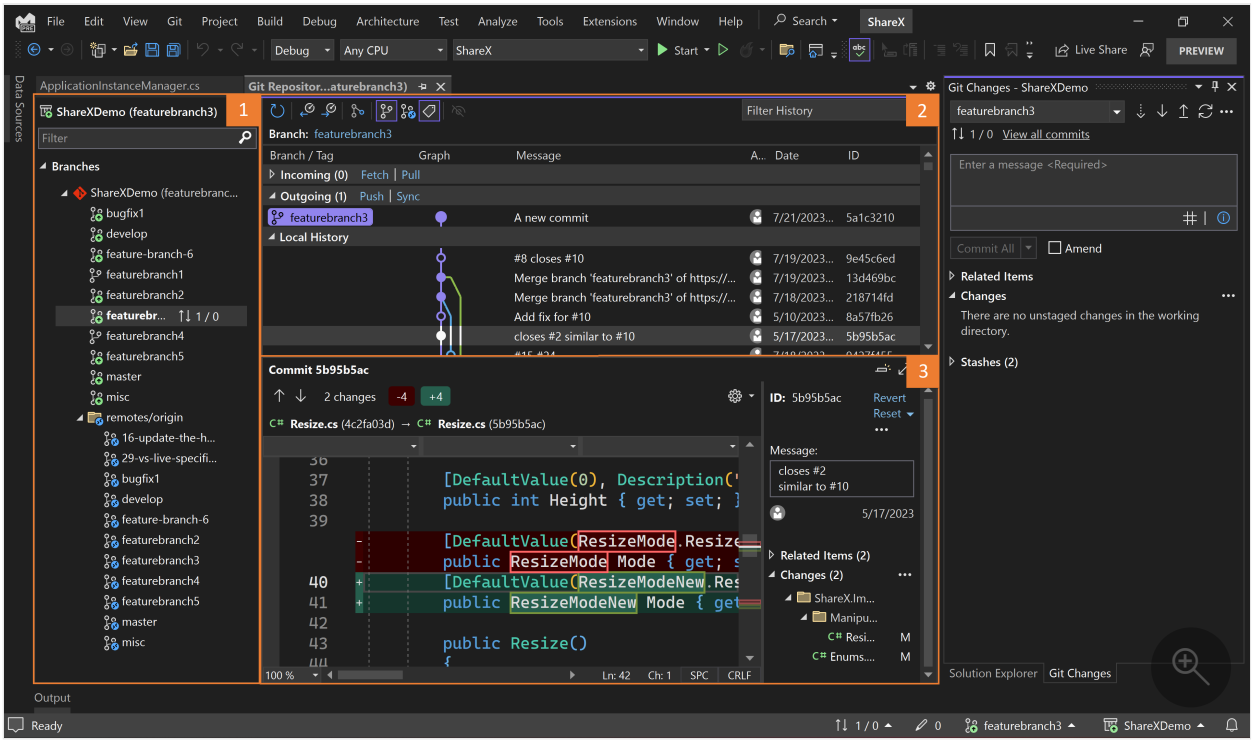
Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

The **Git Changes** window provides a seamless way to interact with Git while coding without having to switch away from your code. But there are times when it makes more sense to focus on your Git repository. For example, you might need to get a good picture of what your team has been working on or maybe compare two commits to investigate a bug.

You can work remotely with the Git provider of your choice, such as GitHub or Azure DevOps.

Browse through local and remote branches

To get started, open the **Git Repository** window by selecting **Git Repository** on the **View** menu (or by using the **Ctrl+0**, **Ctrl+R** keyboard shortcut). You can also access the **Git Repository** window by selecting the **outgoing/incoming** links in the **Git Changes** window and on the status bar.



The **Git Repository** window contains three main sections, as numbered in the preceding screenshot:

1. **Branches**: Git empowers users to multitask and experiment with their code through branches. If you're working on multiple features at the same time or if you want to explore ideas without affecting your working code, branching can be helpful.

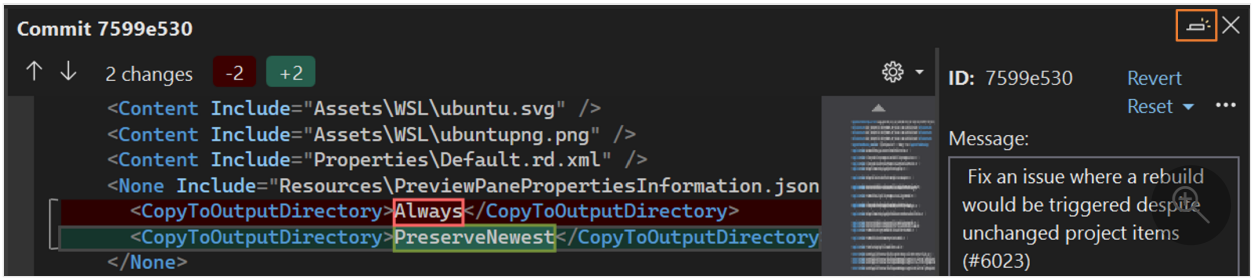
2. **Graph**: This section visualizes the state of your branch. It has three subsections:

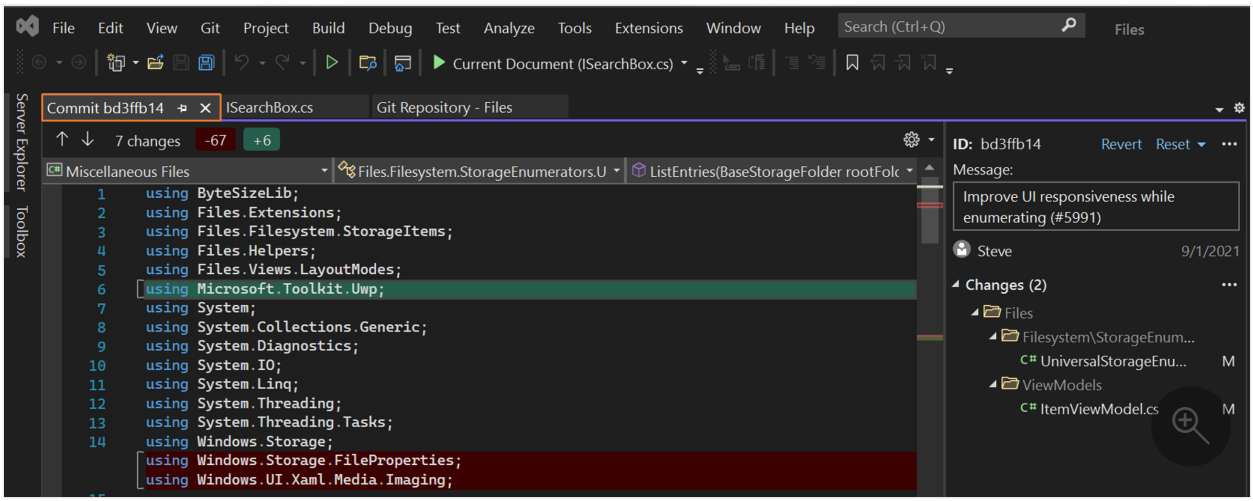
    - **Incoming** shows incoming commits that your team has been contributing.
    - **Outgoing** shows your local commits that you still haven't pushed.
    - **Local History** shows the rest of commits tracked by your local repository.

3. **Commit**: Selecting any commit in the **Graph** section opens its details. You can check the changes that a commit has introduced by selecting them, which shows a difference. For example, the previous screenshot shows the changes that one commit introduced to the *Resize.cs* file.

The **Alt+Up arrow** or **Alt+Down arrow** keyboard shortcuts allow you to jump between these sections.

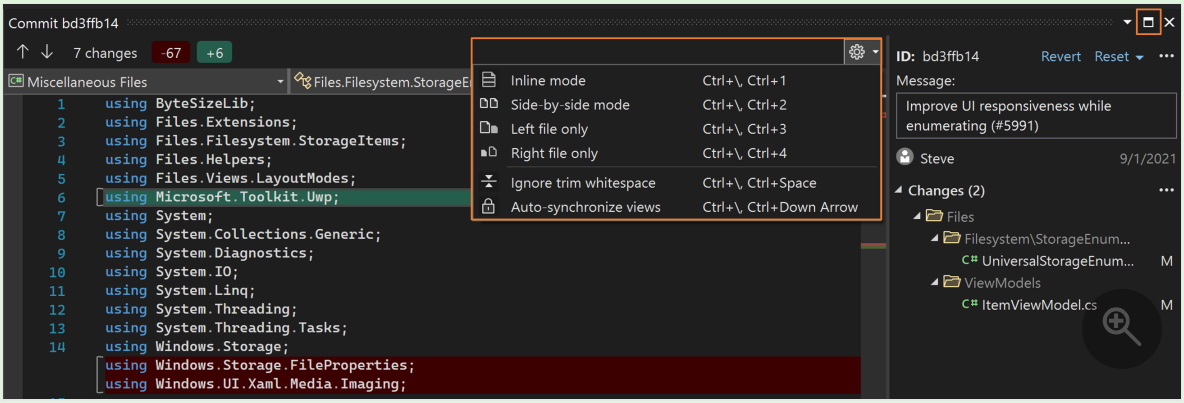
You can browse through any local or remote branch without having to switch your branch. When you find a commit that you want to focus on, select the **Open in New Tab** button to open the commit on a different tab.





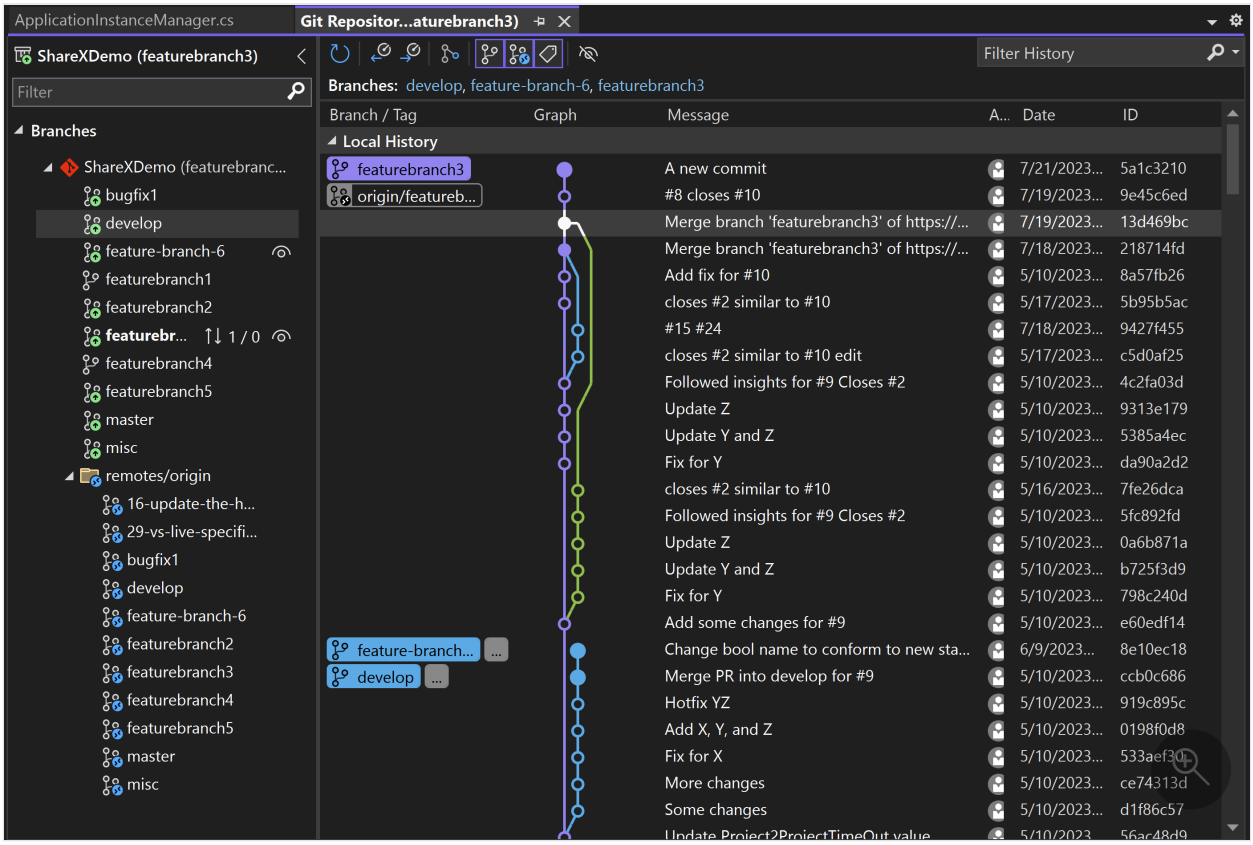
> **Tip**

To display your commit in full screen, detach your **Commit** tab and maximize the **Commit** window by using the **Maximize** button. You can also select your favorite diff configuration by selecting **Diff Configuration** (the gear icon).



Multi-branch graph view

New with version 17.7: When you open the Git Repo window, you start in the single branch view. You can then select any branch to add both the branch and its commits to a multi-branch graph view, which adds visual cues that make it more efficient to interact with multiple branches.

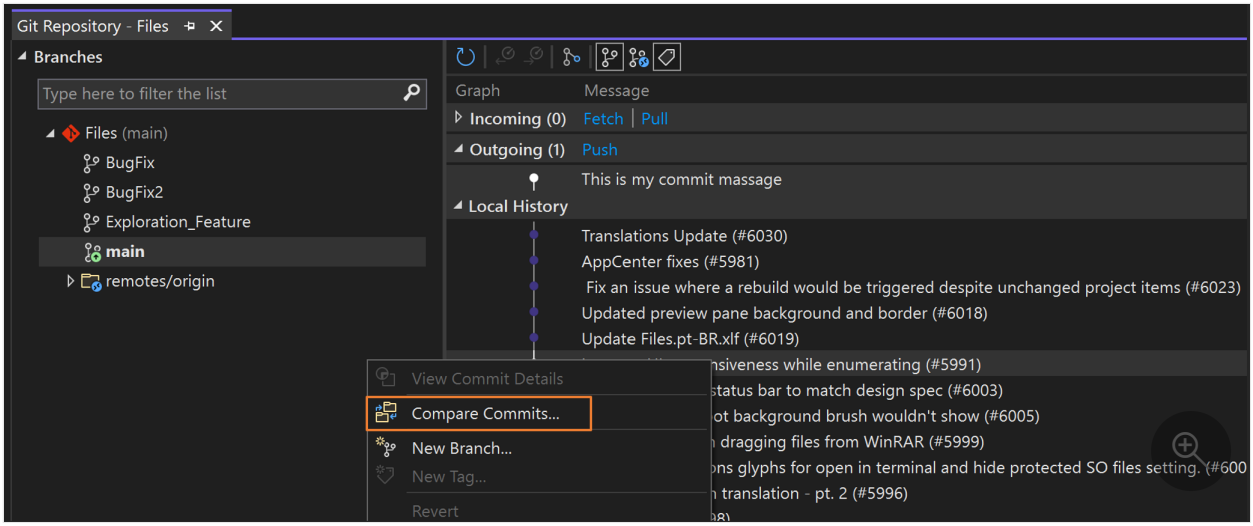


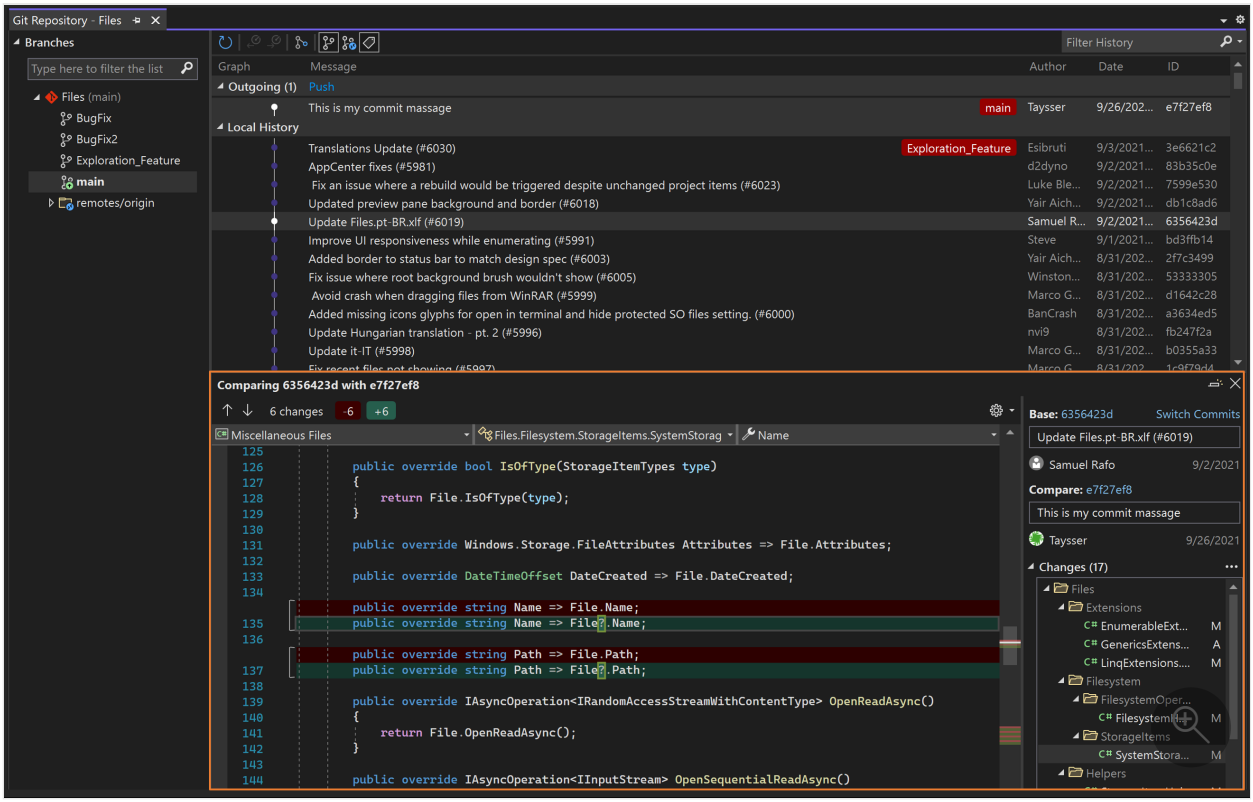
There's both line coloring and branch labels on the left side of the **Local History** view that help make it easier to trace which commits belong to each branch. You can use the

list of branches at the top of the table to scroll between branches more easily and know immediately which branches appear in the graph.

Compare commits

To compare any two commits in your branch, use the **Ctrl** key to select the two commits that you want to compare. Then, right-click one of them and select **Compare Commits**.

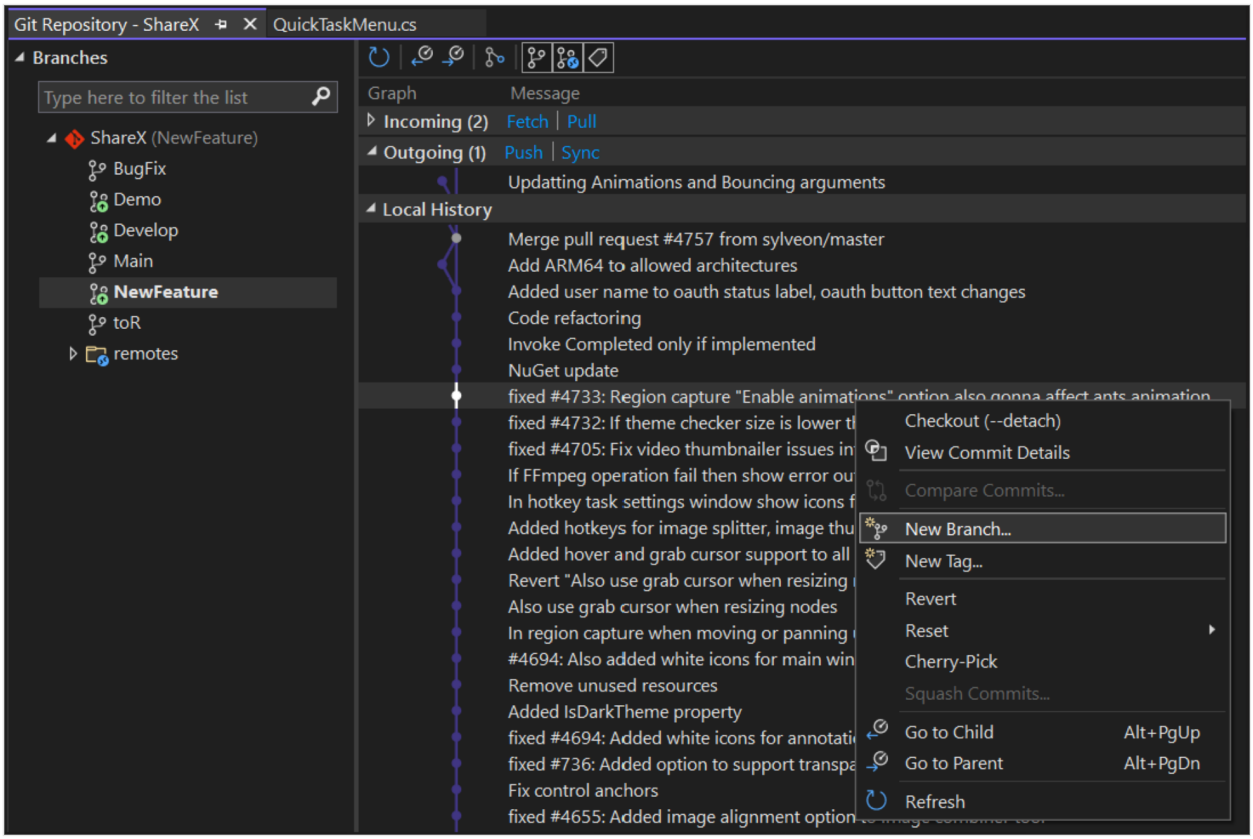




> **Tip**
>
> Similar to **Commit Details**, you can use the **Open in New Tab** button to open the comparison on a different tab or maximize it on the screen.

Create a branch from a commit

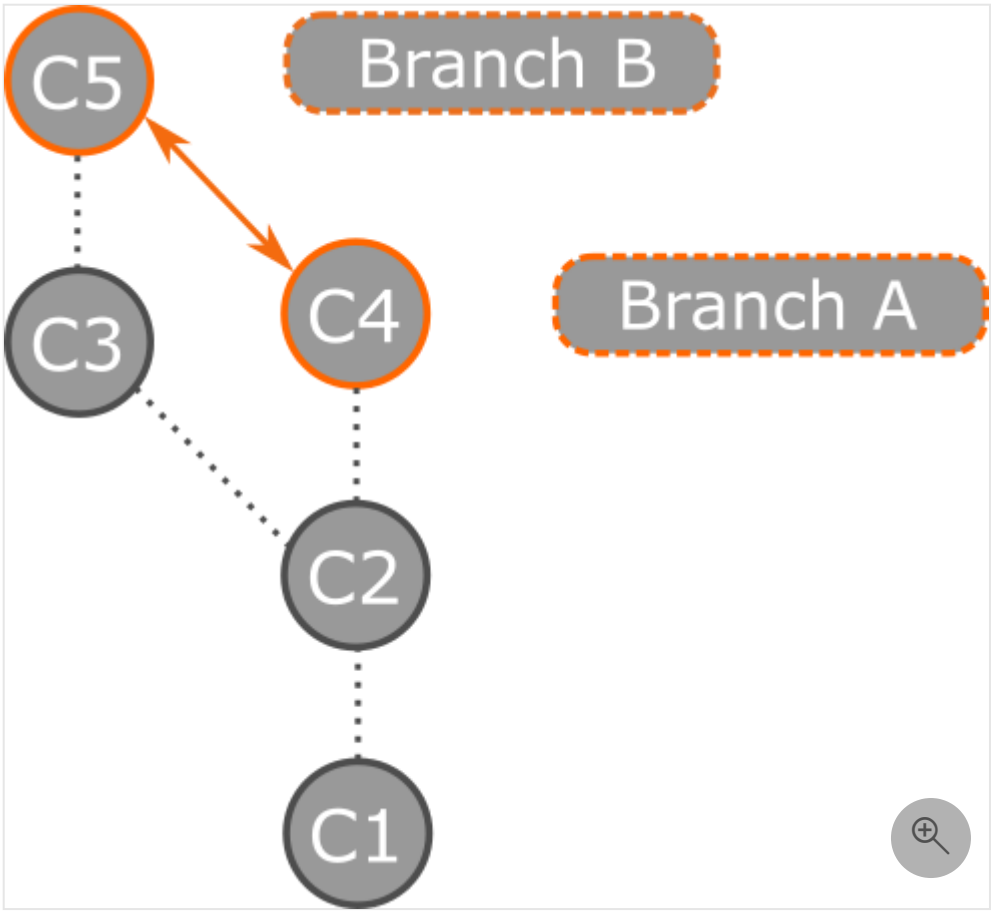
In Visual Studio, you can use the **Git Graph** pane in the **Git Repository** window to create branches from previous commits. To do so, right-click the commit you would like to create a new branch from, and then select **New Branch**.



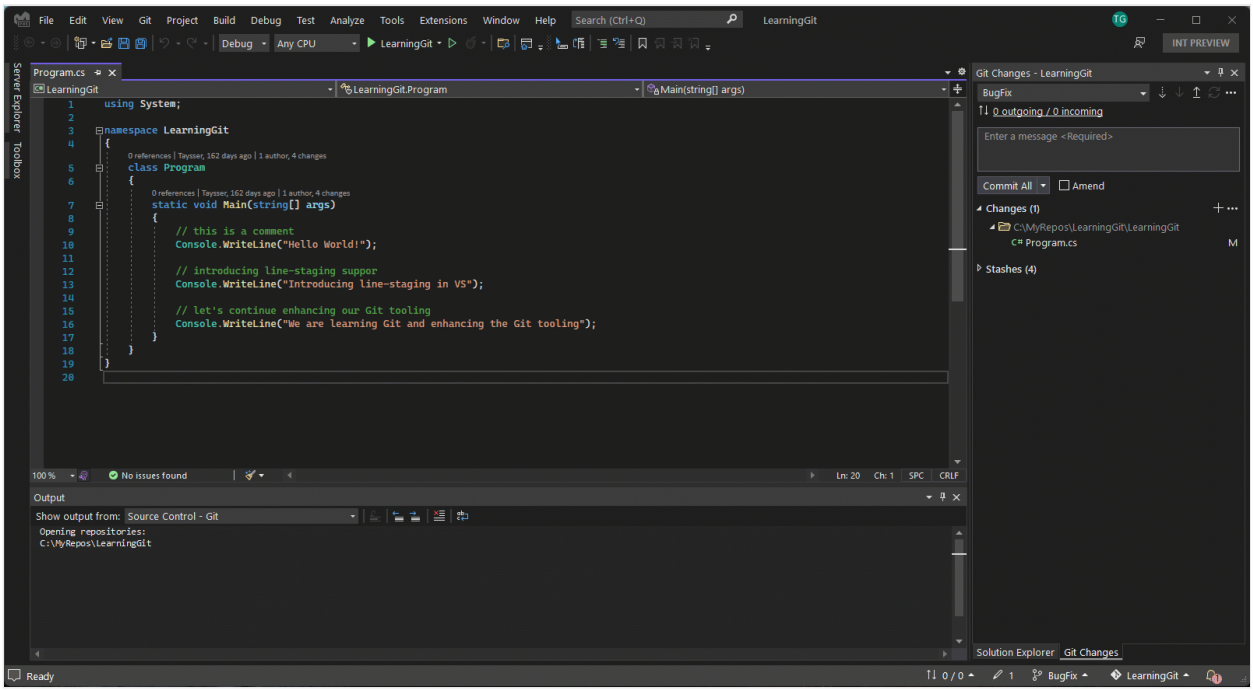
> ⓘ Note
>
> The equivalent command for this action is `git branch <branchname> [<commit-id>]`.

Compare branches

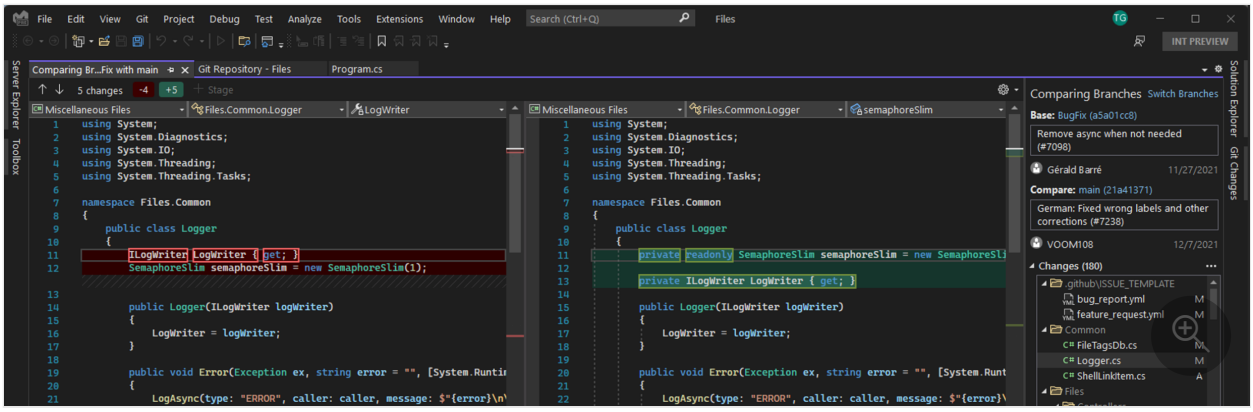
Comparing branches provides an overview of differences between two branches, which can be helpful before creating a pull request, merging, or even deleting a branch.



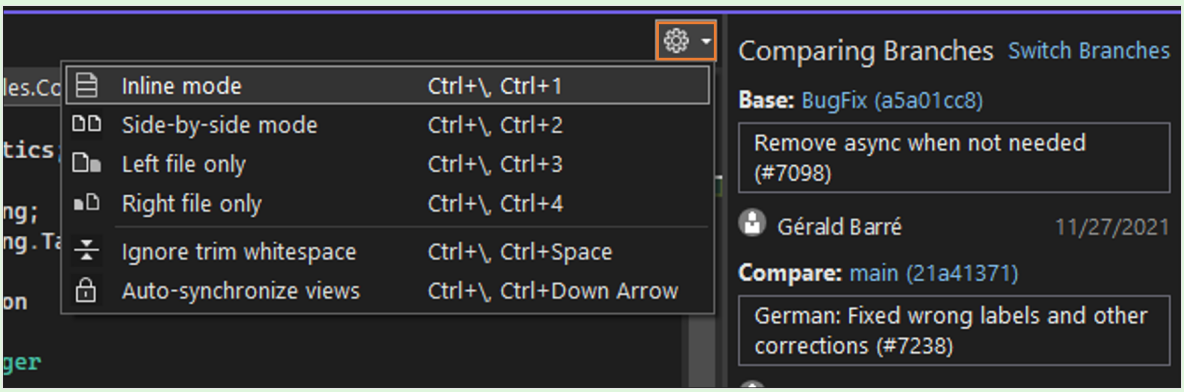
To compare your currently checked out branch with other branches using Visual Studio, you can utilize the branch picker hosted in the status bar and the Git changes tool window to choose any local or remote branch to compare with. Right-click the branch you're targeting and select **Compare with Current Branch**. Alternatively, you can utilize the branch list on the Git Repository window to access the same command.



Selecting **Compare with Current Branch** opens the compare branches experience where you can navigate the **Changes** list and select the file that you would like to compare.

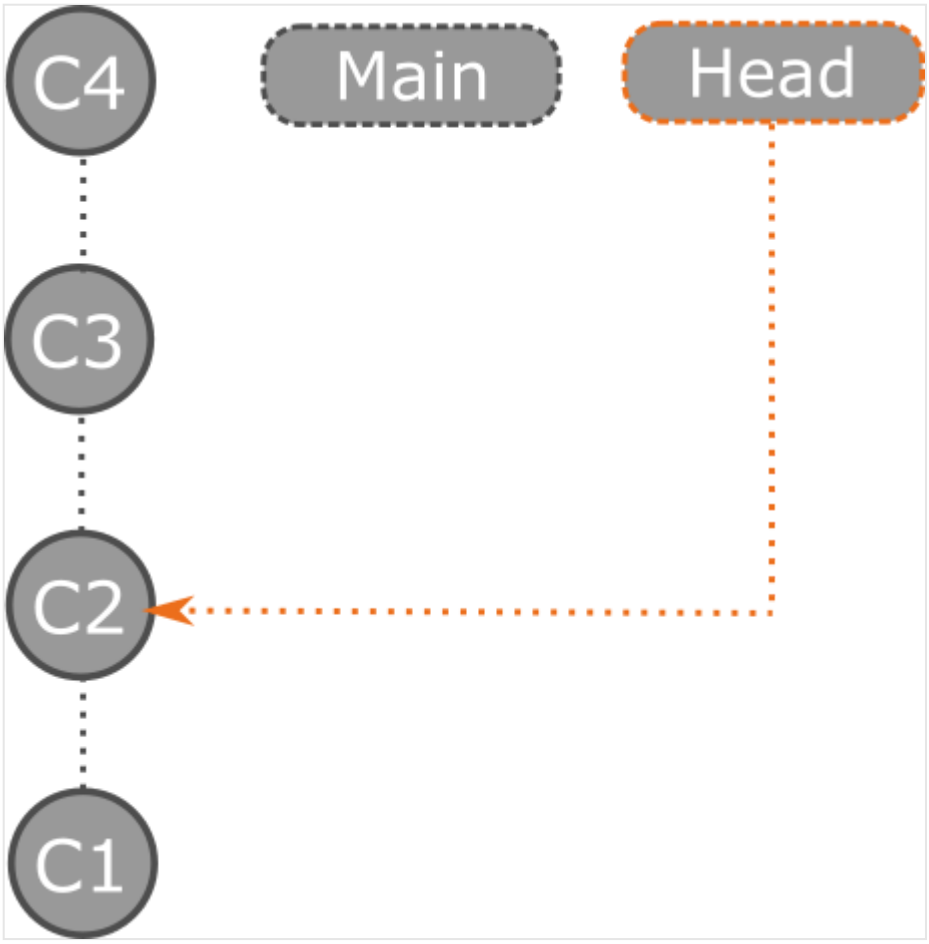


> **Tip**
>
> If you prefer an inline diff, you can use the Diff Configuration Options gear icon and switch to an inline diff view.

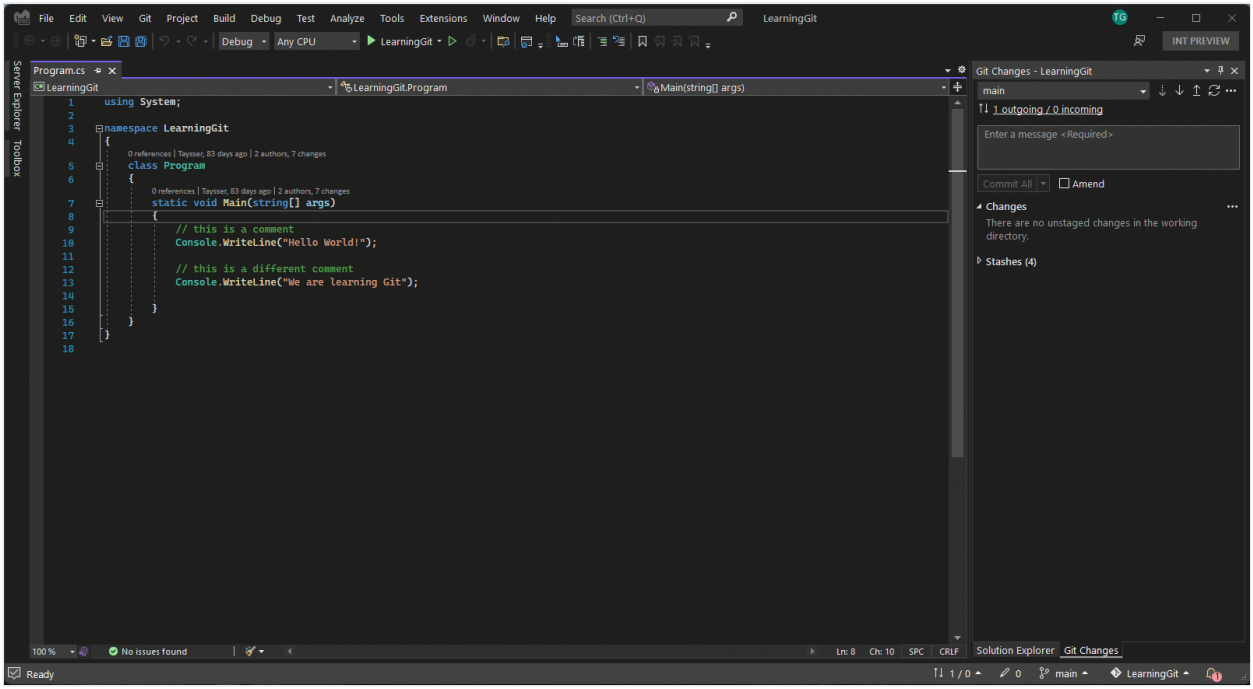


Checkout commits

Checking out a commit can be beneficial in multiple ways. For example, it allows you to go back to a previous point in your repository's history where you can run or test your code. It can also be helpful if you would like to review code from a remote branch (a colleague's branch, for example). That way you don't need to create a local branch if you aren't planning on contributing to it. In this case, you can just check out the tip of the remote branch that you would like to review.

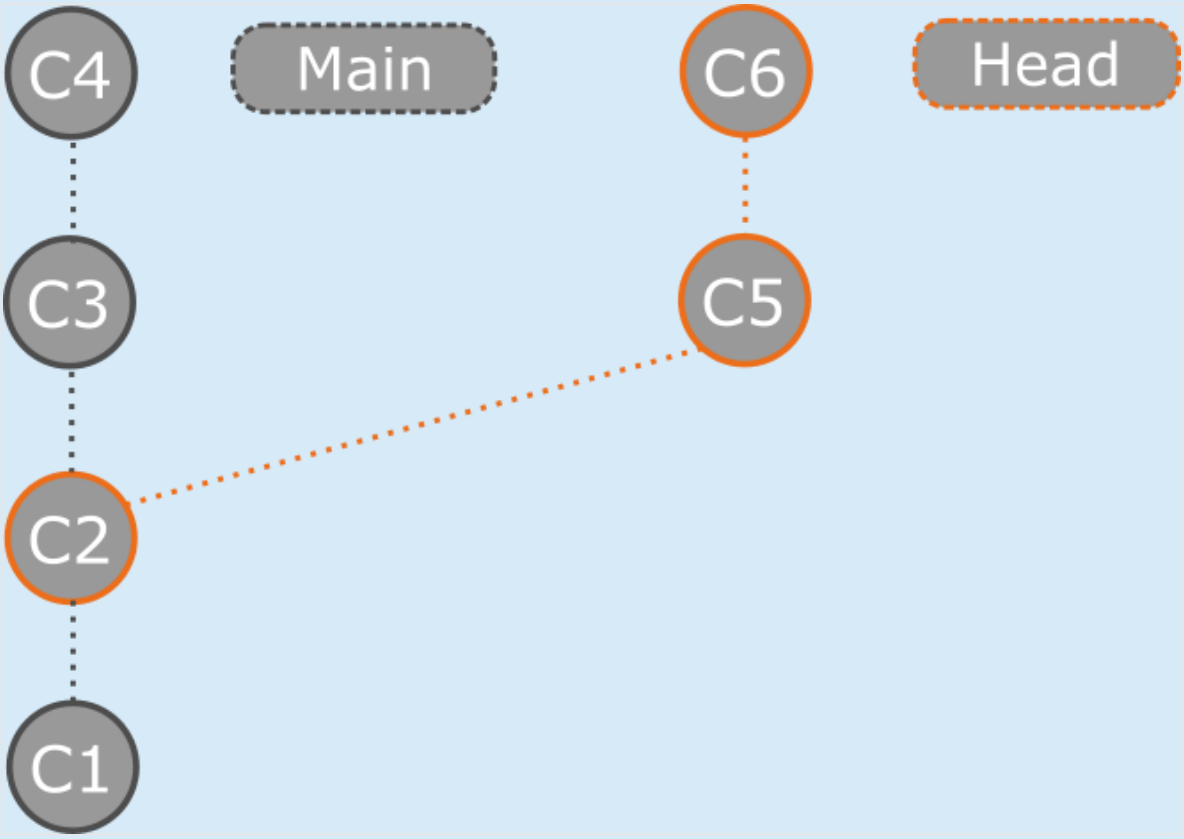


To check out a previous commit in Visual Studio, open the **Git Repository** window, right-click the commit you would like to go back to and select **checkout (–detach)**. Visual Studio shows a confirmation dialog explaining that by checking out a commit, you'll be in a detached HEAD state. Meaning that the HEAD of your repository is going to point directly to a commit instead of a branch.



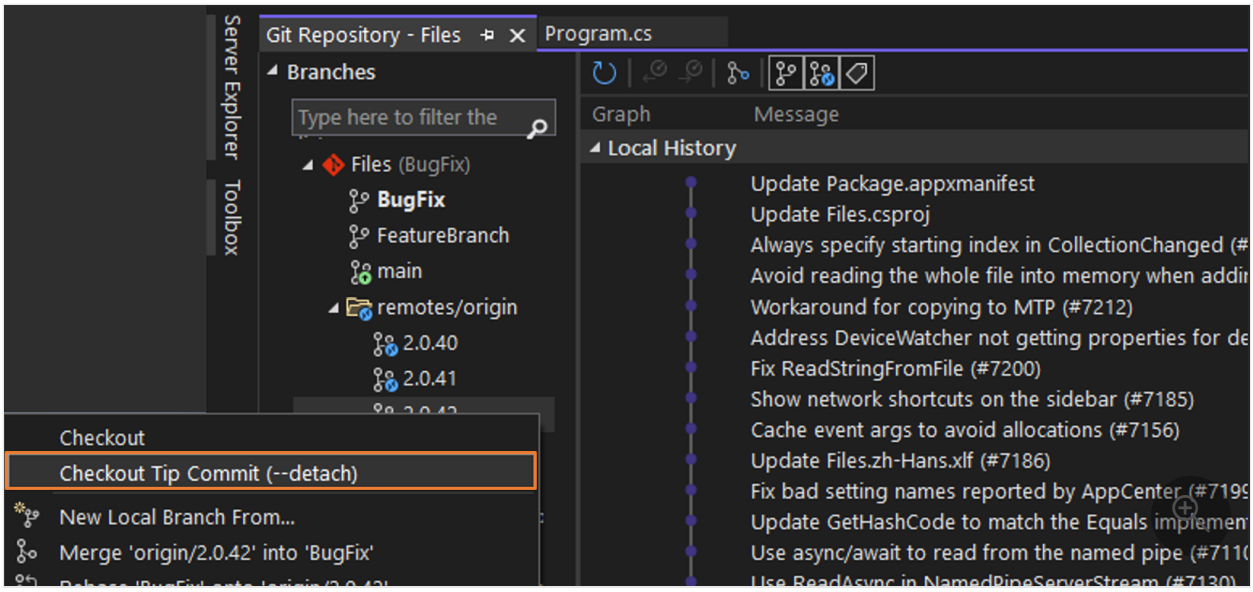
Now that you are in a detached head state, feel free to run and test your code or even explore and commit changes. When you're done exploring and want to go back to your branch, you can choose to discard your changes by checking out an existing branch or choose to keep your changes by **creating a new branch** first.

> **ⓘ Important**
>
> Commits created in a detached head state are not associated with any branch and may be garbage collected by Git after you check out a branch. That is why to keep your changes, it is recommended to create a new branch before checking out a branch. For example, commits C5 and C6 will be garbage collected if we check out Main without creating a new branch.



To learn more about the detached head state, see the **Detached Head** Git documentation.

Checking out the tip of a remote branch can be helpful if you would like to quickly review a pull request and evaluate the latest updates. To do that in Visual Studio, first make sure to fetch and get the latest updates from your remote repository. Then right-click the remote branch you would like to review and select **Checkout Tip Commit**.



Next steps

To continue your journey, see Manage Git repositories in Visual Studio.

See also

- The Git experience in Visual Studio
- Visual Studio and GitHub: Better together

Manage Git repositories in Visual Studio

Article • 11/23/2022

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

The **Git Repository** window provides a full-screen Git experience that helps you manage your Git repository and stay up to date with your team's projects. For example, you might need to reset, revert, or cherry-pick commits, or just clean your commit history. The **Git Repository** window is also a great place to visualize and manage your branches.

Version control with Visual Studio is easy with Git. And, you can work remotely with the Git provider of your choice, such as GitHub or Azure DevOps. Or, you can work locally with no provider at all.

Change the last commit (amend)

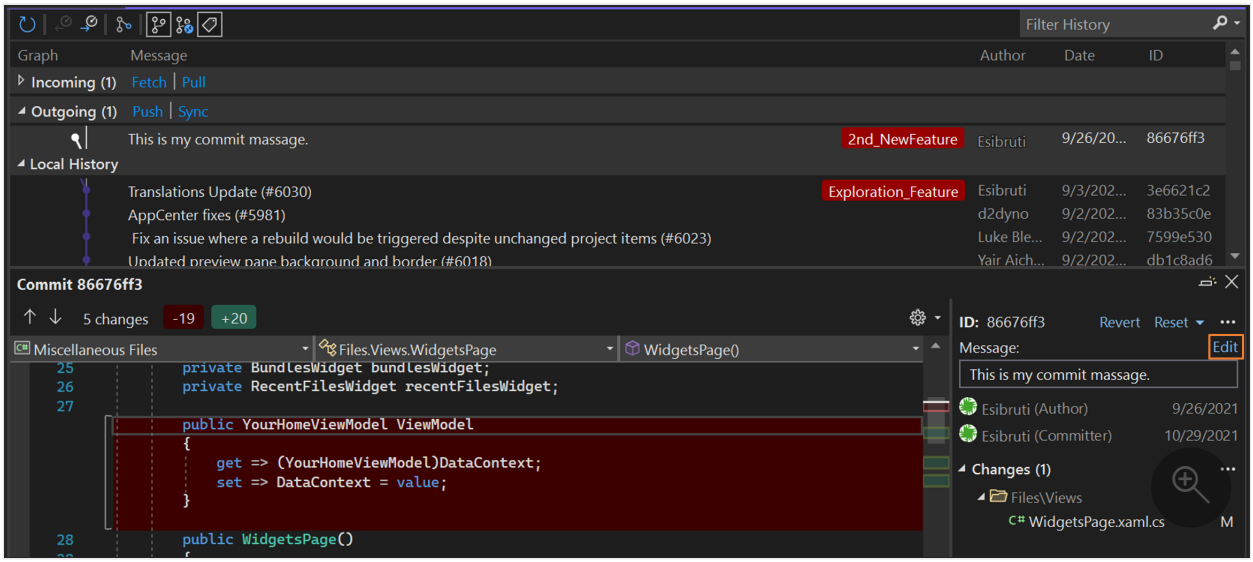
Updating the last commit is called *amending* in Git, and it's a common use case. Sometimes you just need to update your commit message, or you might need to include a last-minute change.

You can amend a commit on the command line by using the following command:

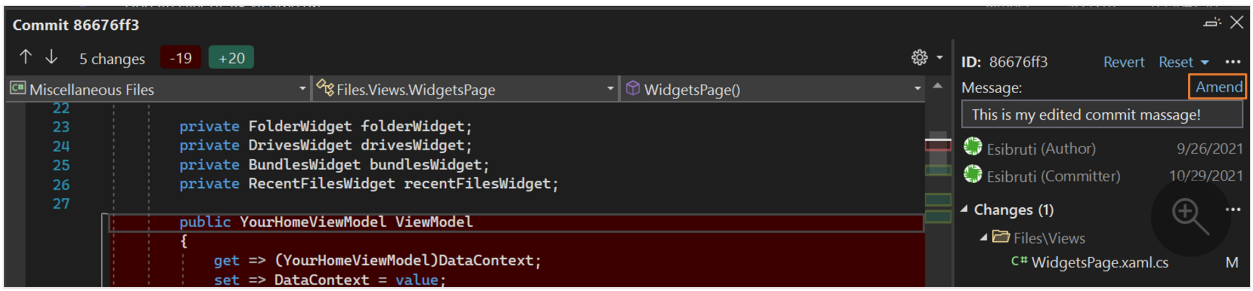
Bash

```
git commit --amend
```

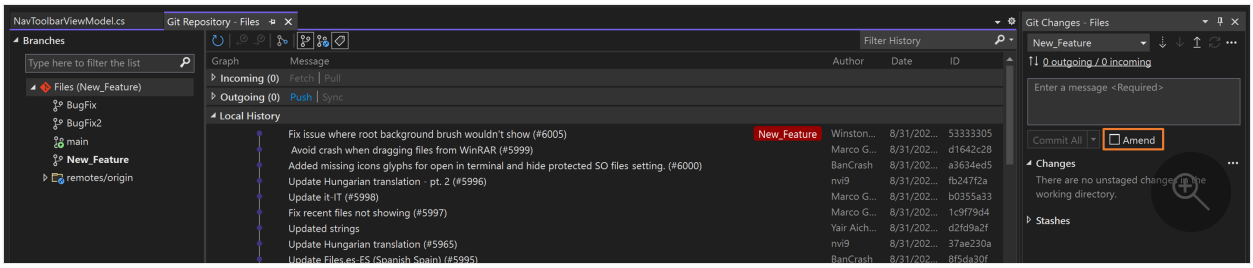
The **Git Repository** window makes it easy to update your commit message. Open the commit details of the last commit by double-clicking it, and then select the **Edit** option next to the commit message.



When you finish editing your commit message, select **Amend**.



If you need to include code changes to your last commit, you can do that in the **Git Changes** window. Select the **Amend** checkbox and then commit your changes.



To learn more about amending, see Git Tools - Rewriting History on the Git website.

Merge commits (squash)

To merge a series of commits, Git provides an option to squash commits down into a single commit. This option can be helpful if you make frequent commits and end up with a long list of commits that you want to clean up before pushing to a remote repository.

You can squash two commits on the command line by using the following command:

Bash

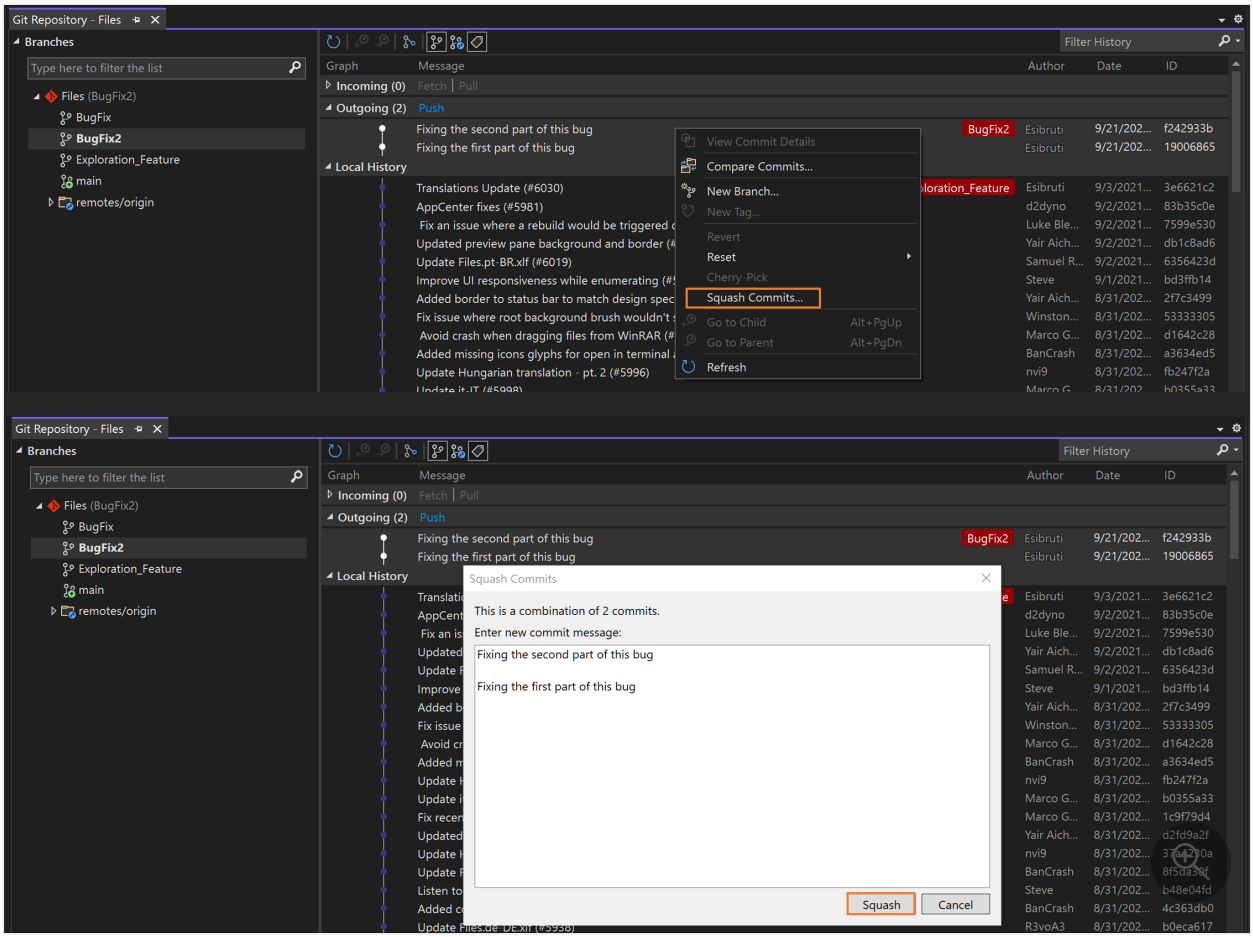
```
git rebase -i HEAD~2
```

Then update `pick` to `squash`, save, and update the commit message.

```
pick 19006865 Fixing the first part of this bug
squash f242933b Fixing the second part of this bug

# Rebase 3e6621c2..f242933b onto 3e6621c2 (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
```

To merge commits in Visual Studio, use the **Ctrl** key to select multiple commits that you want to merge. Then right-click and select **Squash Commits**. Visual Studio automatically combines your commit messages, but sometimes it's better to provide an updated message. After you review and update your commit message, select the **Squash** button.



To learn more about squashing, see Git Tools - Rewriting History on the Git website.

Merge and rebase branches

If you're using Git branches to work on different features, at some point you'll need to include updates introduced to other branches. This can happen while you're still working on your feature branch. It can also happen when you're done working on your

feature branch and need to keep your changes by adding them to a different branch. In Git, you can include these updates by merging or rebasing branches.

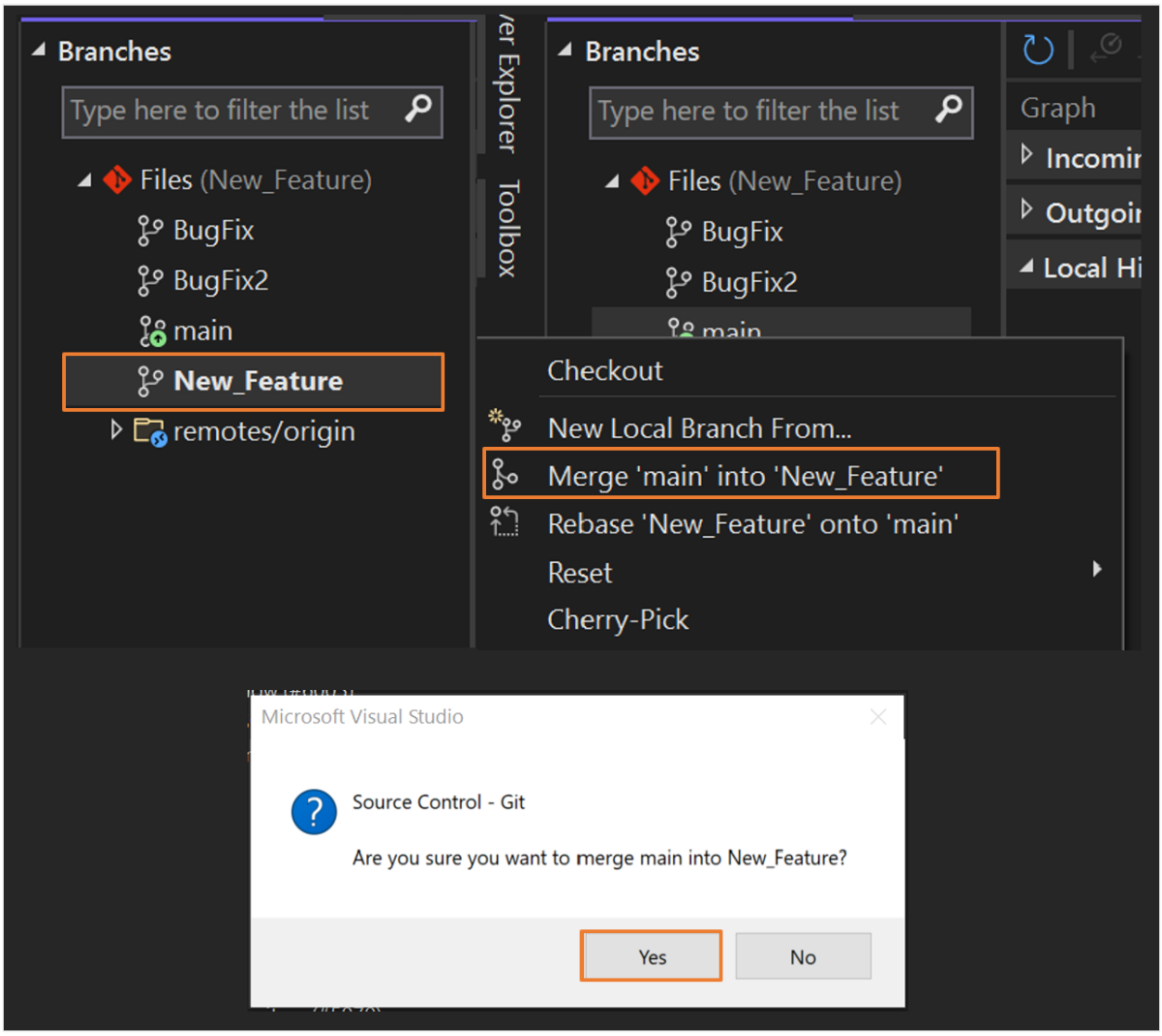
> **Note**
>
> The following instructions use *New_Feature* as an example name for a feature branch. Replace it with the name of your own branch.

To merge the main branch into your feature branch on the command line, use the following commands:

Bash

```
git checkout New_Feature
git merge main
```

To do the same in Visual Studio, check out the feature branch by double-clicking it in the branch list. Then right-click **main** and select **Merge 'main' into 'New_Feature'**.

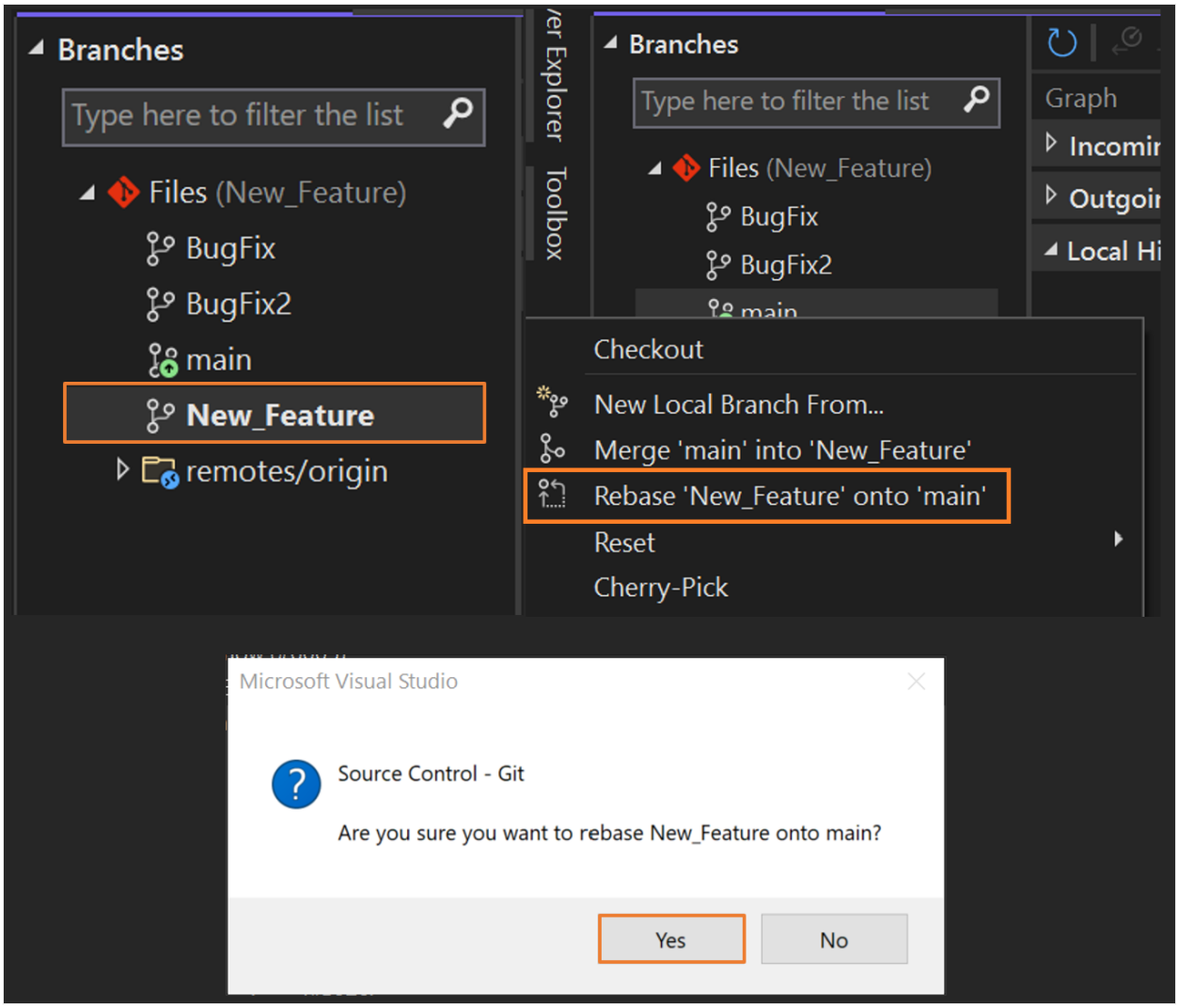


To rebase the main branch into your feature branch on the command line, use the following commands:

```
Bash

git checkout New_Feature
git rebase main
```

To do the same in Visual Studio, check out the feature branch by double-clicking it in the branch list. Then right-click **main** and select **Rebase 'New_Feature' onto 'main'**.



To learn more about merging, rebasing, and branching in general, see Git Branching on the Git website.

Copy commits (cherry-pick)

Copy commits from one branch to another by using the cherry-pick option. Unlike a merge or rebase, cherry-picking brings only the changes from the commits that you select, instead of all the changes in a branch. Cherry-picking is a great way to tackle these common problems:

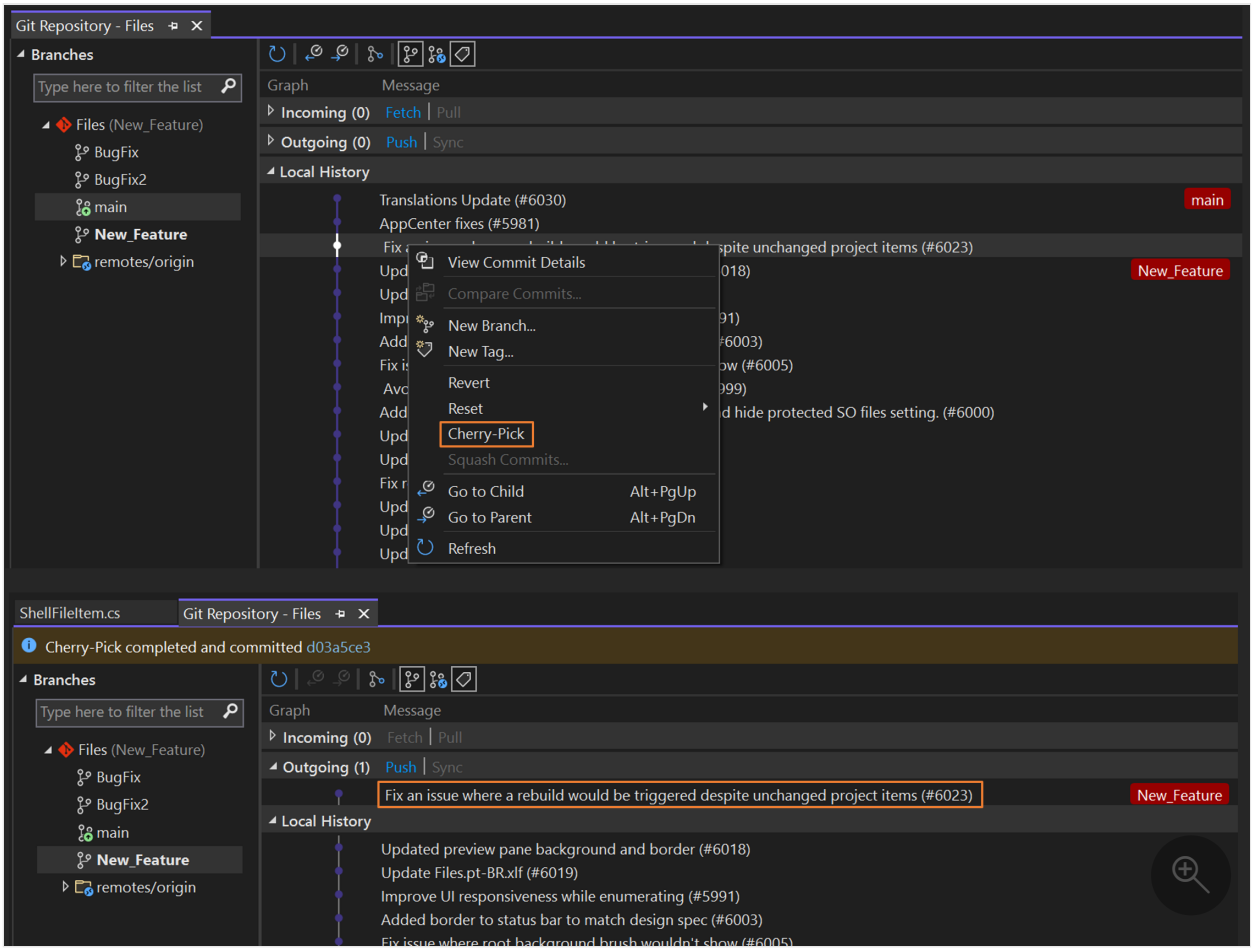
- Accidentally committing in the wrong branch. Cherry-pick the changes over to the correct branch and then reset the original branch to the previous commit.
- Pulling out a set of commits made in a feature branch, so you merge them back to your main branch sooner.
- Porting in specific commits from the main branch without rebasing your branch.

To copy changes from a commit to your current branch by using the command line, use the following command:

```
Bash

git cherry-pick 7599e530
```

To do the same in Visual Studio, preview the branch that you want to cherry-pick a commit from by selecting it with a single click. Then right-click the targeted commit and select **Cherry-Pick**.



When the operation is complete, Visual Studio shows a success message. The commit that you cherry-picked appears in the **Outgoing** section.

To learn more about cherry-picking commits, see the Git webpage for the cherry-pick command.

Revert changes

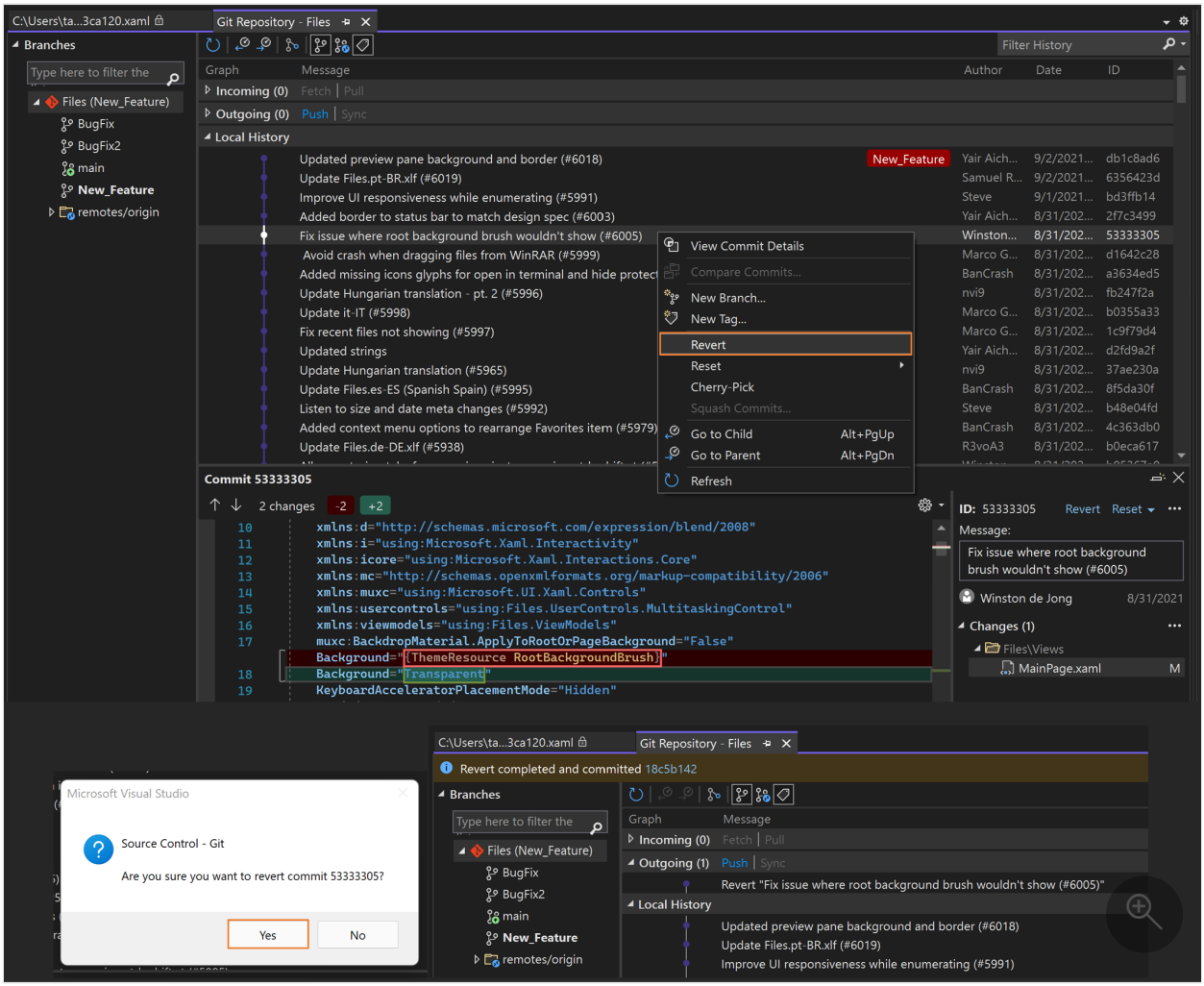
Use the revert command to undo the changes made in commits pushed to shared branches. The revert command creates a new commit that undoes the changes made on a previous commit. The revert command doesn't rewrite the repository history, which makes it safe to use when you're working with others.

To revert changes made in a commit by using the command line, use the following commands. Replace the example ID with the ID of a real commit in your branch.

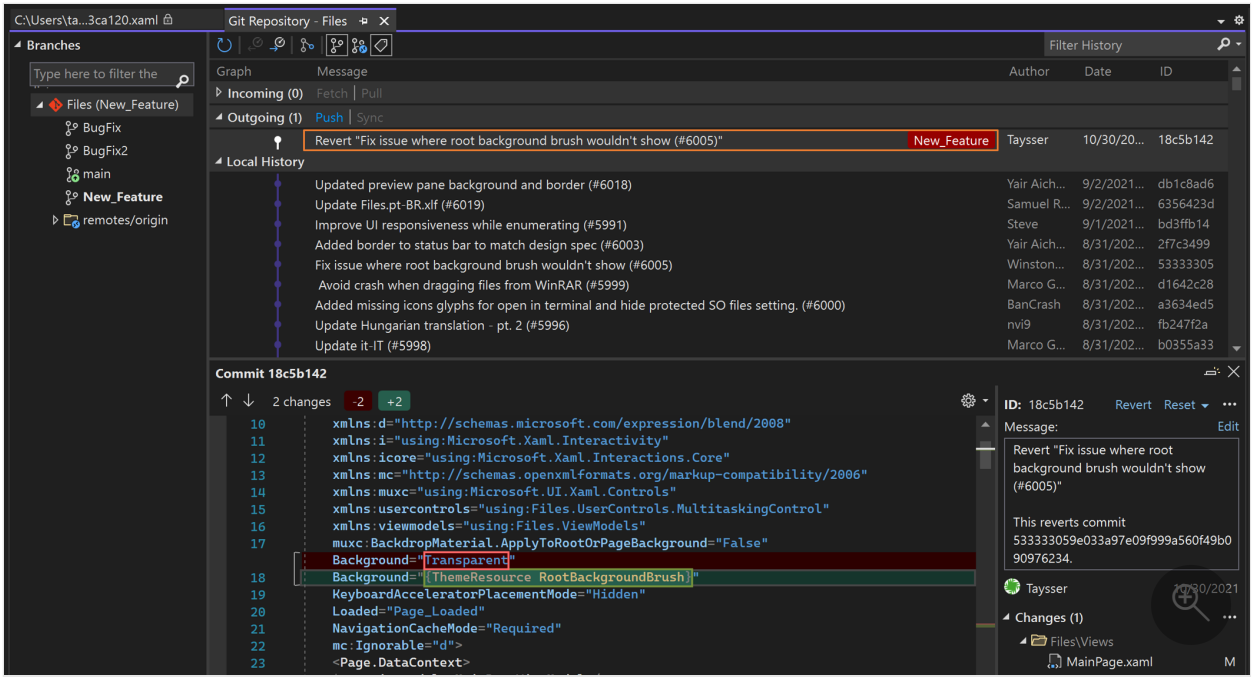
Bash

```
git revert 53333305
git commit
```

In the previous example, the commands will undo the changes made in commit 53333305 and create a new commit on the branch. The original commit is still in the Git history. To do the same in Visual Studio, right-click the commit you want to revert and then select **Revert**. After you confirm your action and the operation is complete, Visual Studio displays a success message and a new commit appears in the **Outgoing** section.



Select the new commit to confirm that it undoes the changes of the reverted commit.



To learn more about reverting changes, see the Git webpage for the revert command.

Reset a branch to a previous state

Use the reset command to bring a branch in your local repository back to the contents of a previous commit. This action discards all changes that have happened since the commit that you're resetting your branch to.

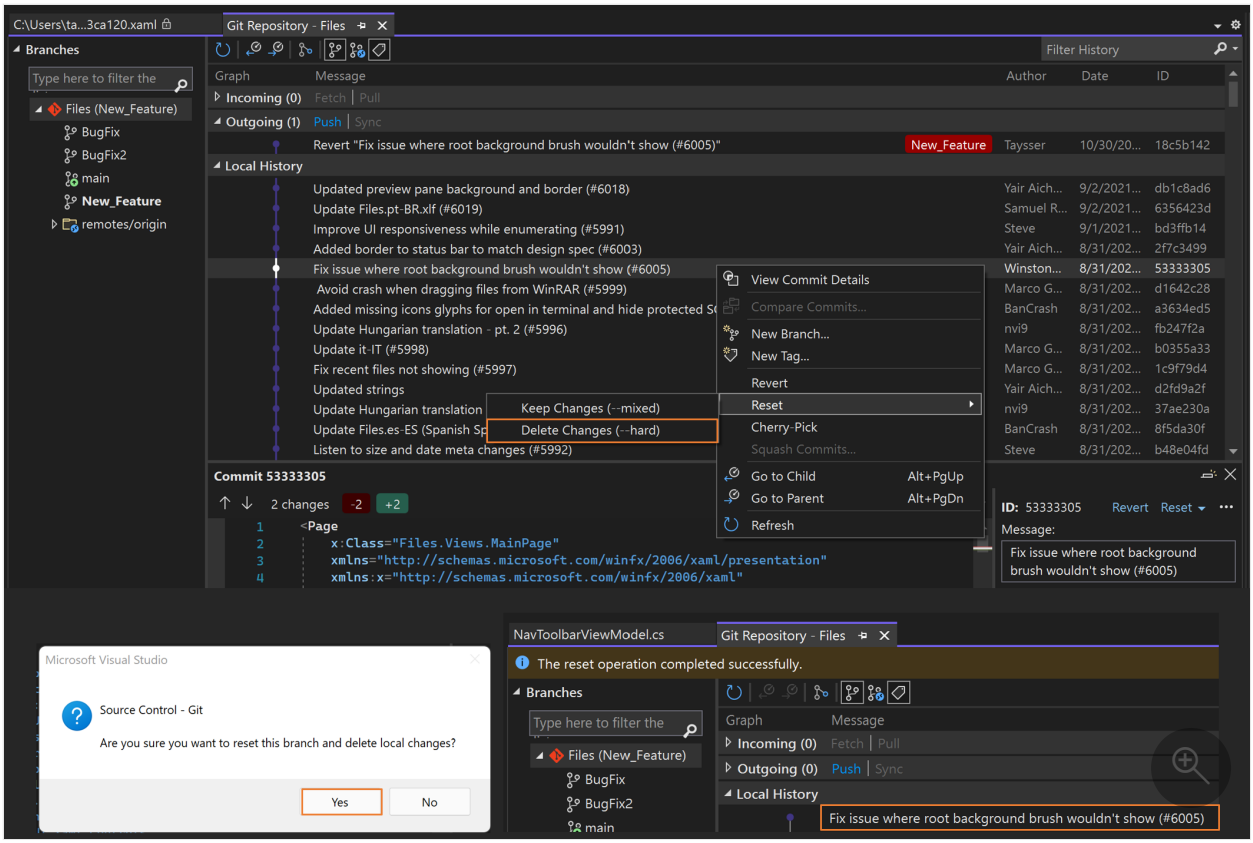
> **⚠ Warning**
>
> Don't reset shared branches, because you might delete other people's work. Use the revert command instead.

To reset a branch to a previous state by using the command line, use the following command. Replace the example ID with the ID of a real commit in your branch.

Bash

```
git reset --hard 53333305
```

The `--hard` part of the command tells Git to reset the files to the state of the previous commit and discard any staged changes. To do the same in Visual Studio, right-click the commit that you want to reset your branch to, and then select **Reset** > **Delete Changes (--hard)**.



To learn more about resetting branches, see the Git webpage for the reset command.

Next steps

To continue your journey, see Work with multiple repos.

See also

- The Git experience in Visual Studio
- Visual Studio and GitHub: Better together

Work with multiple repos

Article • 11/23/2022

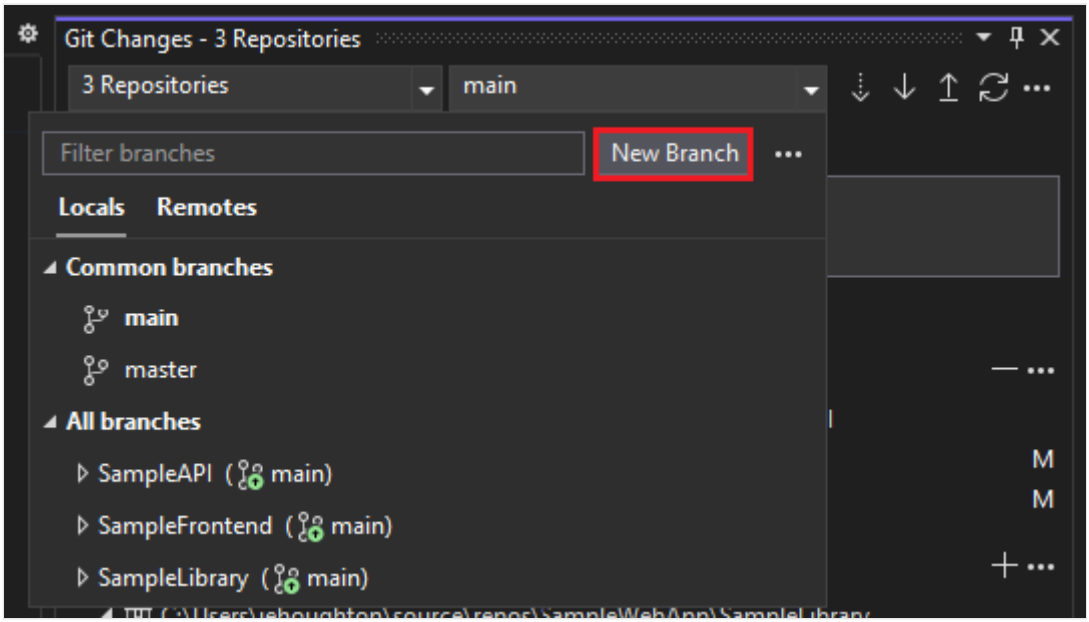
As one of our most highly requested features in the Developer Community, we've added multi-repository support to Visual Studio 2022, starting with version 17.4.

Now, you can have up to 10 active Git repositories at one time in Visual Studio. Multi-repo support means that you can work with a solution that spans more than one repository, and apply Git operations across several repositories at the same time. For example, in a large web project, you might need different repositories to manage work done on the frontend, the API, the database, the documentation, along with various libraries and dependencies. Before multi-repo support, you might have had to open multiple instances of Visual Studio to work between repos. Starting with version 17.4, you can manage, view, and debug them all with one instance of Visual Studio.

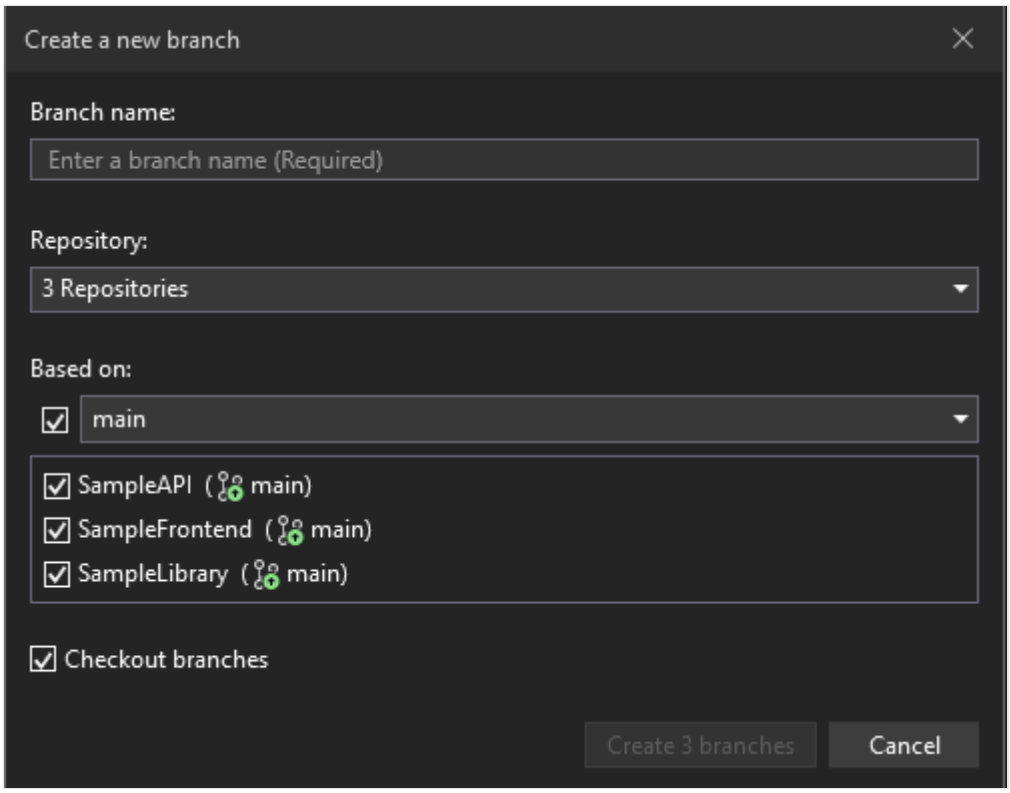
> **Note**
>
> To see multi-repository support in action, view the **Use multiple Git repositories in Visual Studio** video on YouTube.

Branch management and track changes

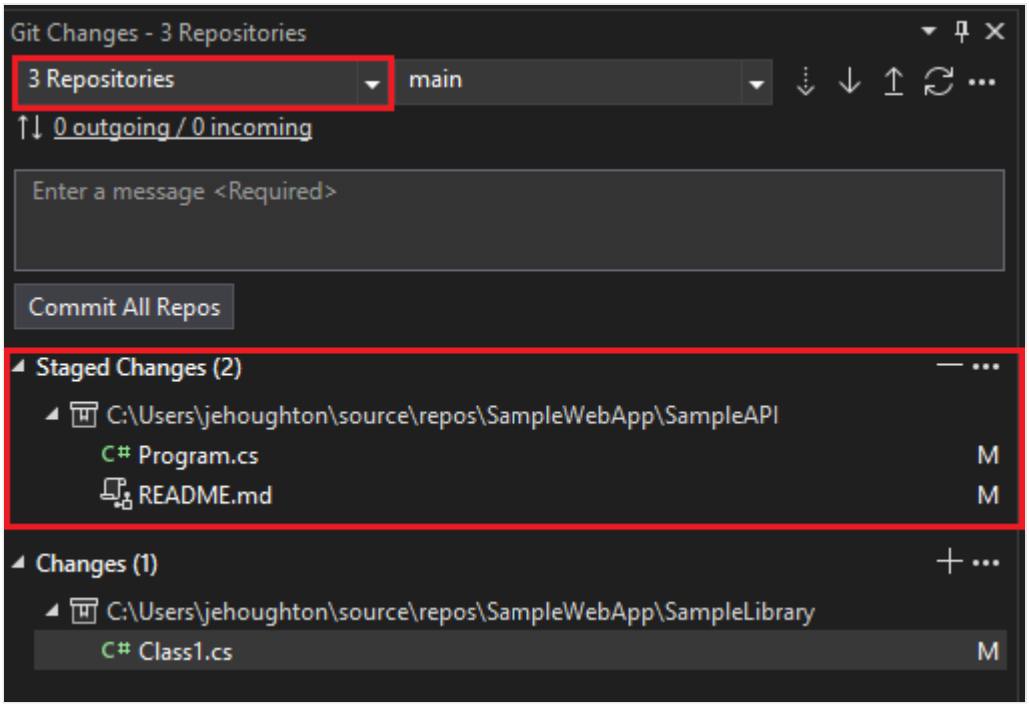
Multi-repository support extends the capabilities of the **Git Changes** and **Git Repository** windows. You can manage your multi-repo scenarios the same way as you would in working with a single repository.



You can create your branches across several repositories at one time by using the extended branch creation dialog.

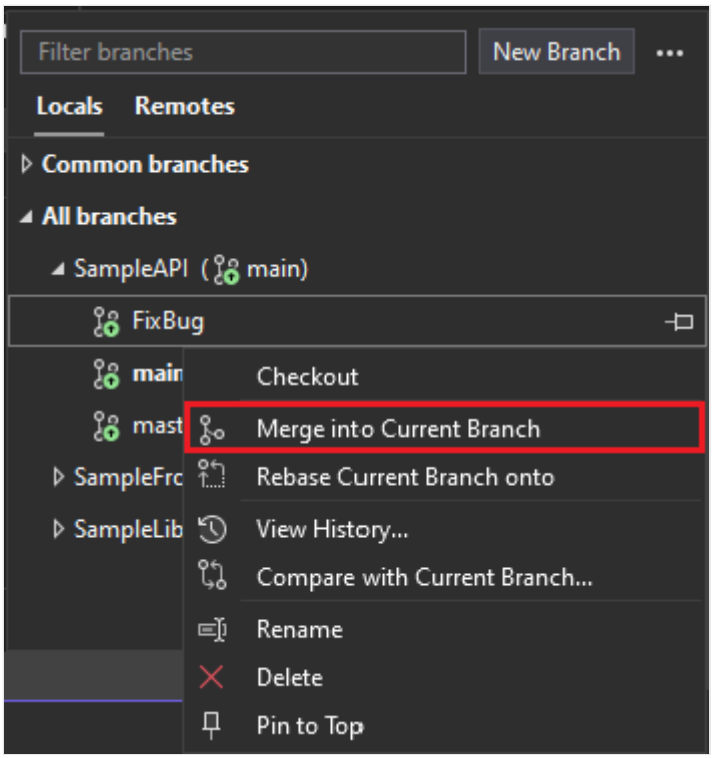


Then, as you make changes, the **Git Changes** window tracks your work separated by repository. You can stage and commit as you typically would.

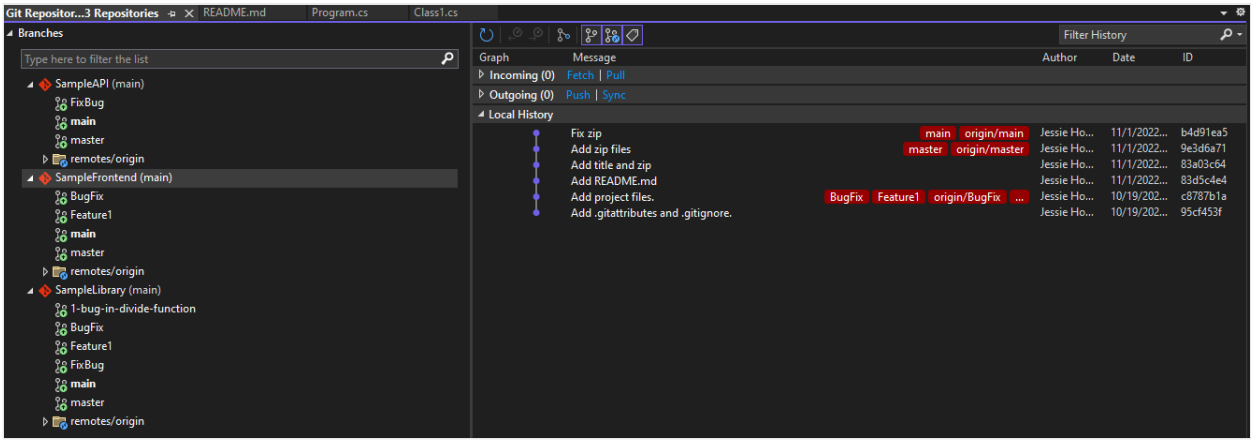


You can use the branch pickers in the status bar and the **Git Changes** tool window to switch branches. All you need to do is select the branch you'd like to check out. Also, you can perform inner loop branching operations like merging, rebasing, renaming,

deleting, and comparing branches by right-clicking the branch to open the context menu.

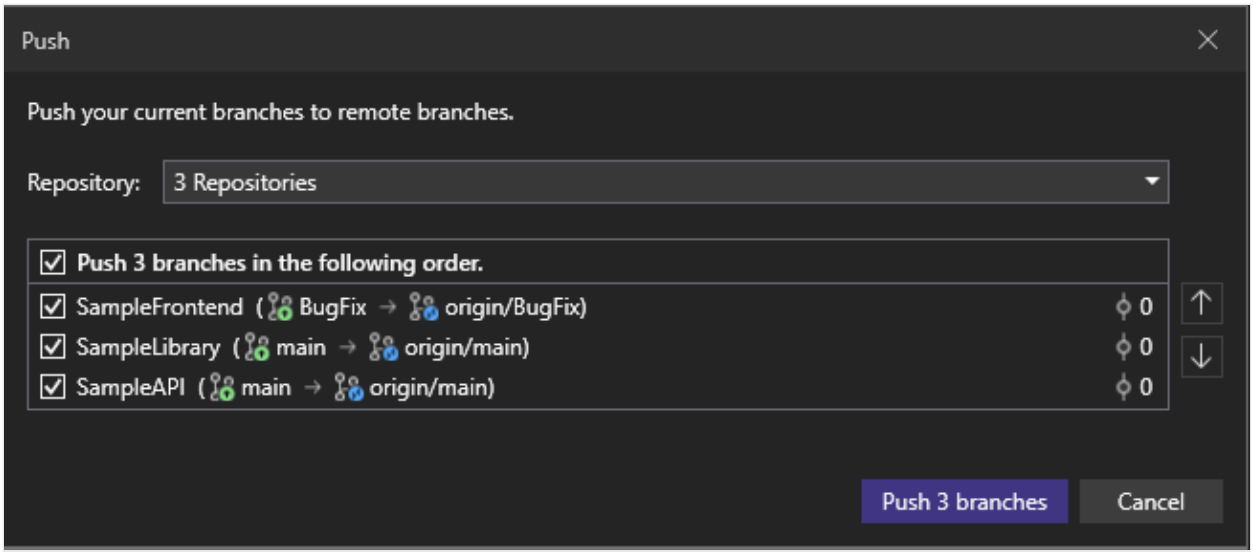


And, you can perform more advanced repository and branch management operations in the **Git Repository** window.



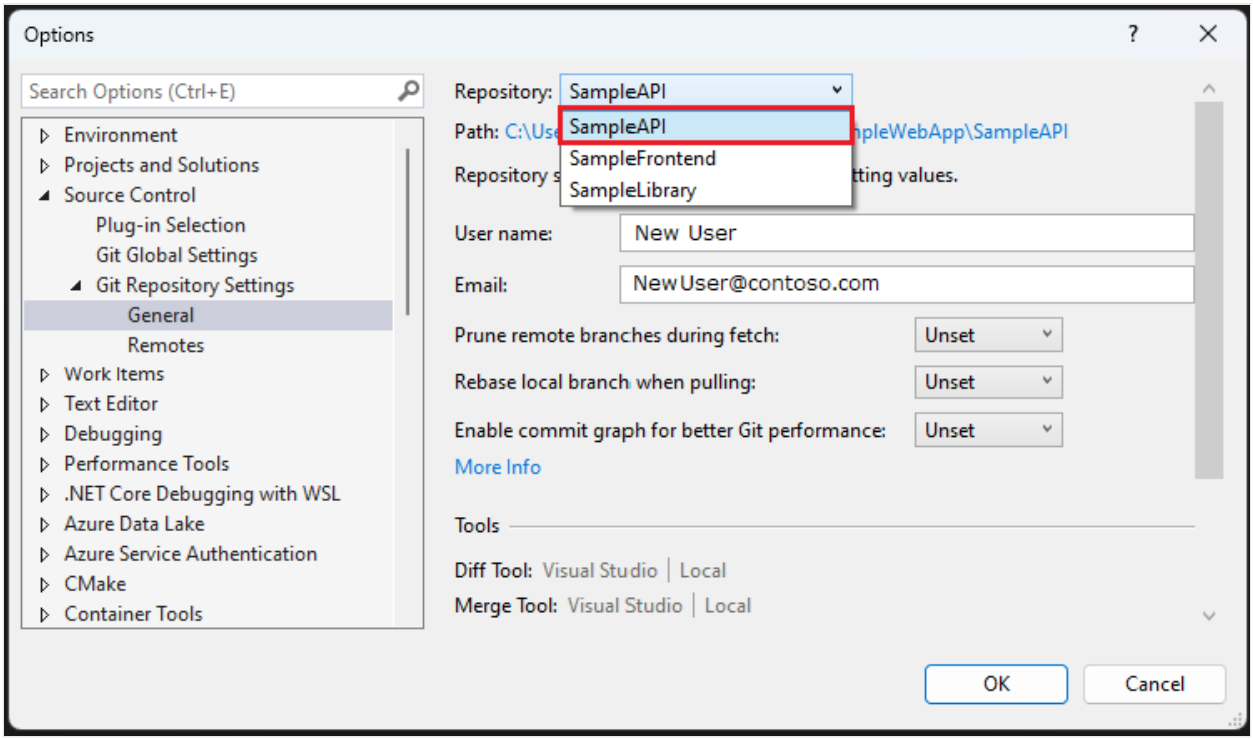
Network operations

Once you're ready to commit, the network operations dialog makes it easy to specify the exact branch you're pushing to and reorder your pushes. This same dialog allows you to exercise more control over other network operations, like fetch and pull.



Repository settings

You can use the **Git Repository Settings** page to specify your options for each repository. For example, you can specify whether to prune remote branches during fetch.



Activate multiple repos from a solution or a folder

You can activate multiple repositories from either a solution or a folder.

Use a solution

Here's how to create a solution with multiple repositories:

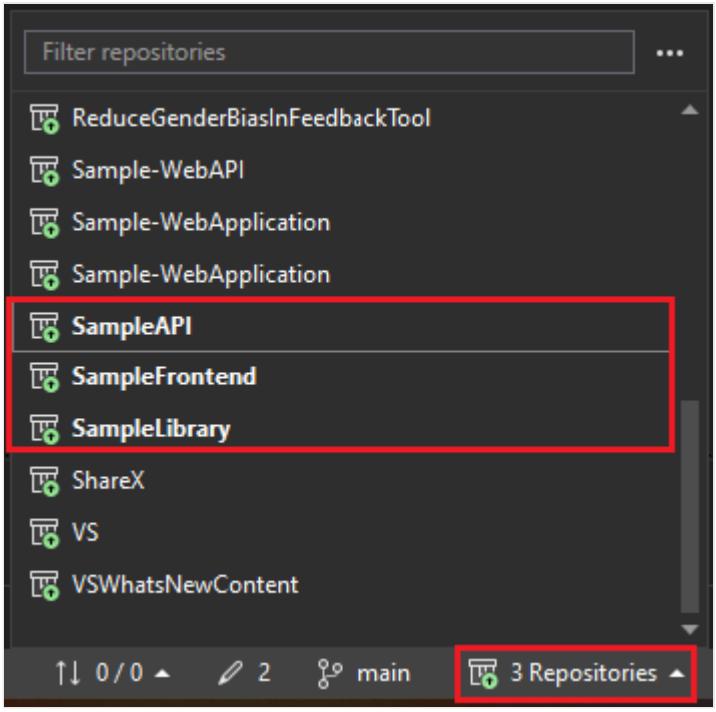
1. Open one of your existing solutions that already has a repository initialized.
2. Right-click the solution in **Solution Explorer**, and then select **Add** > **Existing Project**.
3. Select the .csproj file for another project that has a repository initialized.

   After the project's added, Visual Studio detects and activates the second repo. Then, you can adjust the Project References as necessary.

Use a folder

If the repositories you want to use are independent of each other and don't need to be part of the same solution, you might want to use a folder to open them.

To do so, you can place the local repo directories in the same parent folder and then use the **Open a local folder** option from the Visual Studio launch page to open the parent folder. Visual Studio detects and activates each repo, and then tracks changes by repo (as shown in the following screenshot).



Next steps

Want to know more about how we'll develop further Git functionality within Visual Studio 2022, or to share your feedback with us on the multi-repo feature? See the

"Building with your feedback", "What's next", and "Let us know what you think" sections of the Multi-repository support released blog post.

Next, to continue your journey with repo management, see Resolve merge conflicts in Visual Studio.

Resolve merge conflicts in Visual Studio

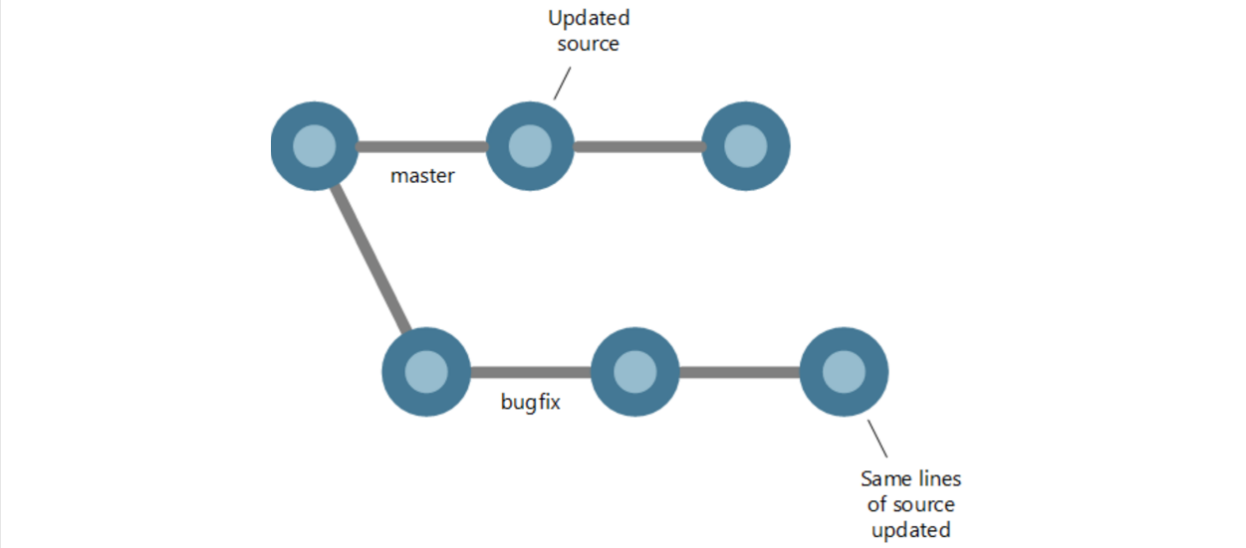
Article • 12/15/2022

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

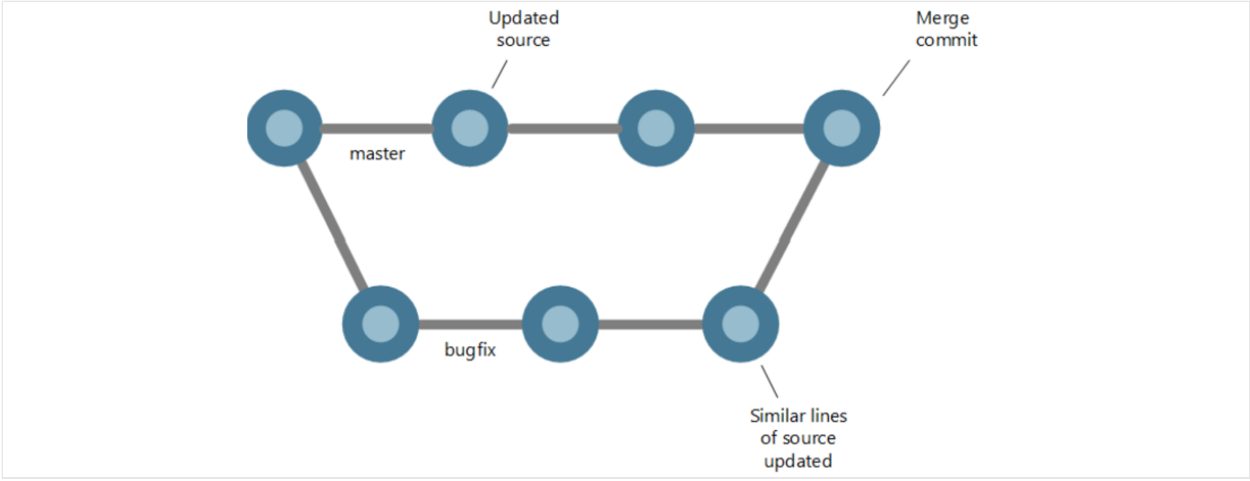
When you merge one branch into another, file changes from commits in one branch can conflict with the changes in the other. Git attempts to resolve these changes by using the history in your repo to determine what the merged files should look like. When it isn't clear how to merge changes, Git halts the merge and tells you which files conflict.

Understand merge conflicts

The following image shows a basic example of how changes conflict in Git. In this example, the main branch and the bugfix branch make updates to the same lines of source code.



If you try to merge the bugfix branch into main, Git can't determine which changes to use in the merged version. You might want to keep the changes in the main branch, the bugfix branch, or some combination of the two. Resolve this conflict with a merge commit in the main branch that reconciles the conflicting changes between the two branches.



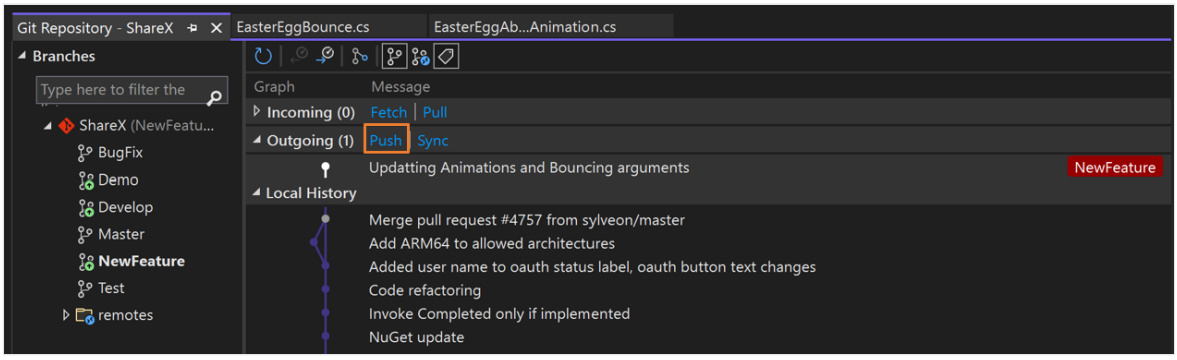
The most common merge conflict scenario occurs when you pull updates from a remote branch to your local branch (for example, from origin/bugfix into your local bugfix branch). You can resolve these conflicts in the same way: create a commit on your local branch to reconcile the changes, and then complete the merge.

Prevent merge conflicts

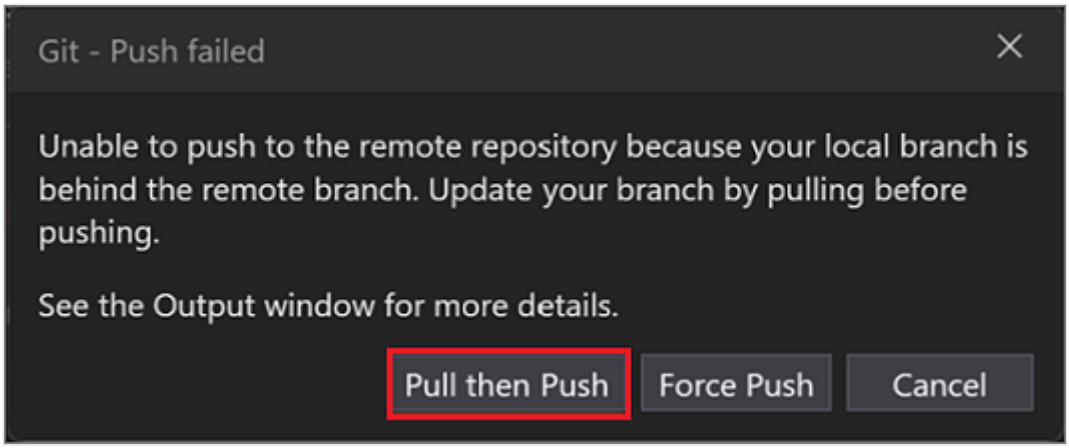
Git is good at automatically merging file changes in most circumstances, as long as the file contents don't change dramatically between commits. If your branch is far behind your main branch, consider rebasing your branches before you open a pull request. Rebased branches will merge into your main branch without conflicts.

Resolve merge conflicts

- If you're collaborating with others in the same branch, you might see merge conflicts when you push your changes.

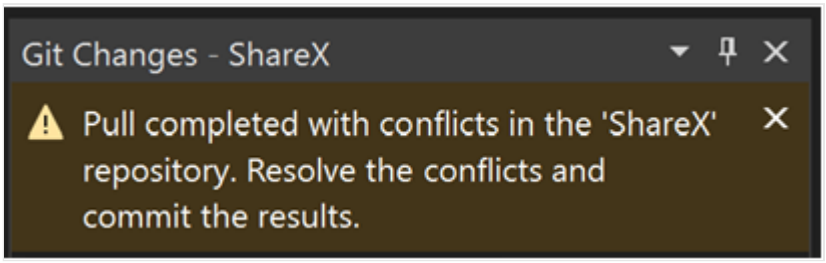


- Visual Studio detects if the local branch you've been working on is behind its remote tracking branch and then gives you options to choose from.

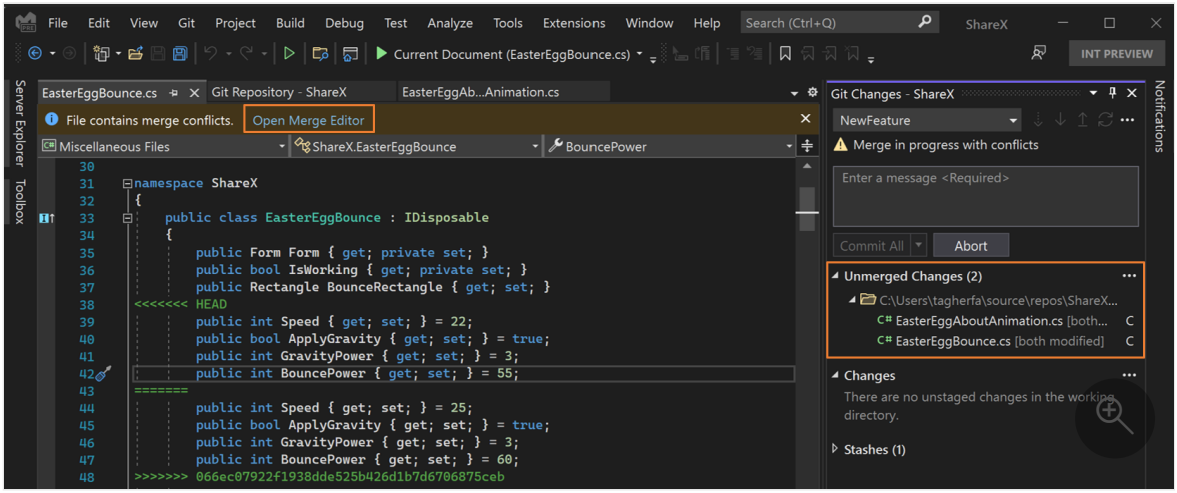


> **Note**
>
> If your remote repository supports **Force Push**, you can enable it by using **Git > Settings**.

In this example, select **Pull then Push** to include changes introduced to the remote repository. If there are any merge conflicts when you're pulling changes or trying to merge two branches, Visual Studio lets you know in the **Git Changes** window, in the **Git Repository** window, and on any files that have conflicts.

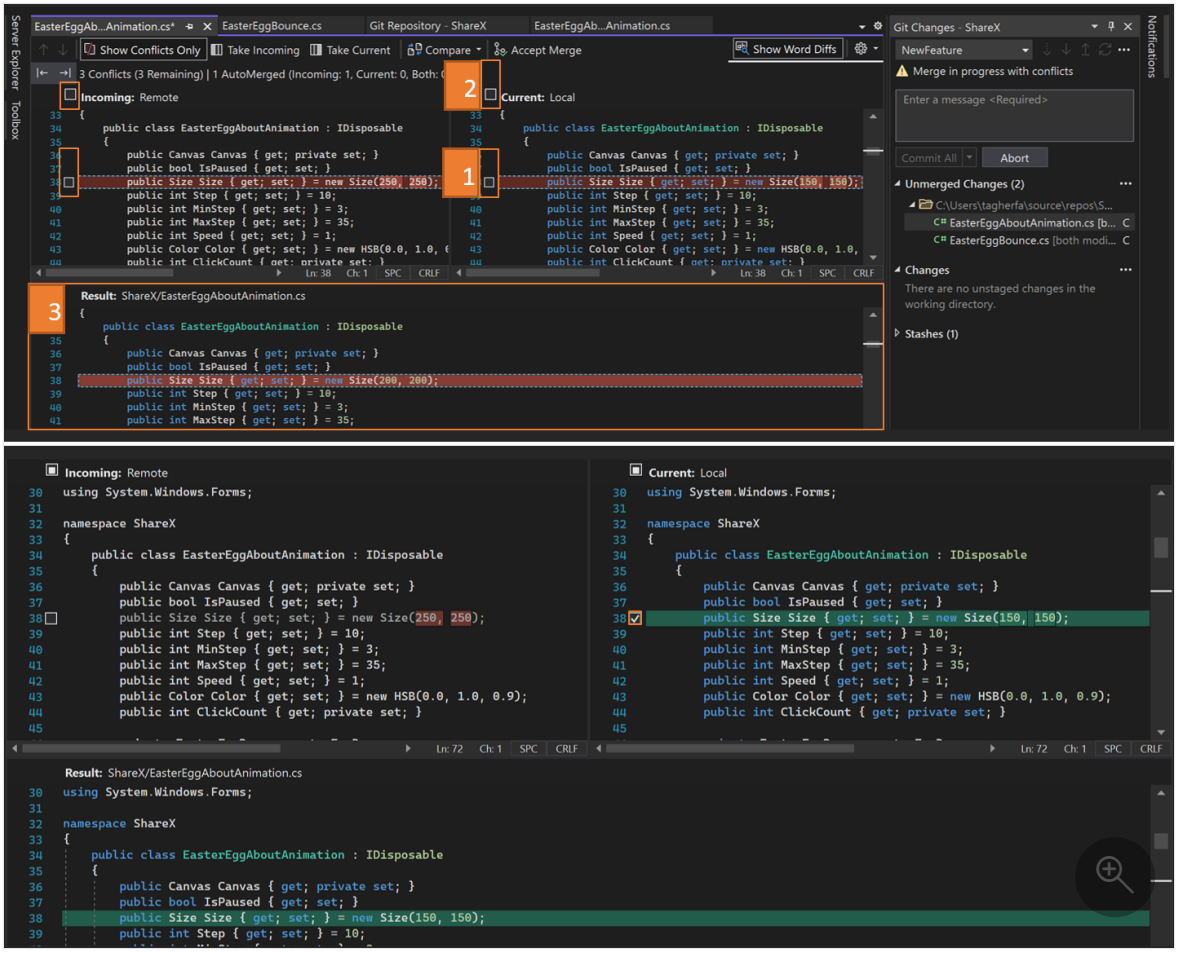


- The **Git Changes** window shows a list of files with conflicts under **Unmerged Changes**. To start resolving conflicts, double-click a file. Or if you have a file with conflicts opened in the editor, you can select **Open Merge Editor**.

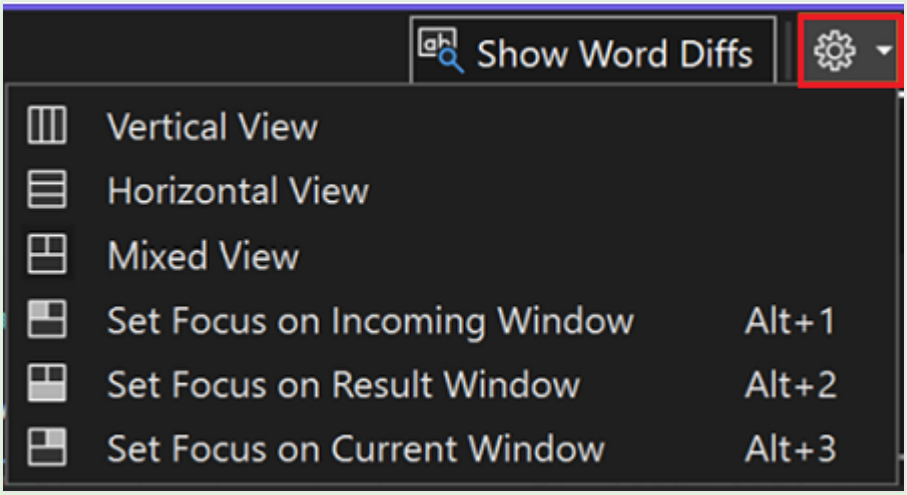


- In Merge Editor, start resolving your conflict by using any of the following methods (as depicted in the numbered screenshot):

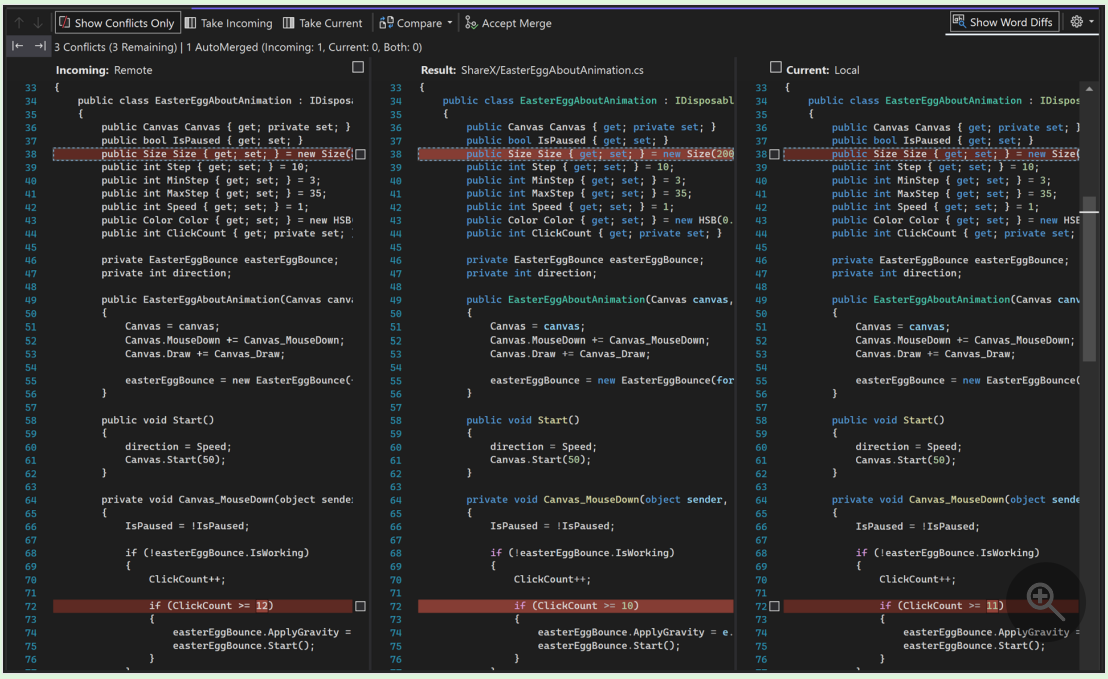
    1. Go over your conflicts line by line, and choose between keeping the right or the left side by selecting the checkboxes.
    2. Keep or ignore all of your conflicting changes.
    3. Manually edit your code in the **Result** window.



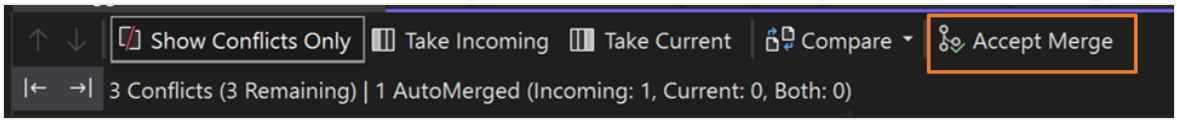
> **Tip**
>
> If you don't like the default layout in Merge Editor, feel free to change it by using the gear dropdown menu.



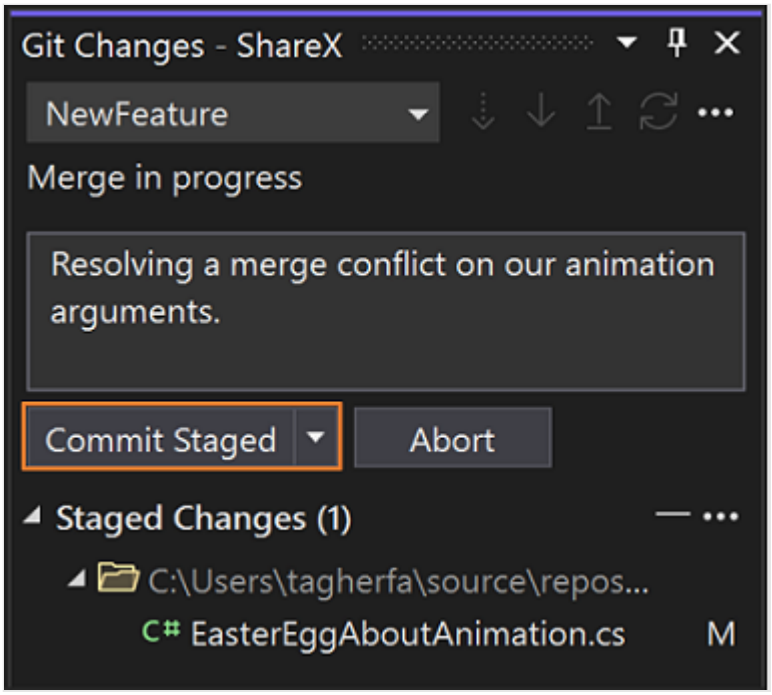
For example, the following screenshot shows what the vertical view looks like:



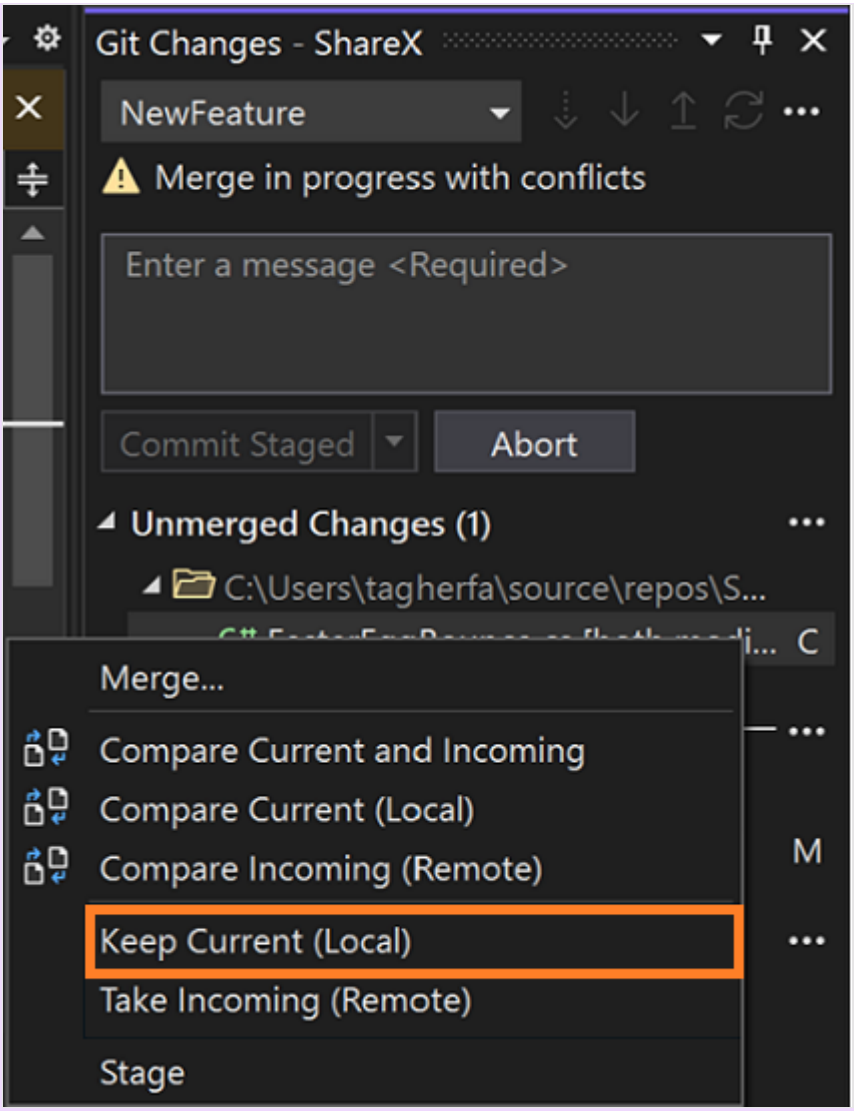
- When you're done resolving the merge conflicts, select **Accept Merge**. Repeat this process for all conflicting files.



- Use the **Git Changes** window to create a merge commit and resolve the conflict.



> **Note**
>
> If you need to keep all of your changes to a file, you can right-click it in the **Unmerged Changes** section and select **Keep Current (Local)** without having to open Merge Editor.



> **Tip**
>
> To learn more about the accessibility options available to you, see the **Git keyboard shortcuts** section of the **Accessibility tips and tricks for Visual Studio** page.

Next steps

To continue your journey and learn more about resolving conflicts, see the Git webpage for the merge command.

See also

- The Git experience in Visual Studio
- Visual Studio and GitHub: Better together

Git Changes window with no active repo set

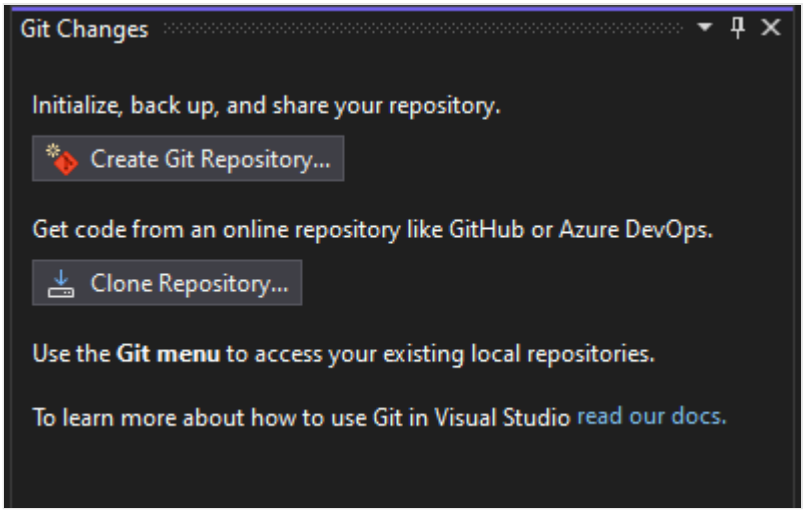
Article • 09/27/2023

Applies to: Visual Studio | Visual Studio for Mac | Visual Studio Code

If you issue a Git instruction in Visual Studio, such as trying to commit a change when there's no active repo set, you see the Git Changes window. From this window, you may choose to:

- Create Git repository, if you don't have an existing Git repo.
- Clone repository, if you have a Git repo hosted on a Git provider like GitHub.

If you're new to Git in Visual Studio, see How Visual Studio makes version control easy with Git or the https://git-scm.com/ website.



> **Important**
>
> If you have a suggestion for us, please let us know! We appreciate the opportunity to engage with you on design decisions via the **Developer Community** portal.

For additional information, see the following:

- Getting started with Git in Visual Studio video on YouTube
- Enhanced productivity with Git in Visual Studio blog post
- Options dialog box

Create a GitHub account to use with Visual Studio

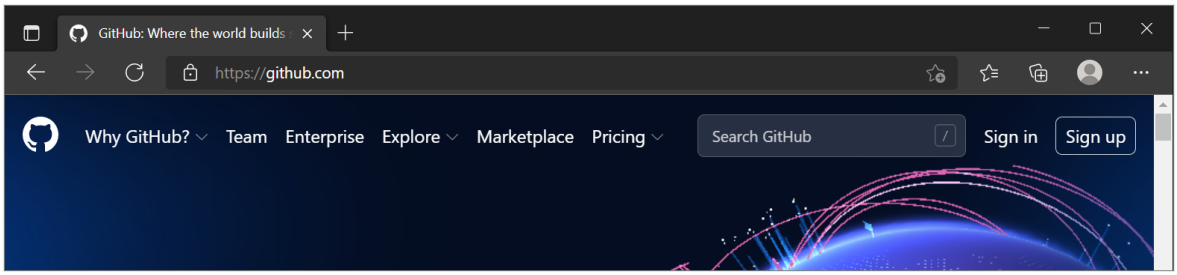
Article • 05/27/2022

Applies to: Visual Studio Visual Studio for Mac Visual Studio Code

With a GitHub account, you get full GitHub support from within the Visual Studio IDE to both manage your code and collaborate with others on development projects.

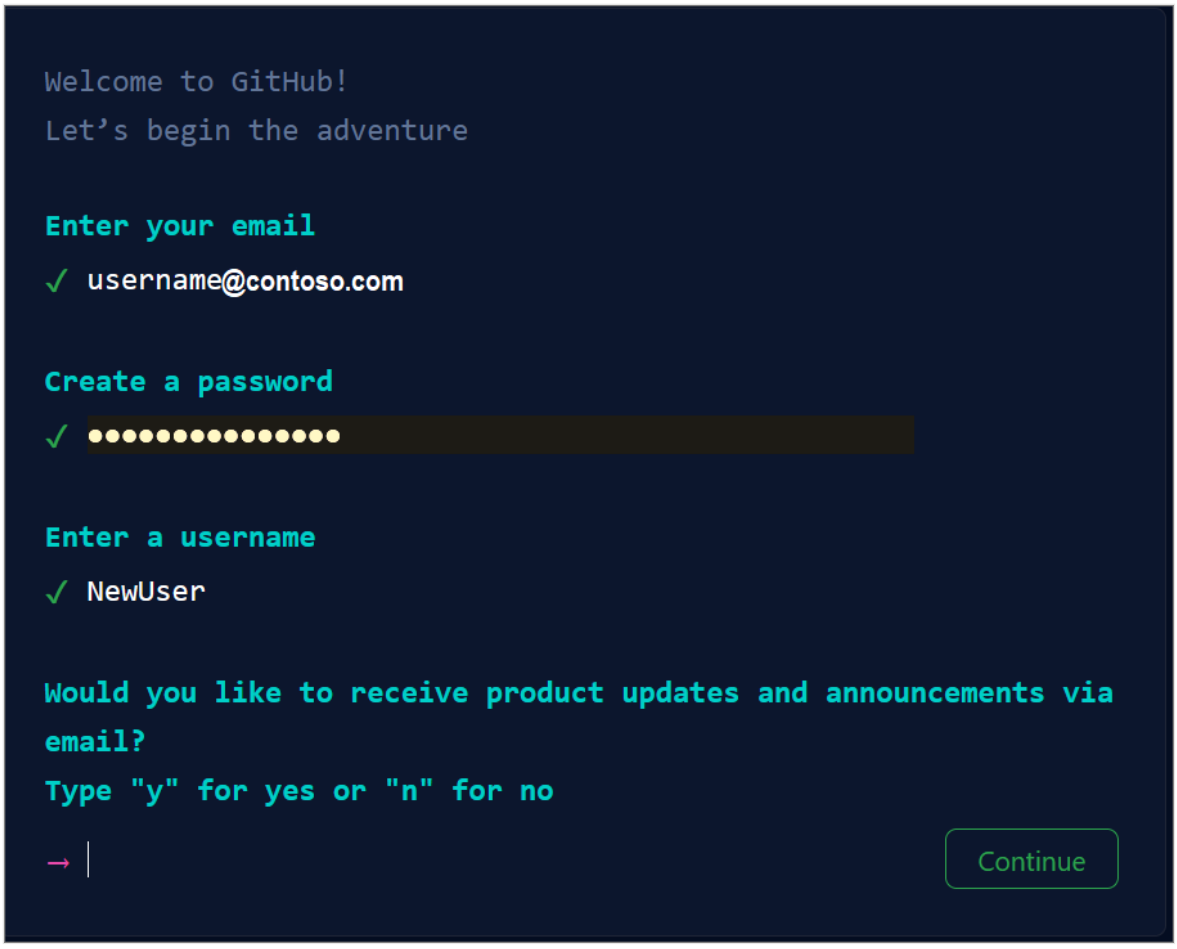
If you don't already have a GitHub account, here's how to create one.

1. Open https://github.com in a web browser, and then select **Sign up**.



2. **Enter your email** address.

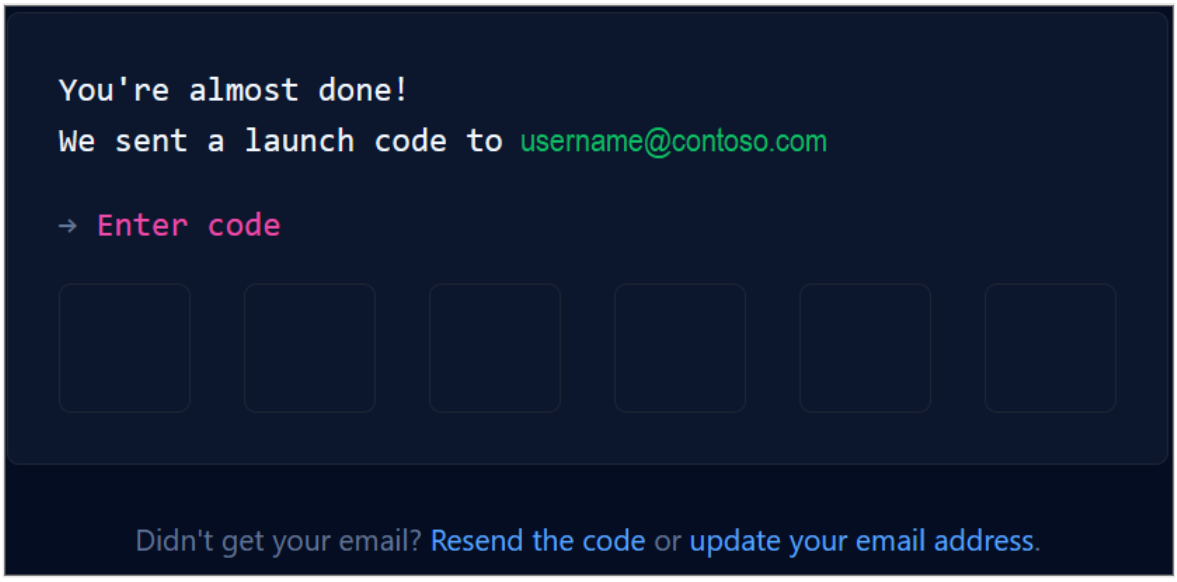
3. **Create a password** for your new GitHub account, and **Enter a username**, too. Next, choose whether you want to receive updates and announcements via email, and then select **Continue**.



4. **Verify your account** by solving a puzzle. Select the **Start Puzzle** button to do so, and then follow the prompts.

5. After you verify your account, select the **Create account** button.

6. Next, GitHub sends a launch code to your email address. Type that launch code in the **Enter code** dialog, and then press **Enter**.



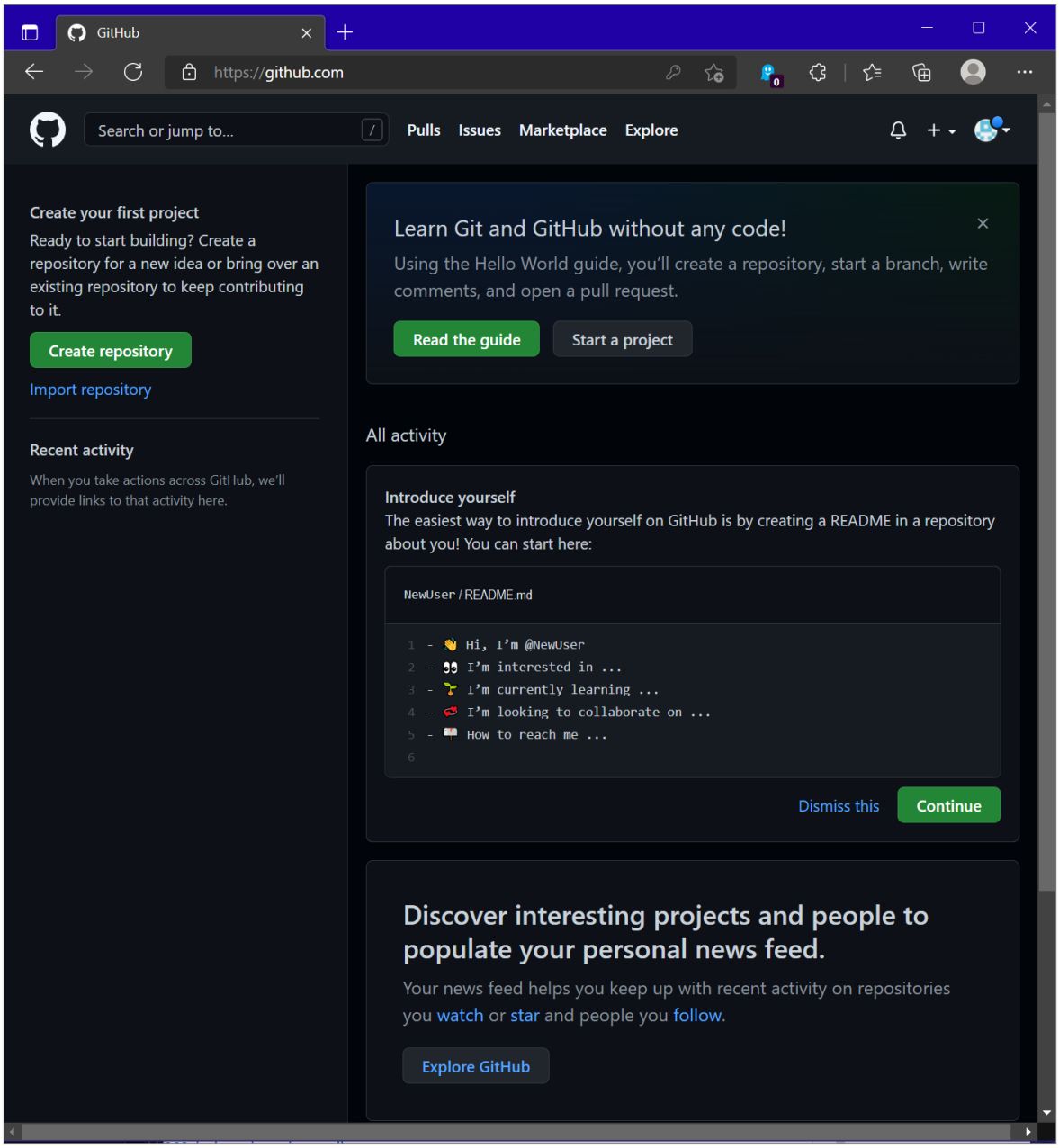
7. GitHub asks you some questions to help tailor your experience. Choose the answers that apply to you in the following dialogs:

- **How many team members will be working with you?**
- **What specific features are you interested in using?**

8. On the **Where teams collaborate and ship** screen, you can choose whether you want to use the Free account or the Team account. To choose the **Free** account, select the **Skip personalization** button.

> **Tip**
>
> You can always upgrade your account later. See the **Types of GitHub accounts** page to learn more.

GitHub opens a personalized page in your browser.



Congratulations! You've successfully created your GitHub account.

Next steps

To continue your journey, visit the Clone a repo page.

See also

- Tutorial: Open a project from a repo
- Visual Studio & GitHub: Better together